

Visual CeTZ

One picture per command or parameter, for CeTZ 0.5.2.

Adapted from Jean Pierre Casteleyn's fantastic work *Visual TikZ* [1], which does the same for TikZ. Its structure, section order and examples inspired every chapter here; the CeTZ code and pictures are new. Source: <https://github.com/RomeoV/VisualCeTZ>.

[1] Jean Pierre Casteleyn, *Visual TikZ*, version 0.65, 2018. [CTAN: visualtikz](#), [PDF](#). LaTeX Project Public License 1.3.

Contents

1	Paths and strokes	3
1.1	Basic figures	3
1.2	Paths	6
1.3	Strokes	8
1.4	Fills	10
2	Marks	12
2.1	Mark symbols	12
2.2	Mark parameters	13
2.3	Placement	18
3	Coordinates	19
3.1	Small pictures	19
3.2	Angles	21
3.3	Grid	22
3.4	Coordinate systems	23
4	Content and anchors	30
4.1	Creating content	30
4.2	Names and anchors	31
4.3	Content body	31
4.4	Behind or in front	32
4.5	Name prefixes	32
4.6	Links	32
4.7	Labels	34
4.8	Pins	34
4.9	Content along a path	35
4.10	Labels on links	35
4.11	Relative placement	36
4.12	Fitting	37
4.13	Circle through points	38
4.14	Matrices	38
4.15	Chains	41
5	Groups, layers, and styles	42
5.1	Transformations	42
5.2	Placing the canvas	43
5.3	Groups and scopes	46
5.4	Background	47
5.5	Colors	49
5.6	Opacity	50
5.7	Own commands	51
5.8	Own styles	52
5.9	Pictures in a canvas	52
5.10	Freehand drawing	53

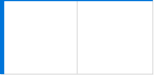
6	Shapes and text	53
6.1	Framed content	53
6.2	Built-in shapes	55
6.3	Shapes from lines	56
6.4	Symbol shapes	57
6.5	Arrow shapes	58
6.6	Callouts	58
6.7	Cross out and strike out	58
6.8	Multi-part content	59
6.9	Text attributes	60
6.10	Anchors	61
7	Decorations	62
7.1	Path morphing	62
7.2	Braces	66
7.3	Path replacing	67
7.4	Markings	67
7.5	Footprints	70
7.6	Shapes along a path	70
7.7	Text along a path	70
7.8	Fractals	71
7.9	Applications	71
8	Plots	73
8.1	Graphs from points and functions	74
8.2	Marks on data points	76
8.3	Graphs from gnuplot	78
8.4	Axes in the style of pgfplots	78
8.5	3D graphs	82
8.6	Data plots with lilaq	86
9	Loops and trees	87
9.1	Loops	87
9.2	Turtle graphics	88
9.3	Trees	88
10	Not ported	93

1 Paths and strokes

1.1 Basic figures

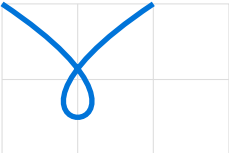
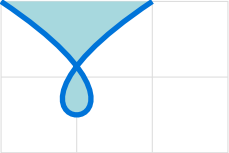
1.1.1 Straight and perpendicular lines

CeTZ manual: perpendicular

	<code>line((0, 0), (2, 1), stroke: 2pt + blue)</code>
	<code>line((0, 0), ((, "- ", (2, 1)), (2, 1), stroke: 2pt + blue)</code>
	<code>line((0, 0), ((, " -", (2, 1)), (2, 1), stroke: 2pt + blue)</code>

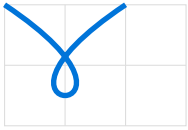
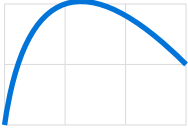
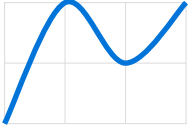
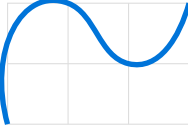
1.1.2 Stroke and fill

CeTZ manual: styling

<code>bezier((0, 2), (2, 2), (3, 0), (-1, 0), VALUE)</code>		
		
<code>stroke: 2pt + blue</code>	<code>fill: eastern.lighten(60%), stroke: none</code>	<code>fill: eastern.lighten(60%), stroke: 2pt + blue</code>

1.1.3 Bézier curves

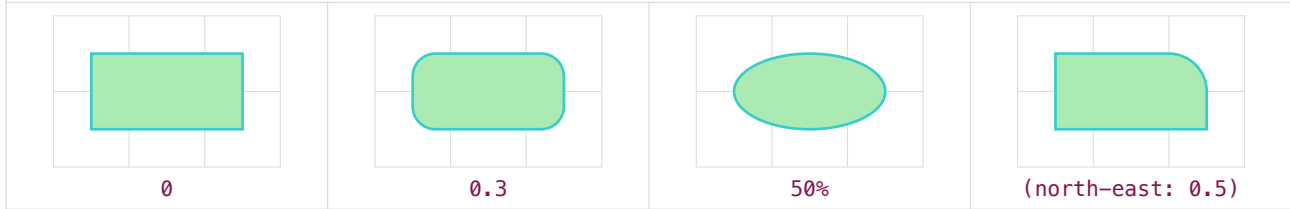
CeTZ manual: bezier

	<code>bezier((0, 2), (2, 2), (3, 0), stroke: 2pt + blue)</code>
	<code>bezier((0, 2), (2, 2), (3, 0), (-1, 0), stroke: 2pt + blue)</code>
	<code>bezier-through((0, 0), (1, 2), (3, 1), stroke: 2pt + blue)</code>
	<code>catmull((0, 0), (1, 2), (2, 1), (3, 2), stroke: 2pt + blue)</code>
	<code>hobby((0, 0), (1, 2), (2, 1), (3, 2), stroke: 2pt + blue)</code>

1.1.4 Rectangle

CeTZ manual: rect

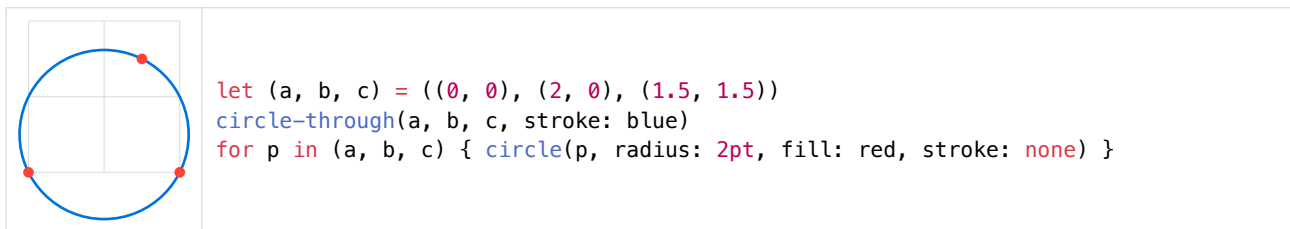
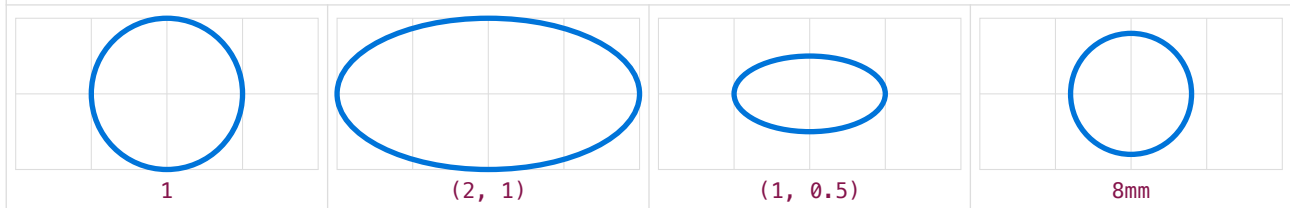
```
rect((0.5, 0.5), (rel: (2, 1)), radius: VALUE, fill: green.lighten(60%), stroke: teal)
```



1.1.5 Circle and ellipse

CeTZ manual: circle

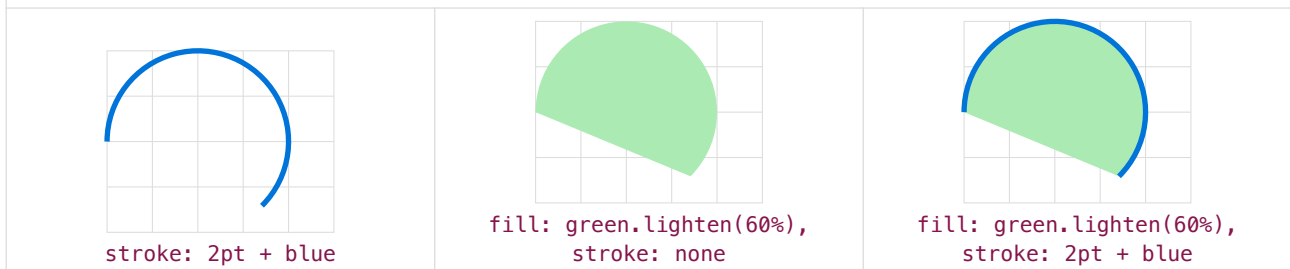
```
circle((1, 1), radius: VALUE, stroke: 2pt + blue)
```



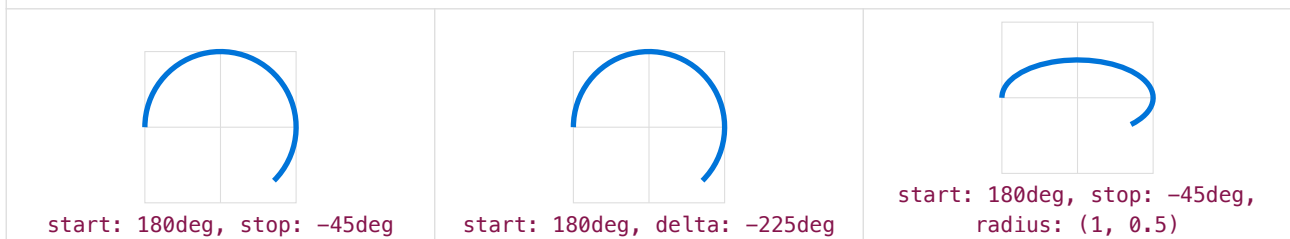
1.1.6 Arc

CeTZ manual: arc

```
arc((-2, 0), start: 180deg, stop: -45deg, radius: 2, VALUE)
```



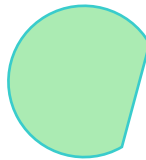
```
arc((0, 0), VALUE, stroke: 2pt + blue)
```



```
arc((0, 0), start: 30deg, stop: 300deg, mode: VALUE, fill: green.lighten(60%), stroke: teal)
```



"OPEN"



"CLOSE"



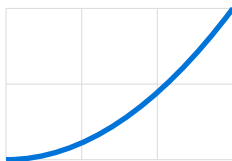
"PIE"

1.1.7 Parabola

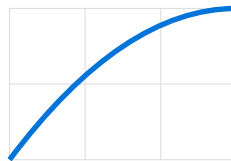
CeTZ manual: bezier

Not in CeTZ: parabola. A quadratic bezier is a parabola. Its control point sets the bend.

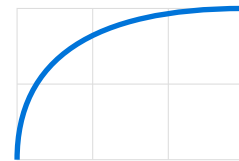
```
bezier((0, 0), (3, 2), VALUE, stroke: 2pt + blue)
```



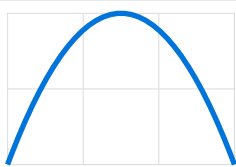
(1.5, 0)



(1.5, 2)



(0, 2)

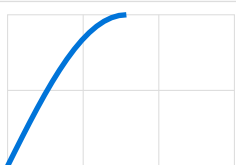


// vertex at (1.5, 2): control point at twice the height
bezier((0, 0), (3, 0), (1.5, 4), stroke: 2pt + blue)

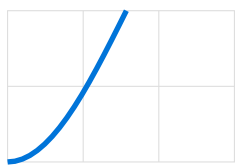
1.1.8 Sine and cosine

CeTZ manual: line

Not in CeTZ: sin and cos path operations. Sample the function into a line, or use a plot library (Section 8).



```
let xs = range(17).map(i => i * calc.pi / 32)  
line(..xs.map(x => (x, 2 * calc.sin(x))), stroke: 2pt + blue)
```

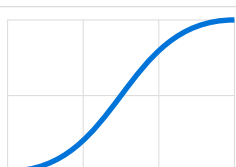


```
let xs = range(17).map(i => i * calc.pi / 32)  
line(..xs.map(x => (x, 2 - 2 * calc.cos(x))), stroke: 2pt + blue)
```

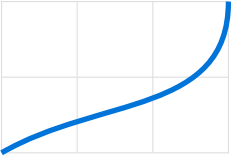
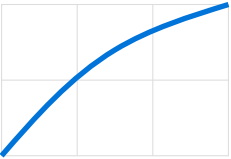
1.1.9 Curved connections

CeTZ manual: bezier

Not in CeTZ: to[out=..., in=...]. Give a cubic bezier control points that leave and enter at the wanted angles, or let hobby find the curve.

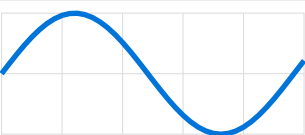


```
// out = 0deg, in = 180deg  
let (a, b) = ((0, 0), (3, 2))  
bezier(a, b, (rel: (1.5, 0), to: a), (rel: (-1.5, 0), to: b), stroke: 2pt + blue)
```

	<pre>// out = 30deg, in = -90deg let (a, b) = ((0, 0), (3, 2)) bezier(a, b, (rel: (30deg, 1.5), to: a), (rel: (0, -1.5), to: b), stroke: 2pt + blue)</pre>
	<pre>hobby((0, 0), (1.5, 1.4), (3, 2), stroke: 2pt + blue)</pre>

1.1.10 Point lists

CeTZ manual: line

	<pre>line((2, 0), (3, 1), (4, 1), (5, 2), stroke: 2pt + blue)</pre>
	<pre>let data = csv(bytes("0,0\n1,1\n1.5,0.5\n2,2\n3,1.5")) line(..data.map(r => r.map(float)), stroke: 2pt + blue)</pre>
	<pre>let xs = range(41).map(i => i / 8) line(..xs.map(x => (x, 1 + calc.sin(1.3 * x))), stroke: 2pt + blue)</pre>

1.2 Paths

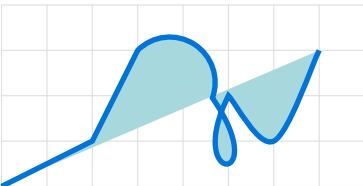
1.2.1 Open and closed

CeTZ manual: line

<pre>line((0, 0), (2, 1), (3, 0), close: VALUE, stroke: 2pt + blue)</pre>	
	false
	true
	<pre>line((0, 0), (2, 1), ((), "- ", (0, 0)), close: true, stroke: 2pt + blue)</pre>
	<pre>line((0, 0), (2, 1), ((), " -", (0, 0)), close: true, stroke: 2pt + blue)</pre>

1.2.2 Mixed segments

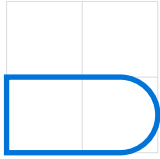
CeTZ manual: merge-path

	<pre>merge-path(fill: eastern.lighten(60%), stroke: 2pt + blue, { line((0, 0), (2, 1), (3, 3)) arc((), start: 135deg, stop: -20deg, radius: 1) bezier((), (5, 2), (6, 0), (4, 0)) catmull((), (6, 1), (7, 3)) })</pre>
---	--

1.2.3 Rounded corners

CeTZ manual: rect

Not in CeTZ: rounded corners on arbitrary paths. rect takes radius (see Rectangle). For other outlines, build each corner from arc segments inside merge-path.



```
merge-path(close: true, stroke: 2pt + blue, {
  line((0, 0), (1.5, 0))
  arc(), start: -90deg, stop: 90deg, radius: 0.5)
  line((0, 1))
})
```

1.2.4 Repeated elements

CeTZ manual: circle

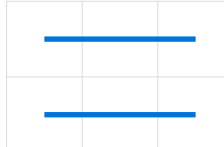
Not in CeTZ: insert path. Loop over the points.



```
let pts = ((0, 0), (1, 2), (3, 1))
line(..pts, stroke: gray)
for p in pts { circle(p, radius: 3pt, stroke: red) }
```

1.2.5 Interrupted paths

CeTZ manual: compound-path

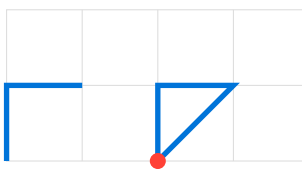


```
compound-path(stroke: 2pt + blue, {
  line((0.5, 0.5), (2.5, 0.5))
  line((0.5, 1.5), (2.5, 1.5))
})
```

1.2.6 Subpath start

CeTZ manual: path anchors

Not in CeTZ: current subpath start. Name the element and use its start anchor.

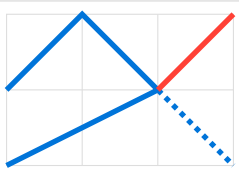


```
line((0, 0), (0, 1), (1, 1), stroke: 2pt + blue)
line((2, 0), (2, 1), (3, 1), close: true, stroke: 2pt + blue, name: "t")
circle("t.start", radius: 3pt, fill: red, stroke: none)
```

1.2.7 Edges

CeTZ manual: line

Not in CeTZ: edge. Draw each branch as its own element from the shared point.



```
let v = (2, 1)
line((0, 0), v, (1, 2), (0, 1), stroke: 2pt + blue)
line(v, (3, 0), stroke: (paint: blue, thickness: 2pt, dash: "dotted"))
line(v, (3, 2), stroke: 2pt + red)
```

1.3 Strokes

1.3.1 Line width

CeTZ manual: stroke

```
line((0, 0), (1, 1), stroke: VALUE + blue)
```

 0.1pt	 0.2pt	 0.4pt	 0.6pt
 0.8pt	 1.2pt	 1.6pt	 0.2cm

Not in CeTZ: named widths such as `thin` or `ultra thin` to `ultra thick`. Give a length. TikZ's `ultra thin` to `ultra thick` are the first seven values above.

1.3.2 Units

CeTZ manual: stroke

```
line((0, 0), (0, 1), stroke: (thickness: VALUE, paint: blue))
```

 10pt	 10mm	 0.5in	 1em	 0.3
---	---	--	---	--

Typst's `pt` is TikZ's `bp`. A plain number is in canvas units. Typst has no `ex`.

1.3.3 Caps

CeTZ manual: stroke

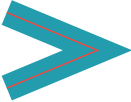
```
line((0, 0), (1.2, 0), stroke: (thickness: 12pt, cap: VALUE, paint: eastern))
line((0, 0), (1.2, 0), stroke: 0.5pt + red)
```

 "butt"	 "square"	 "round"
---	---	--

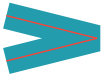
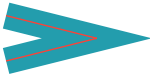
1.3.4 Joins

CeTZ manual: stroke

```
line((0, 0), (1.2, 0.5), (0, 1), stroke: (thickness: 10pt, join: VALUE, paint: eastern))
line((0, 0), (1.2, 0.5), (0, 1), stroke: 0.5pt + red)
```

 "miter"	 "bevel"	 "round"
--	--	--

```
line((0, 0), (1.2, 0.3), (0, 0.6), stroke: (thickness: 10pt, miter-limit: VALUE, paint: eastern))
line((0, 0), (1.2, 0.3), (0, 0.6), stroke: 0.5pt + red)
```

 4	 5
--	--

The default `miter-limit` is 4: this 28° corner needs about 4.1.

1.3.5 Dash styles

CeTZ manual: stroke

<code>line((0, 0), (2, 0.5), stroke: (thickness: 2pt, paint: blue, dash: VALUE))</code>		
 "dotted"	 "densely-dotted"	 "loosely-dotted"
 "dashed"	 "densely-dashed"	 "loosely-dashed"
 "dash-dotted"	 "densely-dash-dotted"	 "loosely-dash-dotted"
 "solid"		

Not in CeTZ: dash dot dot. Give the pattern as an array, as below.

<code>line((0, 0), (4, 0), stroke: (thickness: 2pt, paint: blue, dash: VALUE))</code>	
	<code>(1cm, 0.25cm, 0.25cm, 0.5cm)</code>
	<code>(array: (1cm, 0.25cm, 0.25cm, 0.5cm), phase: 1cm)</code>
	<code>(4pt, 2pt, "dot", 2pt, "dot", 2pt)</code>

1.3.6 Double lines

CeTZ manual: stroke

Not in CeTZ: double. Stroke the path twice: wide, then narrow in the inner colour.

	<pre>for s in (6pt + blue, 2pt + white) { line((0, 0), (2, 1), stroke: s) }</pre>
	<pre>for s in (6pt + blue, 2pt + red) { line((0, 0), (2, 1), stroke: s) }</pre>

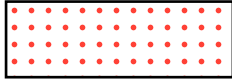
1.4 Fills

1.4.1 Tilings

CeTZ manual: fill

Any Typst `tiling` can fill a shape.

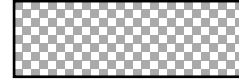
```
rect((0, 0), (3, 1), fill: tiling(size: (6pt, 6pt), VALUE))
```



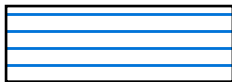
```
place(dx: 2pt, dy: 2pt,
std.circle(radius: 1pt, fill:
red))
```



```
text(5pt, red, sym.star.filled)
```



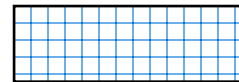
```
place(square(size: 3pt, fill:
gray)) + place(dx: 3pt, dy:
3pt, square(size: 3pt, fill:
gray))
```



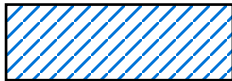
```
std.line(start: (0pt, 3pt),
length: 6pt, stroke: blue)
```



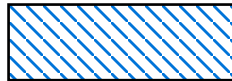
```
std.line(start: (3pt, 0pt),
angle: 90deg, length: 6pt,
stroke: blue)
```



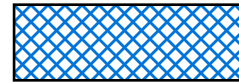
```
std.rect(width: 6pt, height:
6pt, stroke: 0.4pt + blue)
```



```
std.line(start: (0pt, 6pt),
end: (6pt, 0pt), stroke: blue)
```



```
std.line(end: (6pt, 6pt),
stroke: blue)
```

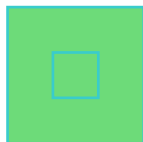


```
place(std.line(end: (6pt, 6pt),
stroke: blue)) +
std.line(start: (0pt, 6pt),
end: (6pt, 0pt), stroke: blue)
```

1.4.2 Fill rule

CeTZ manual: styling

```
compound-path(fill-rule: "non-zero", fill: green.lighten(30%), stroke: teal, {
line((0, 0), (0, 3), (3, 3), (3, 0), close: true)
line(VALUE, close: true)
})
```



```
(1, 1), (1, 2), (2, 2), (2, 1)
```



```
(1, 1), (2, 1), (2, 2), (1, 2)
```

```
compound-path(fill-rule: VALUE, fill: green.lighten(30%), stroke: teal, {
line((0, 0), (2, 1), (1, 2), close: true)
circle((1, 2), radius: 0.5)
})
```



"non-zero"



"even-odd"

1.4.3 Filling with an image

CeTZ manual: fill

Not in CeTZ: path picture. Fill with a tiling whose single tile is the image, placed relative to the shape.

```
let pic = tiling(relative: "self", size: (2cm, 2cm),
  image("/assets/picture.png", width: 2cm, height: 2cm, fit: "cover"))
VALUE
```



```
circle((1, 1), radius: 1, fill:
  pic)
```



```
line((1, 0), (2, 1), (1, 2),
  (0, 1), close: true, fill: pic)
```



```
bezier((0, 0), (2, 0), (1, 4),
  fill: pic)
```

1.4.4 Shading

CeTZ manual: fill

Any Typst `gradient` can fill a shape.

```
rect((0, 0), (3, 1), VALUE)
```



```
fill: gradient.linear(gray, white)
```



```
fill: gradient.linear(gray, white), stroke:
  none
```

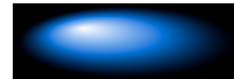
```
rect((0, 0), (3, 1), fill: VALUE)
```



```
gradient.linear(gray, white)
```



```
gradient.radial(gray, white)
```



```
gradient.radial(white, blue,
  black, focal-center: (30%,
    30%))
```



```
gradient.linear(red, white)
```



```
gradient.linear(white, green)
```



```
gradient.linear(red, green)
```



```
gradient.linear(red, white,
  angle: 90deg)
```



```
gradient.linear(white, green,
  angle: 90deg)
```



```
gradient.linear(gray, red,
  gray, angle: 90deg)
```



```
gradient.linear(red, green,
  angle: 45deg)
```



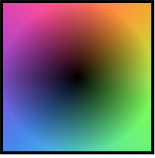
```
gradient.radial(red, white)
```



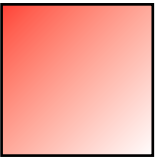
```
gradient.radial(red, green)
```



```
rect((0, 0), (2, 2), fill: gradient.conic(
  .color.map.rainbow))
```

	<pre>rect((0, 0), (2, 2), fill: gradient.conic(..color.map.rainbow)) rect((0, 0), (2, 2), fill: gradient.radial(black, black.transparentize(100%)))</pre>
	<pre>rect((0, 0), (2, 2), fill: gradient.conic(..color.map.rainbow)) rect((0, 0), (2, 2), fill: gradient.radial(white, white.transparentize(100%)))</pre>

Not in CeTZ: four-corner shadings (upper left, ...) and Mandelbrot set. Typst gradients are linear, radial or conic. For one coloured corner, a diagonal gradient comes close.

	<pre>rect((0, 0), (2, 2), fill: gradient.linear(red, white, angle: 45deg))</pre>
---	--

2 Marks

2.1 Mark symbols

2.1.1 Start, end, both ends

CeTZ manual: marks: symbol, start, end

<pre>line((0, 0), (1.5, .8), stroke: 1pt + blue, mark: VALUE)</pre>			
 (end: ">")	 (start: ">")	 (symbol: ">")	 (start: "<", end: ">")
 (end: "straight")	 (end: "o")	 (end: " ")	 (end: ">>")

2.1.2 Symbols

CeTZ manual: marks

<pre>line((0, 0), (1.4, .8), stroke: 1.5pt + blue, mark: (end: VALUE, scale: 2, fill: eastern.lighten(60%)))</pre>			
 "triangle"	 "stealth"	 "curved-stealth"	 "bar"
 "ellipse"	 "circle"	 "bracket"	 "diamond"
 "rect"	 "hook"	 "straight"	 "barbed"
 "plus"	 "x"	 "star"	 "parenthesis"

Shorthands:

> triangle	< triangle reversed	<> diamond	[rect	] bracket	[bracket reversed	 bar
o circle	+ plus	x x	* star	)> curved-stealth	>> stealth	) parenthesis

arrows.meta tips by closest CeTZ shape:

Triangle, Latex	">"	Stealth	">>"
Straight Barb	"straight"	Classical TikZ Rightarrow	"barbed"
Arc Barb, Parenthesis	")"	Bar	" "
Bracket, Tee Barb	"]"	Computer Modern Rightarrow, To	"barbed"
Hooks	"hook"	Circle	"o"
Ellipse	"ellipse"	Diamond, Kite, Turned Square	"<>"
Square, Rectangle	"[]"	Rays	"+", "x", "*"

Not in CeTZ: Cap tips (Butt Cap, Round Cap, Triangle Cap, Fast Round, Fast Triangle, cap angle). Set the stroke cap of the path, see Section 1.

Not in CeTZ: Implies and double lines. Stroke the path twice, see Section 1.

Not in CeTZ: Rays [n=...]. Use "+", "x" or "*", or draw your own with register-mark (Section 2.3.5).

2.1.3 Several marks

CeTZ manual: marks: symbol

	<pre>line((0, 0), (3, 1), stroke: 1.5pt + blue, mark: (start: ">", end: "o", scale: 2, fill: eastern.lighten(60%)))</pre>
	<pre>line((0, 0), (3, 1), stroke: 1.5pt + blue, mark: (end: ("o", ">"), scale: 2, fill: eastern.lighten(60%)))</pre>
	<pre>line((0, 0), (3, 1), stroke: 1.5pt + blue, mark: (scale: 2, end: ((symbol: "o", sep: .3, fill: red), ">", ">")))</pre>

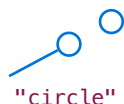
2.2 Mark parameters

2.2.1 Distance between marks: sep

CeTZ manual: marks: sep

<pre>line((0, 0), (1.5, .8), stroke: 1pt + blue, mark: (end: (">", ">"), sep: VALUE, scale: 2))</pre>				
 -0.1	 0	 0.1	 0.3	
<pre>line((0, 0), (1.5, .8), stroke: 1pt + blue, mark: (end: (VALUE, VALUE), sep: .15, scale: 2))</pre>				
 "straight"	 "bracket"	 "hook"	 "parenthesis"	

```
line((0, 0), (1.5, .8), stroke: 1pt + blue,
mark: (end: (VALUE, VALUE), sep: .15, scale: 2))
```



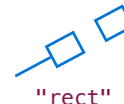
"circle"



"diamond"



"stealth"

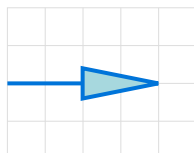


"rect"

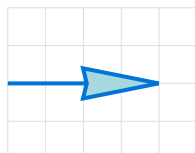
2.2.2 Length

CeTZ manual: marks: length

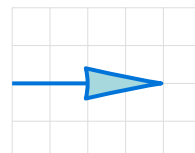
```
line((0, 0), (2, 0), stroke: 1.5pt + blue,
mark: (end: VALUE, length: 1, width: .4, fill: eastern.lighten(60%)))
```



"triangle"



"stealth"



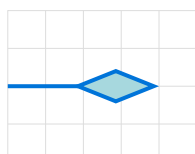
"curved-stealth"



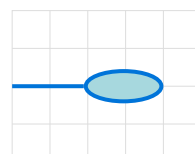
"straight"



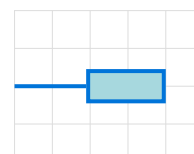
"barbed"



"diamond"



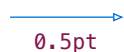
"ellipse"



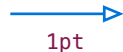
"rect"

A ratio is relative to the stroke thickness. This also holds for width, inset and sep.

```
line((0, 0), (1.5, 0), stroke: VALUE + blue,
mark: (end: ">", length: 600%, width: 500%))
```



0.5pt



1pt



2pt

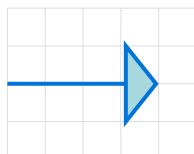


4pt

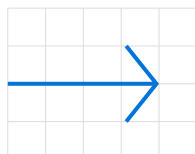
2.2.3 Width

CeTZ manual: marks: width

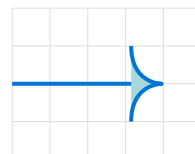
```
line((0, 0), (2, 0), stroke: 1.5pt + blue,
mark: (end: VALUE, length: .4, width: 1, fill: eastern.lighten(60%)))
```



"triangle"



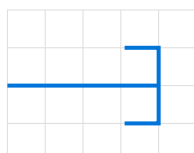
"straight"



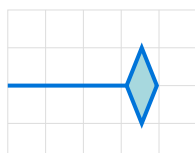
"barbed"



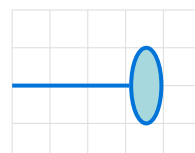
"bar"



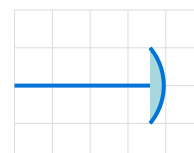
"bracket"



"diamond"



"ellipse"



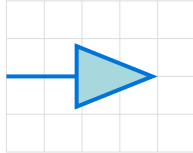
"parenthesis"

Not in CeTZ: width' (width relative to length). Use angle, or compute width from length.

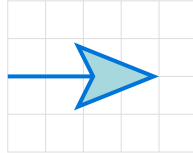
2.2.4 Inset

CeTZ manual: marks: inset

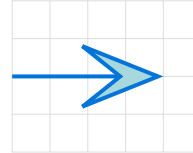
```
line((0, 0), (2, 0), stroke: 1.5pt + blue,
  mark: (end: ">>", length: 1, width: .8, inset: VALUE, fill: eastern.lighten(60%)))
```



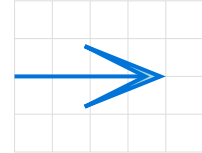
0



0.2



0.5

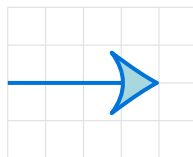


0.8

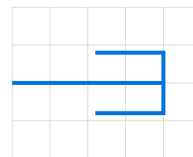
```
line((0, 0), (2, 0), stroke: 1.5pt + blue,
  mark: (end: VALUE, length: .6, width: .8, inset: .3, fill: eastern.lighten(60%)))
```



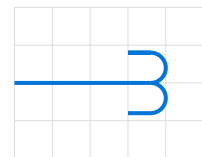
"stealth"



"curved-stealth"



"bracket"



"hook"

2.2.5 Angle

CeTZ manual: marks: width

angle is not in the manual. It sets width from length: $w = 2l \tan(\alpha/2)$.

```
line((0, 0), (1.2, .6), stroke: 1pt + blue,
  mark: (end: ">", length: .5, angle: VALUE, fill: eastern.lighten(60%)))
```



20deg



45deg



60deg



90deg

```
line((0, 0), (1.2, .6), stroke: 1pt + blue,
  mark: (end: "straight", length: .5, angle: VALUE))
```



20deg



45deg



60deg



90deg

2.2.6 Scale

CeTZ manual: marks: scale

```
line((0, 0), (1.5, 0), stroke: 1pt + blue,
  mark: (end: "stealth", scale: VALUE, fill: eastern.lighten(60%)))
```



1



2



3



4

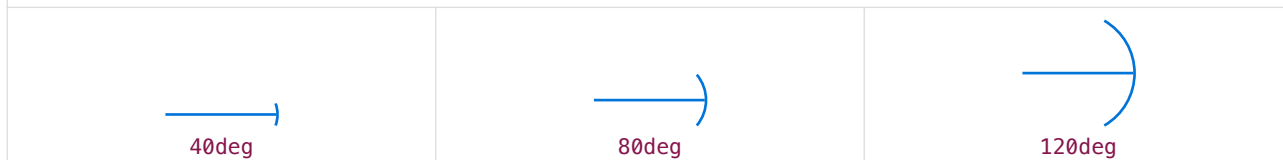
Not in CeTZ: scale length, scale width. Set length or width directly.

2.2.7 Arc

CeTZ manual: marks

Not in CeTZ: arc of Arc Barb and Hooks. The ")" mark reads angle as its arc span; the width follows from angle and length. Keep angle below 180deg.

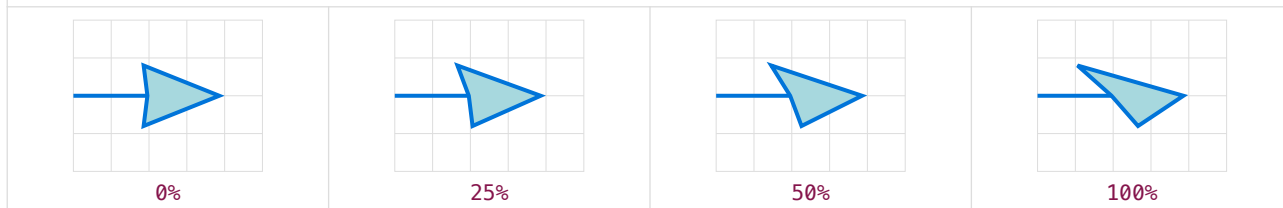
```
line((0, 0), (1.5, 0), stroke: 1pt + blue,
mark: (end: ")", angle: VALUE, scale: 2))
```



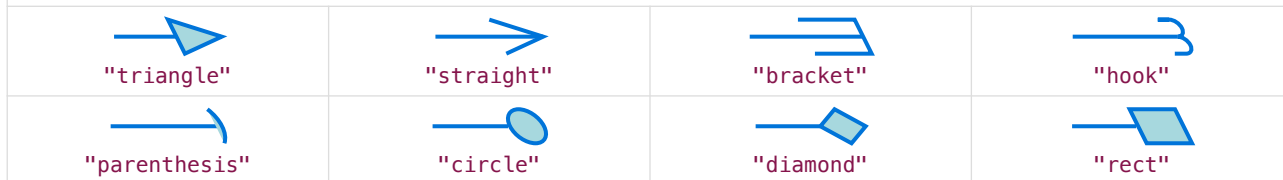
2.2.8 Slant

CeTZ manual: marks: slant

```
line((0, 0), (2, 0), stroke: 1.5pt + blue,
mark: (end: ">>", length: 1, width: .8, slant: VALUE, fill: eastern.lighten(60%)))
```



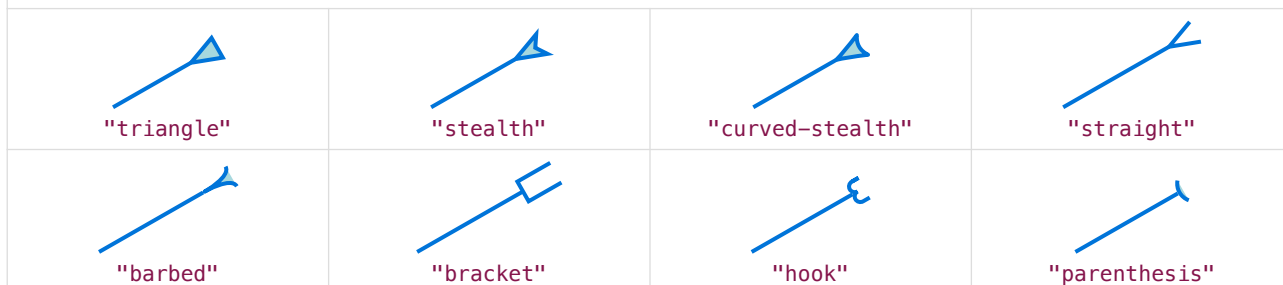
```
line((0, 0), (1.5, 0), stroke: 1.5pt + blue,
mark: (end: VALUE, scale: 3, slant: 50%, fill: eastern.lighten(60%)))
```



2.2.9 Reverse

CeTZ manual: marks: reverse

```
line((0, 0), (1.4, .8), stroke: 1.5pt + blue,
mark: (end: VALUE, scale: 2, fill: eastern.lighten(60%), reverse: true))
```



2.2.10 Harpoon and flip

CeTZ manual: marks: harpoon, flip

harpoon keeps one half (TikZ left); add flip for the other half (right, swap).

```
line((0, 0), (1.4, .8), stroke: 1.5pt + blue,
mark: (end: VALUE, scale: 2, fill: eastern.lighten(60%), harpoon: true))
```

 "triangle"	 "stealth"	 "straight"	 "barbed"	 "bar"
 "bracket"	 "hook"	 "diamond"	 "ellipse"	 "circle"

```
line((0, 0), (1.4, .8), stroke: 1.5pt + blue,
mark: (end: VALUE, scale: 2, fill: eastern.lighten(60%), harpoon: true, flip: true))
```

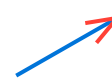
 "triangle"	 "stealth"	 "straight"	 "barbed"	 "bar"
 "bracket"	 "hook"	 "diamond"	 "ellipse"	 "circle"

2.2.11 Stroke and fill

CeTZ manual: styling

The mark takes the stroke and fill of its path. Paths have no fill by default, so marks are open. TikZ open is fill: none.

```
line((0, 0), (1.4, .8), stroke: 1.5pt + blue,
mark: (end: VALUE, scale: 2, fill: eastern.lighten(60%), stroke: red))
```

 "triangle"	 "stealth"	 "straight"	 "bracket"
 "hook"	 "circle"	 "diamond"	 "star"

```
line((0, 0), (1.4, .8), stroke: 1.5pt + blue,
mark: (end: "stealth", scale: 3, fill: VALUE))
```

 none	 blue	 red	 red.lighten(60%)
---	---	---	---

2.2.12 Cap, join, round, sharp

CeTZ manual: styling

```
line((0, 0), (2, 0), stroke: 1pt + blue,
mark: (end: "straight", scale: 5, stroke: (thickness: 8pt, cap: VALUE)))
```

 "butt"	 "round"	 "square"
---	--	---

<pre>line((0, 0), (2, 0), stroke: 1pt + blue, mark: (end: "straight", scale: 5, stroke: (thickness: 8pt, join: VALUE)))</pre>		
		
"miter"	"round"	"bevel"

TikZ round is stroke: (cap: "round", join: "round"), sharp is stroke: (cap: "butt", join: "miter").

2.2.13 Line width

CeTZ manual: styling

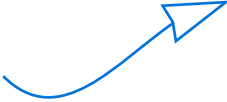
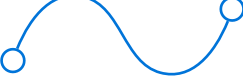
<pre>line((0, 0), (1.5, 0), stroke: 4pt + blue, mark: (end: ">", scale: 3, stroke: VALUE))</pre>			
			
0.5pt	1pt	2pt	4pt

Not in CeTZ: line width' and line widths relative to the path. Give the mark stroke an absolute thickness.

2.3 Placement

2.3.1 Marks on curves

CeTZ manual: marks: position-samples

	<pre>bezier((0, 0), (3, 1), (1, -1), (2, 1), stroke: blue, mark: (end: ">>", length: .8, width: .5))</pre>
	<pre>arc((0, 0), start: 0deg, stop: 270deg, radius: .6, stroke: blue, mark: (start: " ", end: ">", scale: 2, fill: red))</pre>
	<pre>hobby((0, 0), (1, 1), (2, 0), (3, 1), stroke: blue, mark: (symbol: "o", scale: 2))</pre>

Not in CeTZ: bending, flex, quick. Marks stay straight. CeTZ aligns a mark with the chord between its tip and base on the curve, and shortens curves along the curve. position-samples sets the precision.

2.3.2 Position along the path

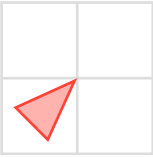
CeTZ manual: marks: pos, offset, anchor, shorten-to

<pre>line((0, 0), (2, 0), stroke: 1pt + blue, mark: (end: ">", scale: 2, VALUE)) circle((2, 0), radius: .05, fill: red, stroke: none)</pre>			
			
pos: 50%	offset: .5	anchor: "center"	anchor: "base"

<pre>line((0, 0), (2.5, 0), stroke: 1pt + blue, mark: (end: ("o", ">", ">"), scale: 2, shorten-to: VALUE))</pre>		
		
auto	0	none

2.3.3 Standalone mark

CeTZ manual: mark

	<pre>grid((-1, -1), (1, 1), stroke: silver) mark((0, 0), (1, 1), symbol: ">", scale: 4, stroke: red, fill: red.lighten(60%))</pre>
	<pre>grid((-1, -1), (1, 1), stroke: silver) mark((0, 0), 30deg, ">>", anchor: "center", scale: 4, stroke: blue, fill: eastern)</pre>

2.3.4 Transformed marks

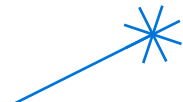
CeTZ manual: marks: transform-shape

The default is false (the manual says true).

<pre>scale(y: .4) line((0, 0), (2, 1.5), stroke: blue, mark: (end: ">", scale: 3, transform-shape: VALUE))</pre>	
 false	 true

2.3.5 Custom marks

CeTZ manual: register-mark

	<pre>register-mark("rays", style => { for i in range(8) { line((0, 0), (i * 45deg, style.length)) } anchor("tip", (0, 0)) anchor("base", (0, 0)) }) line((0, 0), (2, 1), stroke: blue, mark: (end: "rays", scale: 2))</pre>
---	--

3 Coordinates

3.1 Small pictures

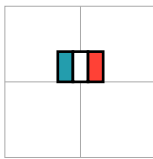
A TikZ `pic` is a Typst function that returns draw elements. Wrap them in `group` so transforms and styles stay local.

3.1.1 Define and place

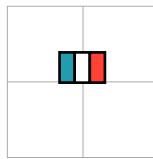
CeTZ manual: group

	<pre>let flag(pos, name: none) = group(name: name, { set-origin(pos) rect((-0.3, 0), (-0.1, 0.4), fill: eastern) rect((-0.1, 0), (0.1, 0.4), fill: white) rect((0.1, 0), (0.3, 0.4), fill: red) }) flag((0, 0))</pre>
---	---

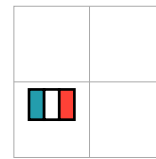
```
grid((0, 0), (2, 2), help-lines: true)
flag(VALUE)
```



(1, 1)



(45deg, 1.4)



((0, 0), 25%, (2, 2))

3.1.2 Pass style through

CeTZ manual: styling



```
let seal(pos, ..style) = group({
  set-origin(pos)
  circle((0, 0), radius: .4, ..style)
  line((-0.3, -0.3), (0.3, 0.3), stroke: gray)
})
seal((0, 0), stroke: red)
```

```
seal((0, 0), VALUE)
```



fill: aqua



stroke: 2pt + red



fill:
gradient.radial(white,
eastern, focal-center:
(30%, 30%))

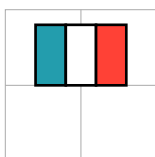


stroke: none, fill:
red.lighten(50%)

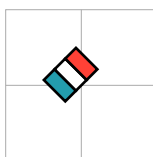
3.1.3 Transform a picture

CeTZ manual: scale, rotate

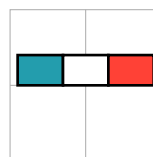
```
grid((0, 0), (2, 2), help-lines: true)
group({
  set-origin((1, 1))
  VALUE
  flag((0, 0))
})
```



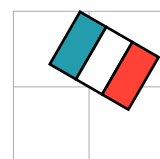
scale(2)



rotate(45deg)



scale(x: 3)



scale(2)
rotate(-30deg)

Transforms move and resize shapes but not text; auto-scale scales content too, like TikZ transform shape.

CeTZ manual: content

Aa

Aa

```
scale(2)
content((0, 0), [Aa])
content((1, 0), [Aa], auto-scale: true)
```

3.1.4 Repeat a picture



```
for x in range(10) {
  flag((x * .8, 0))
}
```

3.1.5 Behind or in front: layers

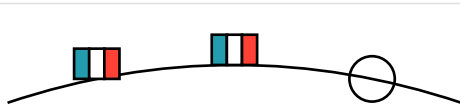
CeTZ manual: on-layer

```
rect((0, 0), (1.5, 1), fill: lime, stroke: none)
VALUE
```



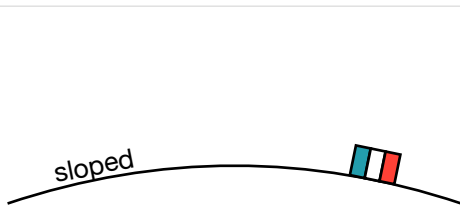
3.1.6 Place on a path

CeTZ manual: path anchors



```
bezier((0, 0), (6, 0), (3, 1), name: "p")
flag("p.20%")
flag("p.mid")
circle("p.80%", radius: .3)
```

Not in CeTZ: sloped for pictures. Rotate by the path direction yourself; for content, pass a coordinate to angle.



```
bezier((0, 0), (6, 0), (3, 1), name: "p")
get-ctx(ctx => {
  let (_, a, b) = cetz.coordinate.resolve(
    ctx, "p.80%", "p.81%")
  group({
    set-origin(a)
    rotate(cetz.vector.angle2(a, b))
    flag((0, 0))
  })
})
content("p.20%", angle: "p.21%", anchor: "south", [sloped])
```

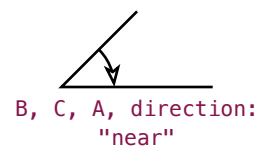
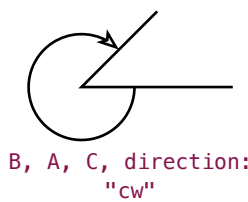
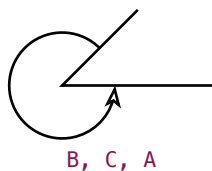
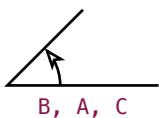
3.2 Angles

CeTZ manual: angle

cetz.angle.angle(origin, a, b) marks the angle from a to b around origin. Below, A = (2, 0), B = (0, 0), C = (1, 1), drawn as line(A, B, C).

3.2.1 Side order and direction

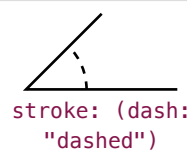
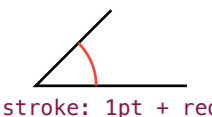
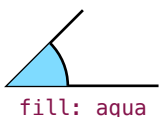
```
cetz.angle.angle(VALUE, radius: .7, mark: (end: "stealth"))
```



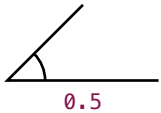
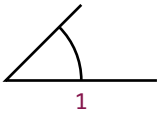
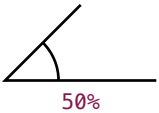
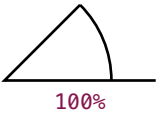
Default direction: "ccw"; "near" and "far" pick the smaller or larger angle.

3.2.2 Style

```
cetz.angle.angle(B, A, C, radius: .8, VALUE)
```

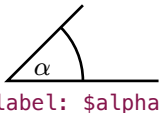
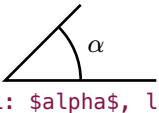
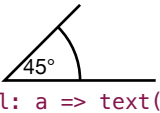


3.2.3 Radius

<code>cetz.angle.angle(B, A, C, radius: VALUE)</code>			
			
0.5	1	50%	100%

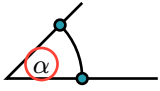
Default 0.5. A ratio is relative to the shorter side.

3.2.4 Label

<code>cetz.angle.angle(B, A, C, radius: 1, VALUE)</code>			
			
label: \$alpha\$	label: \$alpha\$, label-radius: 100%	label: \$alpha\$, label-radius: 130%	label: a => text(8pt) [#calc.round(a.deg())°]

Default label-radius: 50%, relative to radius.

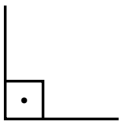
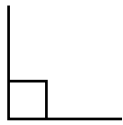
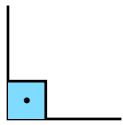
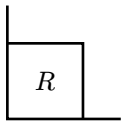
3.2.5 Name and anchors

	<pre> let (A, B, C) = ((2, 0), (0, 0), (1, 1)) line(A, B, C) cetz.angle.angle(B, A, C, radius: 1, label: \$alpha\$, name: "x") circle("x.label", radius: 6pt, stroke: red) circle("x.start", radius: 2pt, fill: eastern) circle("x.end", radius: 2pt, fill: eastern) </pre>
---	---

Anchors: origin, a, b, start, end, label, center.

3.2.6 Right angle

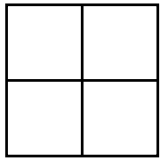
CeTZ manual: right-angle

<pre> line((1.5, 0), (0, 0), (0, 1.5)) cetz.angle.right-angle((0, 0), (1.5, 0), (0, 1.5), VALUE) </pre>			
			
radius: .5	label: none	fill: aqua	radius: 1, label: \$R\$

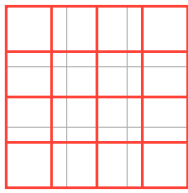
Defaults: radius: .5, label: ".".

3.3 Grid

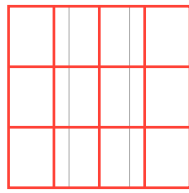
CeTZ manual: grid

	<code>grid((0, 0), (2, 2))</code>
---	-----------------------------------

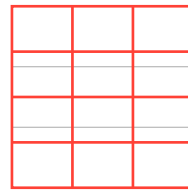
```
grid((0, 0), (3, 3), help-lines: true)
grid((0, 0), (3, 3), stroke: red, step: VALUE)
```



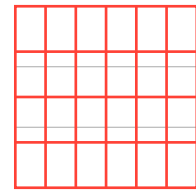
0.75



(x: 0.75)

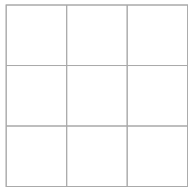


(y: 0.75)

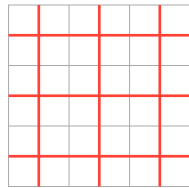


(0.5, 0.75)

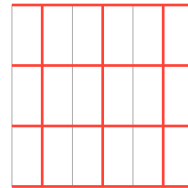
```
grid((0, 0), (3, 3), help-lines: true)
grid((0, 0), (3, 3), stroke: red, VALUE)
```



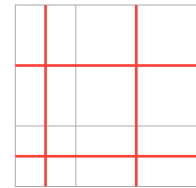
help-lines: true



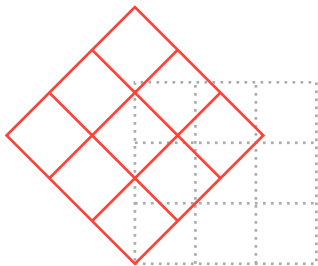
shift: 0.5



shift: (0.5, 0)



step: 1.5, shift: 0.5



```
grid((0, 0), (3, 3), stroke: (paint: gray, dash: "dotted"))
rotate(45deg)
grid((0, 0), (3, 3), stroke: red)
```

3.4 Coordinate systems

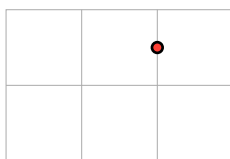
CeTZ manual: coordinate systems

Every argument that takes a position accepts any of the forms below. Numbers are canvas units (cetz.canvas(length: 1cm) by default); lengths are absolute.

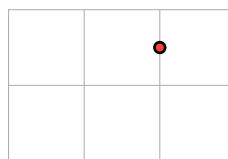
3.4.1 Canvas (xyz)

CeTZ manual: xyz

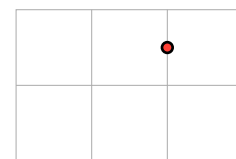
```
circle(VALUE, radius: 2pt, fill: red)
```



(x: 2, y: 1.5)



(2, 1.5)

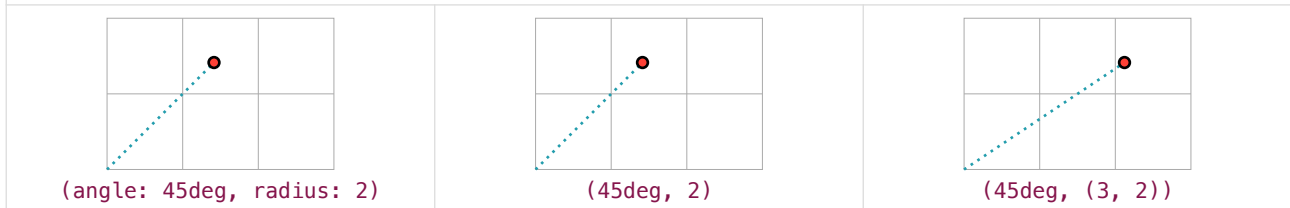


(2cm, 15mm)

3.4.2 Polar

CeTZ manual: polar

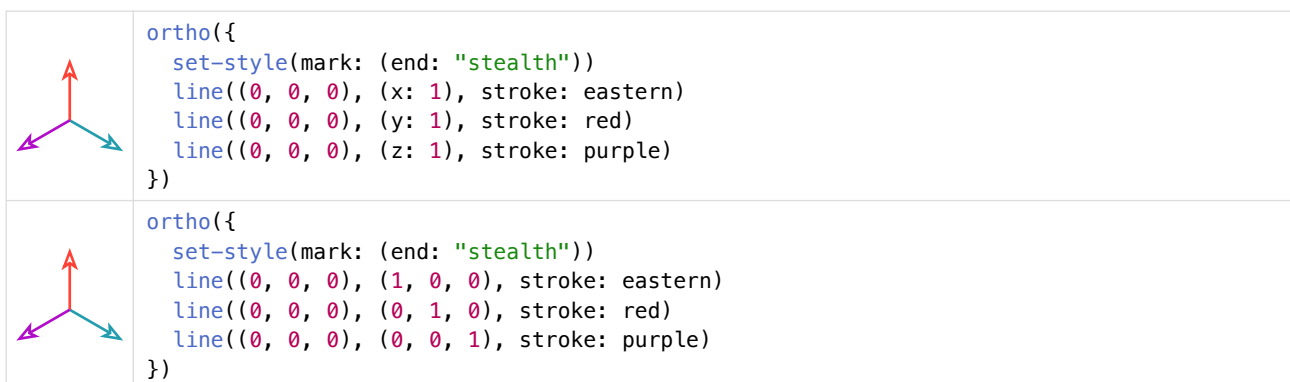
```
line((0, 0), VALUE, stroke: (paint: eastern, dash: "dotted"))
circle(VALUE, radius: 2pt, fill: red)
```



3.4.3 Three axes

CeTZ manual: xyz

A z component needs a projection such as ortho to show.

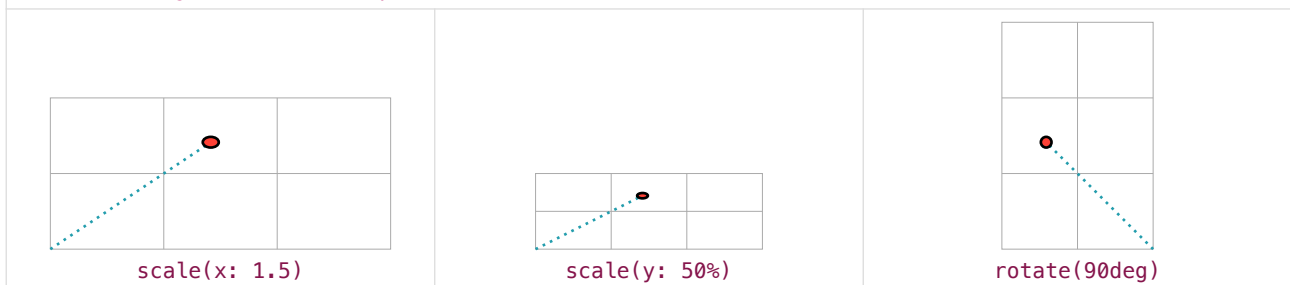


3.4.4 Other unit vectors

CeTZ manual: scale, rotate

TikZ x=, y= options: transform the axes, or pass x:, y:, z: vectors to cetz.canvas for the whole picture.

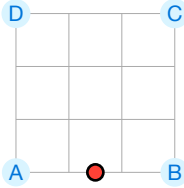
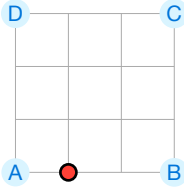
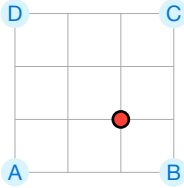
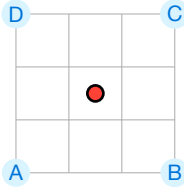
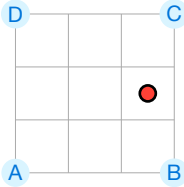
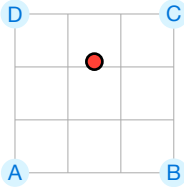
```
VALUE
grid((0, 0), (3, 2), help-lines: true)
line((0, 0), (45deg, 2), stroke: (paint: eastern, dash: "dotted"))
circle((45deg, 2), radius: 2pt, fill: red)
```



3.4.5 Barycentric

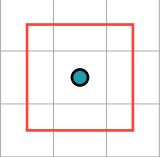
CeTZ manual: barycentric

A...D are anchors at the corners of the square.

<code>circle((bary: VALUE), radius: 3pt, fill: red)</code>		
 (A: 1, B: 1)	 (A: 2, B: 1)	 (A: 1, B: 1, C: 1)
 (A: 1, B: 1, C: 1, D: 1)	 (A: .2, B: .4, C: .6)	 (A: .2, B: .4, C: .6, D: .8)

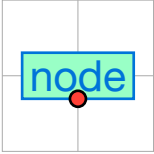
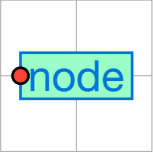
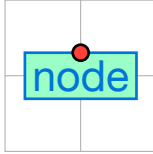
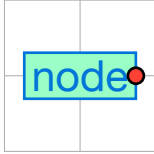
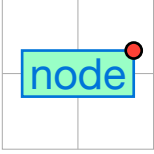
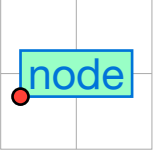
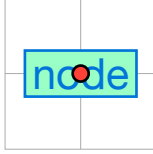
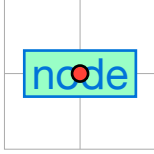
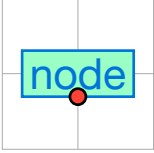
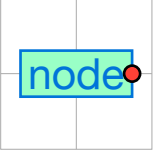
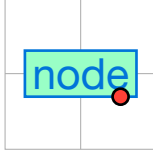
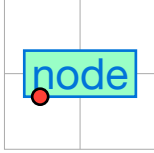
3.4.6 Named positions

CeTZ manual: anchor

	<pre>grid((0, 0), (3, 3), help-lines: true) anchor("centre", (1.5, 1.5)) anchor("A", (.5, .5)) anchor("B", (2.5, 2.5)) circle("centre", radius: 3pt, fill: eastern) rect("A", "B", stroke: red)</pre>
---	---

3.4.7 Anchors of an element

CeTZ manual: anchors

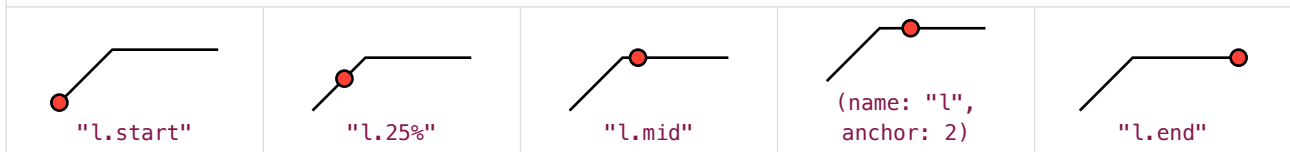
<code>circle(VALUE, radius: 3pt, fill: red)</code>			
 "A.south"	 "A.west"	 "A.north"	 "A.east"
 "A.north-east"	 "A.south-west"	 "A.center"	 "A"
<code>circle((name: "A", anchor: VALUE), radius: 3pt, fill: red)</code>			
 "south"	 0deg	 -30deg	 -150deg

"A" alone is the default anchor. An angle is a border anchor: the ray from the center at that angle.

3.4.8 Positions along a path

CeTZ manual: path anchors

```
line((0, 0), (1, 1), (3, 1), name: "l")
circle(VALUE, radius: 3pt, fill: red)
```

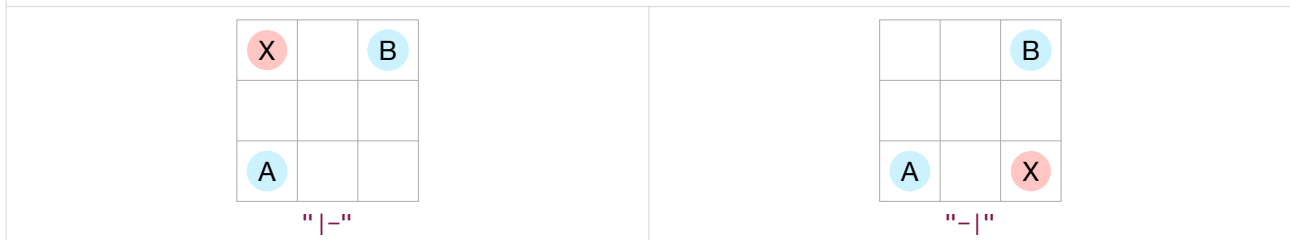


A ratio is a fraction of the path length; a number is a distance along the path.

3.4.9 Perpendicular and projection

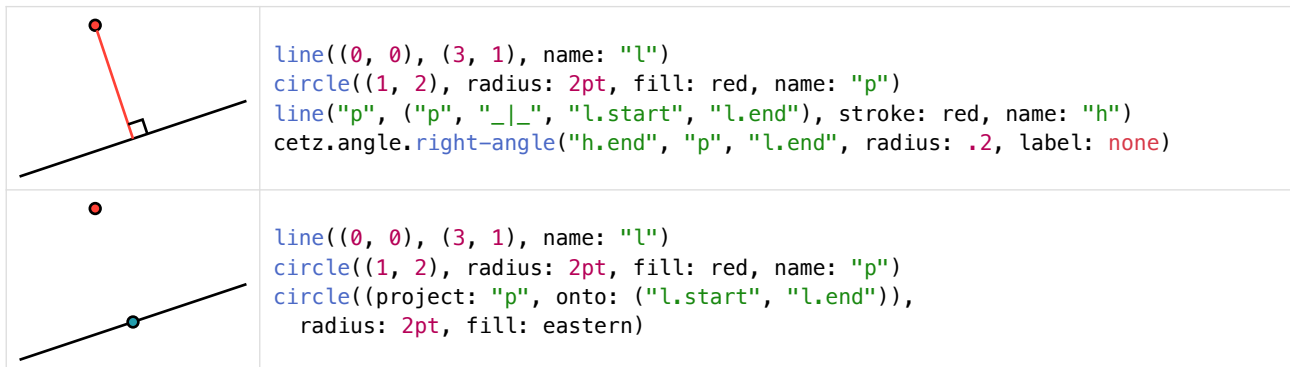
CeTZ manual: perpendicular

```
content(("A", VALUE, "B"), [X], fill: red.lighten(70%))
```



(p, "|-", q) has the x of p and the y of q.

CeTZ manual: projection

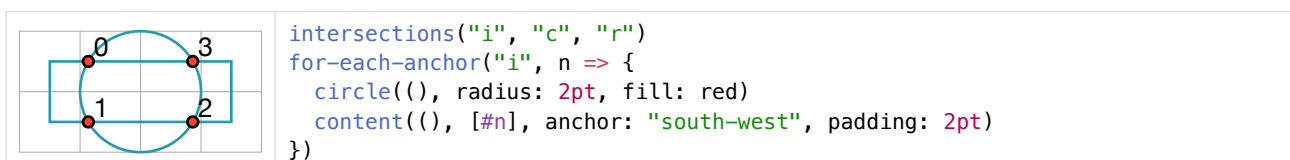
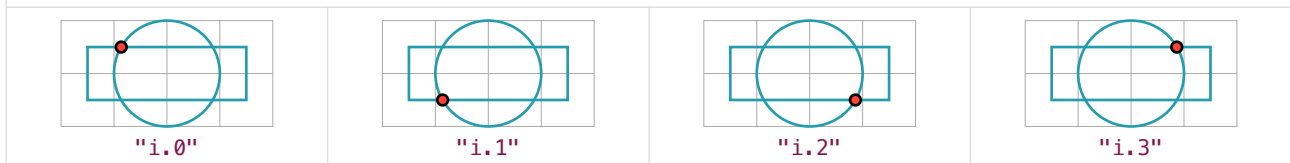


3.4.10 Intersections

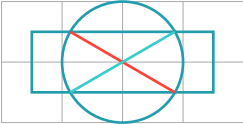
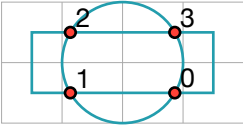
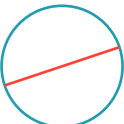
CeTZ manual: intersections

c is circle((2, 1)), r is rect((.5, .5), (3.5, 1.5)). Intersections become anchors i.0, i.1, ...

```
intersections("i", "c", "r")
circle(VALUE, radius: 2pt, fill: red)
```



```
intersections("i", "c", "r")
for-each-anchor("i", n => {
  circle((n), radius: 2pt, fill: red)
  content((n), [#n], anchor: "south-west", padding: 2pt)
})
```

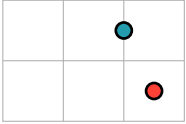
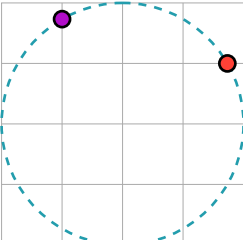
	<pre> intersections("i", "c", "r") line("i.0", "i.2", stroke: red) line("i.1", "i.3", stroke: teal) </pre>
	<pre> let by-angle = cetz.sorting.points-by-angle.with(reference: (2, 1)) intersections("i", "c", "r", sort: by-angle) for-each-anchor("i", n => { circle((0, 0), radius: 2pt, fill: red) content((0, 0), [#n], anchor: "south-west", padding: 2pt) }) </pre>
	<pre> intersections("i", { circle((0, 0), stroke: eastern) hide(line((-1.5, -.5), (1.5, .5))) }) line("i.0", "i.1", stroke: red) </pre>

Not in CeTZ: `by={a,b,...}` renaming. Destructure the names: `let (a, b) = ("i.0", "i.1").`

3.4.11 Computed positions

[CeTZ manual: xyz](#)

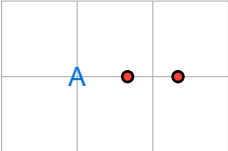
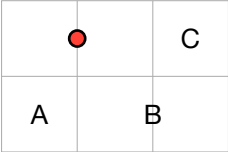
Coordinates are Typst arrays, so any Typst expression works.

	<pre> grid((0, 0), (3, 2), help-lines: true) circle((2, 1.5), radius: 3pt, fill: eastern) circle((2 + .5, 1.5 - 1), radius: 3pt, fill: red) </pre>
	<pre> grid((0, 0), (4, 4), help-lines: true) circle((2, 2), radius: 2, stroke: (paint: eastern, dash: "dashed")) circle((2 + 2 * calc.cos(30deg), 2 + 2 * calc.sin(30deg)), radius: 3pt, fill: red) circle((rel: (120deg, 2), to: (2, 2)), radius: 3pt, fill: purple) </pre>

3.4.12 Vector arithmetic

[CeTZ manual: function](#)

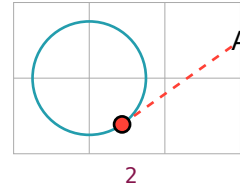
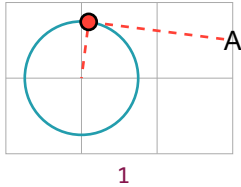
A function coordinate receives the resolved coordinates as vectors; `cetz.vector` has the arithmetic.

	<pre> grid((0, 0), (3, 2), help-lines: true) content((1, 1), text(blue)[A], name: "a") circle((rel: (2/3, 0), to: "a"), radius: 2pt, fill: red) circle((v => cetz.vector.add(v, (4/3, 0)), "a"), radius: 2pt, fill: red) </pre>
	<pre> grid((0, 0), (3, 2), help-lines: true) content((.5, .5), [A], name: "A") content((2, .5), [B], name: "B") content((2.5, 1.5), [C], name: "C") let f = (a, b, c) => cetz.vector.add(a, cetz.vector.sub(c, b)) circle((f, "A", "B", "C"), radius: 3pt, fill: red) </pre>

3.4.13 Tangents

CeTZ manual: tangent

```
circle((1, 1), radius: .75, name: "c", stroke: eastern)
content((3, 1.5), [A], name: "A")
line("A", (element: "c", point: "A", solution: VALUE), "c",
  stroke: (paint: red, dash: "dashed"))
circle((element: "c", point: "A", solution: VALUE), radius: 3pt, fill: red)
```

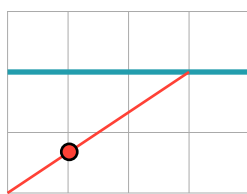
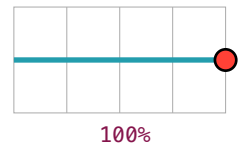
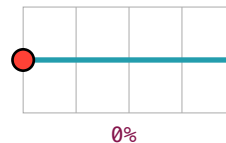
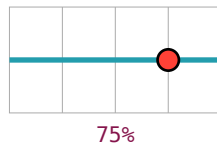
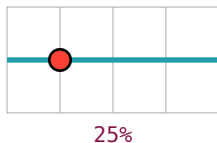


Works for circles and ellipses only.

3.4.14 Interpolation by ratio

CeTZ manual: interpolation

```
circle(((0, 1), VALUE, (4, 1)), radius: 4pt, fill: red)
```

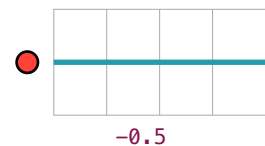
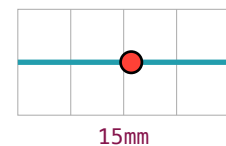
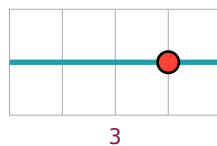
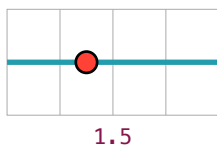


```
grid((0, 0), (4, 3), help-lines: true)
line((0, 2), (4, 2), stroke: 2pt + eastern)
line((0, 0), ((0, 2), 75%, (4, 2)), stroke: red)
circle(((0, 2), 75%, (4, 2)), 66%, (0, 0)), radius: 3pt, fill: red)
```

3.4.15 Interpolation by distance

CeTZ manual: interpolation

```
circle(((0, 1), VALUE, (4, 1)), radius: 4pt, fill: red)
```



An angle rotates the target around the start first:



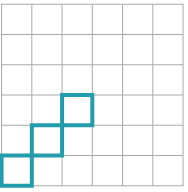
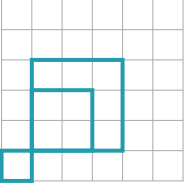
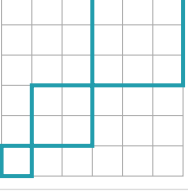
```
grid((0, 0), (4, 3), help-lines: true)
line((0, 0), (4, 1), stroke: 2pt + eastern)
line((2, .5), ((2, .5), 2, 90deg, (4, 1)), stroke: 2pt + red)
```

3.4.16 Relative

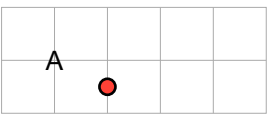
CeTZ manual: relative

(rel: v) is ++v: it moves the current position. With update: false it is +v.

	<pre>line((0, 1), (1, 1), (2, 2), (2, 0))</pre>
	<pre>line((0, 1), (1, 1), (rel: (2, 1), update: false), (rel: (2, -1), update: false))</pre>
	<pre>line((0, 1), (1, 1), (rel: (2, 1)), (rel: (2, -1)))</pre>

	<pre>rect((0, 0), (1, 1)) rect((1, 1), (2, 2)) rect((2, 2), (3, 3))</pre>
	<pre>rect((0, 0), (1, 1)) rect((1, 1), (rel: (2, 2), update: false)) rect((2, 2), (rel: (3, 3), update: false))</pre>
	<pre>rect((0, 0), (1, 1)) rect((1, 1), (rel: (2, 2))) rect((2, 2), (rel: (3, 3)))</pre>

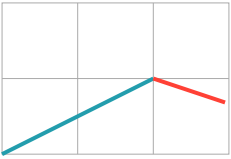
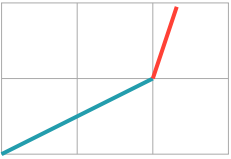
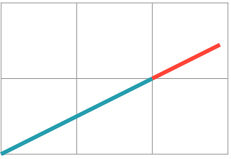
() is the current position. Polar and named coordinates work inside rel; to: sets the origin.

	<pre>line((0, 1), (rel: (0deg, 1)), (rel: (30deg, 2)), (rel: (-30deg, 2)))</pre>
	<pre>content((1, 1), [A], name: "A") circle((rel: (1, -.5), to: "A"), radius: 3pt, fill: red)</pre>

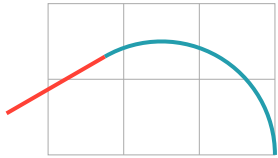
3.4.17 Turning

CeTZ manual: interpolation

Not in CeTZ: [turn] polar coordinates. Interpolate with an angle and a negative distance, which continues the last segment past its end.

<pre>line((0, 0), (2, 1), name: "l", stroke: 1.5pt + eastern) line("l.end", ("l.end", -1, VALUE, "l.start"), stroke: 1.5pt + red)</pre>		
		
-45deg	45deg	0deg

The same trick continues an arc along its tangent:



```

grid((0, 0), (3, 2), help-lines: true)
arc((3, 0), start: 0deg, stop: 120deg, radius: 1.5, name: "a",
stroke: 1.5pt + eastern)
line("a.arc-end", ("a.arc-end", 1.5, -90deg, "a.origin"),
stroke: 1.5pt + red)

```

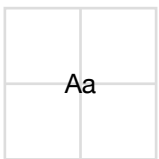
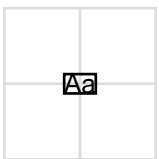
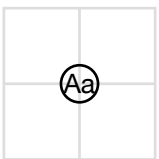
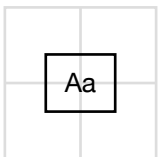
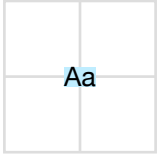
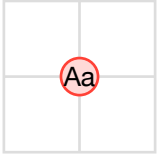
4 Content and anchors

CeTZ has no nodes. content places Typst content at a coordinate, frame draws a box or circle around it, and name makes its anchors available to later elements.

4.1 Creating content

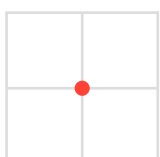
[CeTZ manual: content](#)

```
content((1, 1), [Aa], VALUE)
```

 frame: none	 frame: "rect"	 frame: "circle"
 frame: "rect", padding: .25	 frame: "rect", stroke: none, fill: aqua.lighten(50%)	 frame: "circle", stroke: red, fill: red.lighten(80%)

4.1.1 A named point

[CeTZ manual: anchor](#)



```

anchor("P", (1, 1))
circle("P", radius: .1, fill: red, stroke: none)

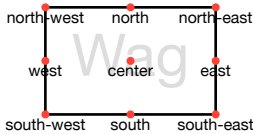
```

4.1.2 Several at once

1 2 3 4 5	<pre> for x in range(1, 6) { content((x, 0), [#x]) } </pre>
1 2 3 4 5	<pre> set-style(content: (frame: "rect", stroke: red, padding: .1)) for x in range(1, 6) { content((x, 0), [#x]) } </pre>
0 1 2 3 4	<pre> let sq = (frame: "rect", stroke: red) let ci = (frame: "circle", stroke: blue) for (x, s) in (sq, ci, ci, sq, sq).enumerate() { content((x, 0), [#x], padding: .1, ..s) } </pre>

4.2 Names and anchors

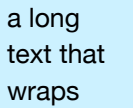
CeTZ manual: anchors

	<pre>content((0, 0), [A], name: "A", frame: "rect", padding: .1) circle("A", radius: .5, stroke: eastern)</pre>		
	<pre>set-style(circle: (radius: 1.5pt, stroke: none)) content((0, 0), text(22pt, silver)[Wag], frame: "rect", padding: .35, name: "c") for a in ("north", "north-east", "east", "south-east", "south", "south-west", "west", "north-west", "center") { circle("c." + a, fill: red) content("c." + a, text(6pt, a), anchor: "north", padding: .05) }</pre>		
<pre>content((0, 0), text(22pt, gray)[Wag], frame: "rect", padding: .15, stroke: blue, name: "c") circle("c." + VALUE, radius: 2pt, fill: red, stroke: none)</pre>			
			
"mid-west"	"mid"	"mid-east"	"text"
			
"base-west"	"base"	"base-east"	"center"

Not in CeTZ: alias.anchor("B", "A") names a second point at A's default anchor.

4.3 Content body

CeTZ manual: content

<pre>content((0, 0), VALUE, frame: "rect", stroke: none, fill: aqua.lighten(60%), padding: .1)</pre>		
		
[XXX]	text(red)[XXX]	text(14pt, weight: "bold")[XXX]
		
$x^2 + y^2$	[XXX \ YYY]	box(width: 1.6cm)[a long text that wraps]

4.3.1 Rotation

<pre>content((0, 0), [XXX], angle: VALUE, frame: "rect", padding: .1)</pre>			
			
0deg	30deg	90deg	-45deg

4.3.2 Text style for all content

A	B	<pre>set-style(content: (wrap: text.with(fill: red, weight: "bold"))) content((0, 0), [A]) content((1, 0), [B])</pre>
---	---	---

4.3.3 Content inside a rectangle

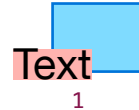
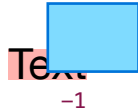
The text fills the box between the two coordinates.

```
rect((0, 0), (4, 1.6), stroke: silver)
content((0, 0), (4, 1.6), box(width: 100%, height: 100%, inset: 4pt,
  par(justify: true)[The text fills the box between the two
  coordinates.]))
```

4.4 Behind or in front

CeTZ manual: on-layer

```
on-layer(VALUE, content((0, 0), text(16pt)[Text], frame: "rect",
  stroke: none, fill: red.lighten(60%)))
rect((0, -1), (1.2, .8), fill: aqua, stroke: blue)
```

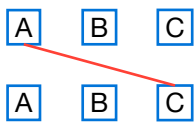


More on layers in Section 5.

4.5 Name prefixes

CeTZ manual: group

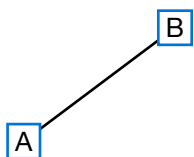
Not in CeTZ: name prefix, name suffix. A named group prefixes the names of its children: "top.A".



```
set-style(content: (frame: "rect", padding: .1, stroke: blue))
for (g, y) in (("top", 1), ("bottom", 0)) {
  group(name: g, {
    for (x, t) in ("A", "B", "C").enumerate() {
      content((x, y), t, name: t)
    }
  })
}
line("top.A.south", "bottom.C.north", stroke: red)
```

4.6 Links

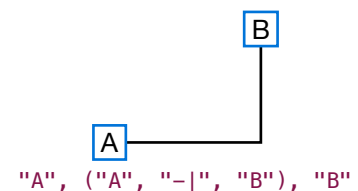
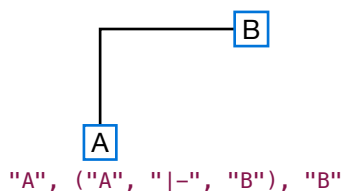
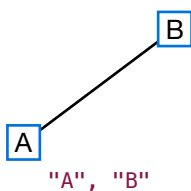
CeTZ manual: line



```
set-style(content: (frame: "rect", padding: .1,
  fill: white, stroke: blue))
on-layer(1, {
  content((0, 0), [A], name: "A")
  content((2, 1.5), [B], name: "B")
})
line("A", "B")
```

line stops at the frames of named elements. Curves do not, so the pictures below keep the setup above: the frames sit on layer 1 and hide the curve ends.

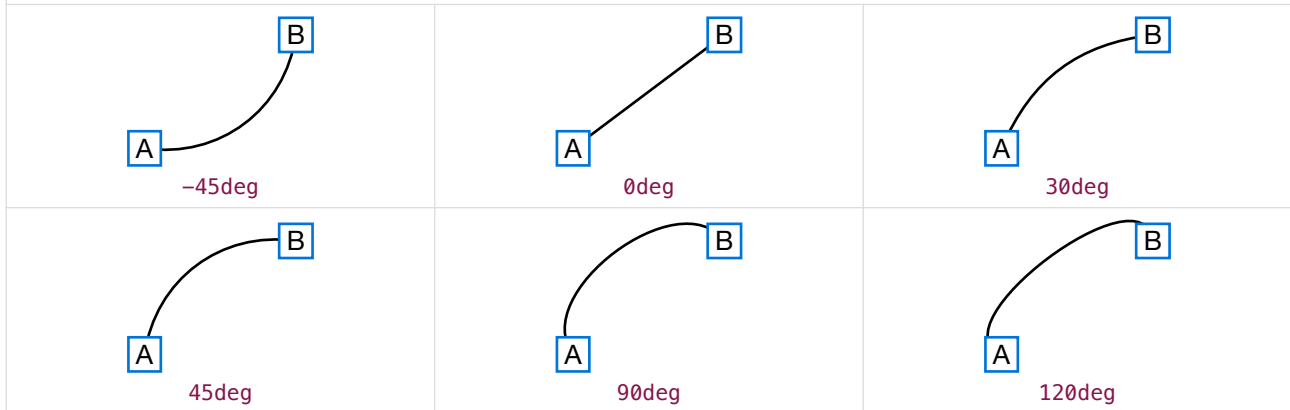
line(VALUE)



4.6.1 Bent links

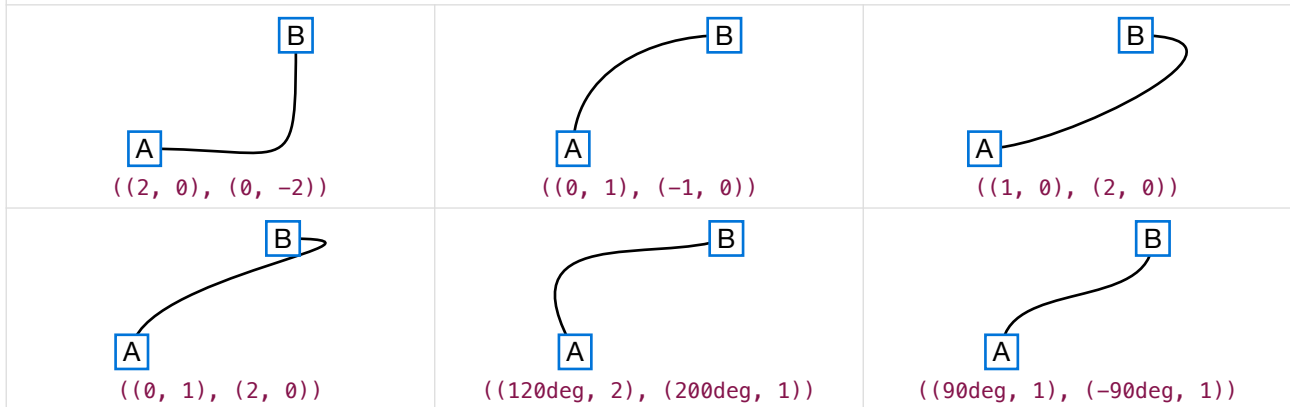
CeTZ manual: bezier

```
bezier("A", "B", ("A", 40%, VALUE, "B"), ("B", 40%, -VALUE, "A"))
```

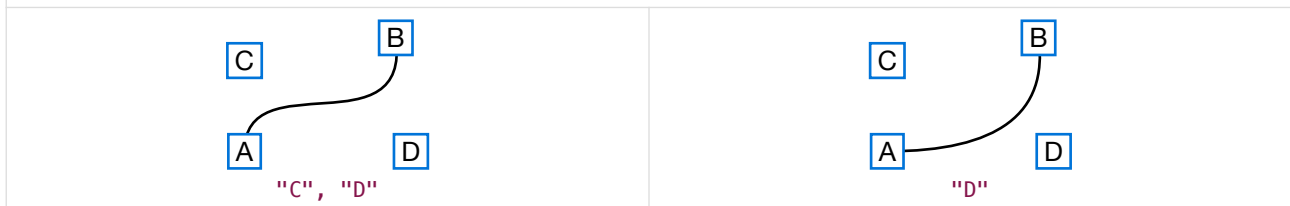


4.6.2 Control points

```
let (a, b) = VALUE
bezier("A", "B", (rel: a, to: "A"), (rel: b, to: "B"))
```

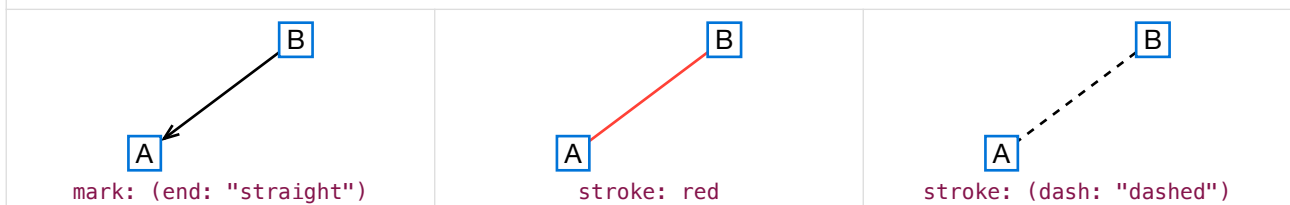


```
bezier("A", "B", VALUE)
```



4.6.3 Link style

```
line("B", "A", VALUE)
```

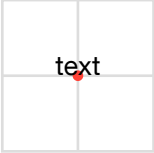
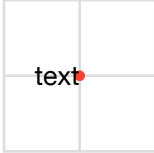
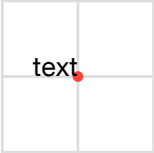
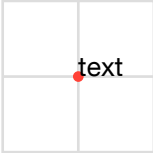
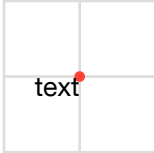
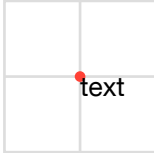


4.7 Labels

CeTZ manual: anchors

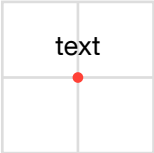
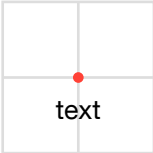
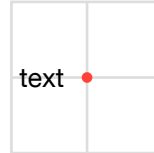
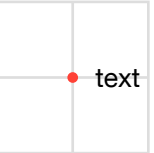
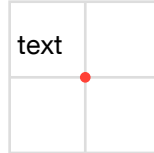
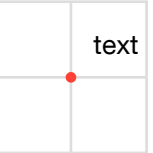
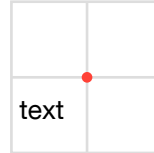
Place content at a point and choose which of its anchors lands there.

```
circle((0, 0), radius: 2pt, fill: red, stroke: none)
content((0, 0), [text], anchor: VALUE)
```

 "south"	 "north"	 "east"	 "west"
 "south-east"	 "south-west"	 "north-east"	 "north-west"

4.7.1 Distance

```
circle((0, 0), radius: 2pt, fill: red, stroke: none)
content((0, 0), [text], anchor: VALUE, padding: .3)
```

 "south"	 "north"	 "east"	 "west"
 "south-east"	 "south-west"	 "north-east"	 "north-west"

4.7.2 Label on a named element

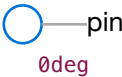
```
rect((-0.2, -0.2), (0.2, 0.2), stroke: blue, name: "n")
content((name: "n", anchor: VALUE), [label], anchor: VALUE + 180deg, padding: .05)
```

 0deg	 90deg	 180deg	 270deg	 45deg
---	--	---	---	--

4.8 Pins

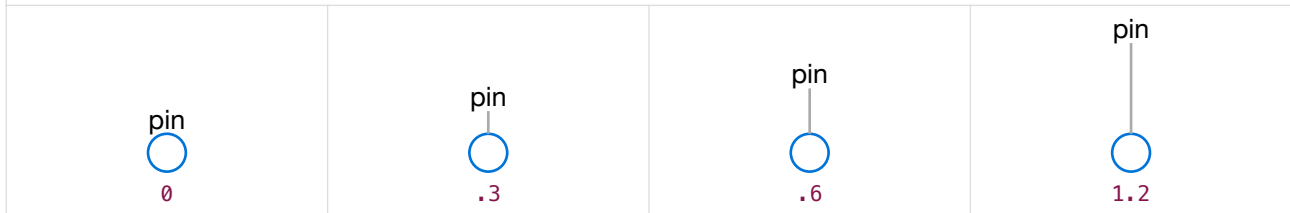
CeTZ manual: border anchors

```
circle((0, 0), radius: .25, stroke: blue, name: "c")
line((name: "c", anchor: VALUE), (rel: (VALUE, .6)), stroke: gray)
content((), [pin], anchor: VALUE + 180deg)
```

 90deg	 60deg	 0deg	 210deg
--	--	--	---

4.8.1 Pin length

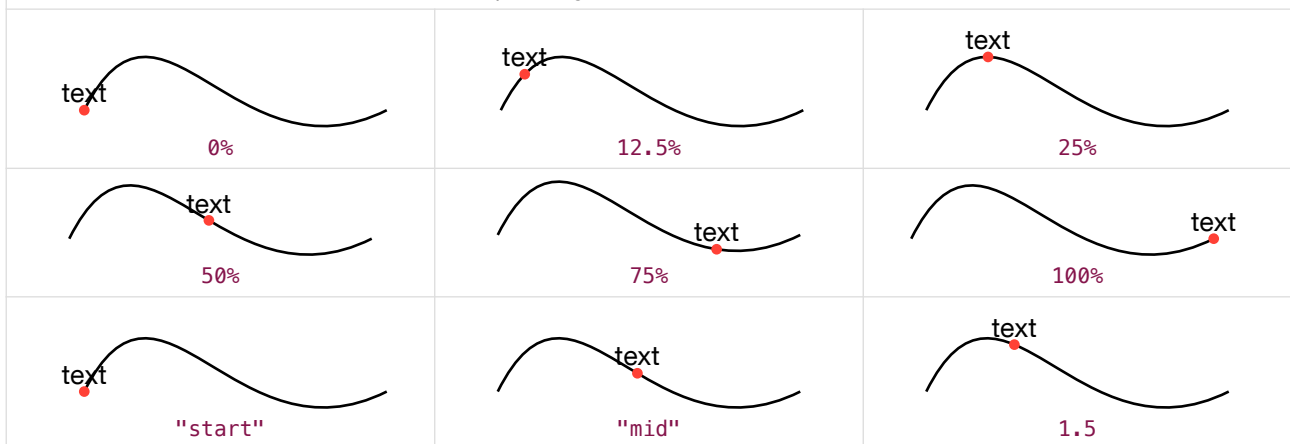
```
circle((0, 0), radius: .25, stroke: blue, name: "c")
line("c.north", (rel: (0, VALUE)), stroke: gray)
content((), [pin], anchor: "south")
```



4.9 Content along a path

CeTZ manual: path anchors

```
bezier((0, 0), (4, 0), (1, 2), (2, -1), name: "p")
circle((name: "p", anchor: VALUE), radius: 2pt, fill: red, stroke: none)
content((), [text], anchor: "south", padding: .1)
```



"mid" is 50% of the path length; a number is an absolute distance along the path.

4.9.1 Sloped

```
bezier((0, 0), (4, 0), (1, 2), (2, -1), name: "p")
content("p.50%", angle: "p.51%", [text], anchor: VALUE, padding: .1)
```



4.10 Labels on links

CeTZ manual: content

```
line((0, 0), (3, 0), name: "l")
content(VALUE, padding: .1)
```

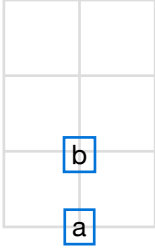
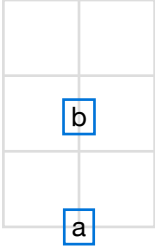
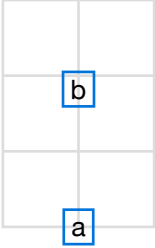
<pre>"l.mid", [abc], anchor: "south"</pre>	<pre>"l.20%", [abc], anchor: "south"</pre>	<pre>"l.mid", [abc], anchor: "north"</pre>
<pre>"l.mid", text(14pt)[abc], anchor: "south"</pre>	<pre>"l.mid", text(red)[abc], anchor: "south"</pre>	<pre>"l.mid", [abc], frame: "rect", fill: white</pre>
<pre>"l.mid", [abc], frame: "rect", fill: white, stroke: none</pre>	<pre>"l.mid", [abc], frame: "rect", stroke: none, fill: yellow</pre>	<pre>"l.mid", [abc], angle: 10deg, anchor: "south"</pre>

4.10.1 Style for all labels

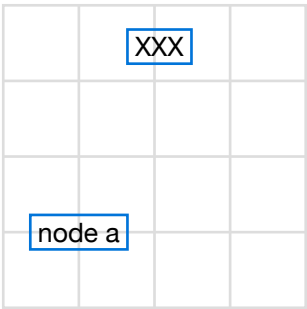
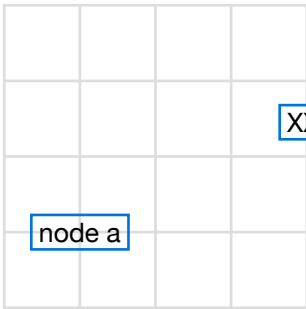
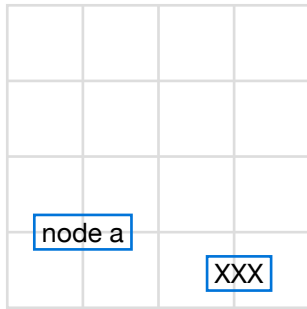
	<pre>set-style(content: (frame: "rect", stroke: none, fill: yellow)) line((0, 0), (3, 0), name: "l") content("l.mid", [abc])</pre>
---	--

4.11 Relative placement

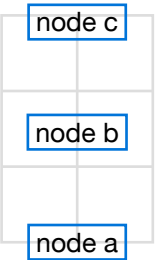
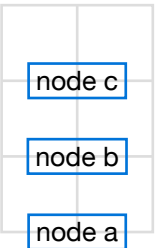
CeTZ manual: relative coordinates

<pre>set-style(content: (frame: "rect", padding: .1, stroke: blue)) content((0, 0), [a], name: "a") content((rel: (0, VALUE), to: "a.north"), [b], anchor: "south")</pre>		
		
.5	1	.5 + calc.sin(60deg)

4.11.1 Diagonal

<pre>set-style(content: (frame: "rect", padding: .1, stroke: blue)) content((0, 0), [node a], name: "a") content((rel: VALUE, to: "a.north-east"), [XXX], anchor: "south-west")</pre>		
		
(0, 2)	(2, 1)	(1, -1)

4.11.2 Border to border or centre to centre

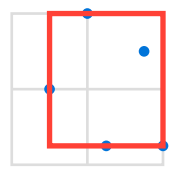
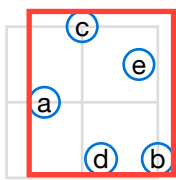
	<pre>set-style(content: (frame: "rect", padding: .1, stroke: blue)) content((0, 0), [node a], name: "a") content((rel: (0, 1), to: "a.north"), [node b], anchor: "south", name: "b") content((rel: (0, 1), to: "b.north"), [node c], anchor: "south")</pre>
	<pre>set-style(content: (frame: "rect", padding: .1, stroke: blue)) content((0, 0), [node a], name: "a") content((rel: (0, 1), to: "a"), [node b], name: "b") content((rel: (0, 1), to: "b"), [node c])</pre>

4.11.3 Baseline alignment

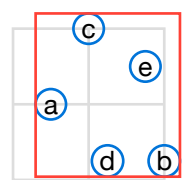
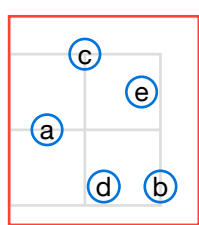
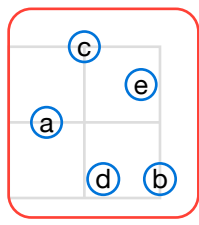
	<pre> set-style(content: (frame: "rect", padding: .05, stroke: blue)) content((0, 0), text(28pt)[X], name: "X") content((rel: (.6, 0), to: "X.east"), text(12pt)[a], anchor: "west", name: "a") content((rel: (.6, 0), to: "a.east"), text(28pt)[g], anchor: "west") </pre>
	<pre> set-style(content: (frame: "rect", padding: .05, stroke: blue)) content((0, 0), text(28pt)[X], name: "X") content((rel: (.6, 0), to: "X.base-east"), text(12pt)[a], anchor: "base-west", name: "a") content((rel: (.6, 0), to: "a.base-east"), text(28pt)[g], anchor: "base-west") </pre>

4.12 Fitting

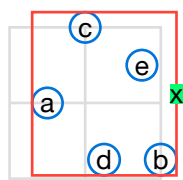
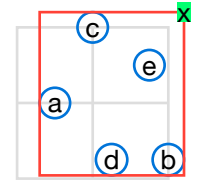
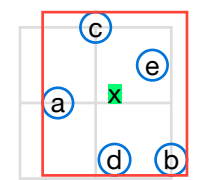
CeTZ manual: rect-around

	<pre> for p in ((.5, 1), (2, .25), (1, 2), (1.25, .25), (1.75, 1.5)) { circle(p, radius: 2pt, fill: blue, stroke: none) } rect-around((.5, 1), (2, .25), (1, 2), (1.25, .25), (1.75, 1.5), stroke: 2pt + red) </pre>
	<pre> for (n, p) in (a: (.5, 1), b: (2, .25), c: (1, 2), d: (1.25, .25), e: (1.75, 1.5)) { content(p, [#n], name: n, frame: "circle", padding: .03, stroke: blue) } rect-around("a", "b", "c", "d", "e", stroke: 2pt + red) </pre>

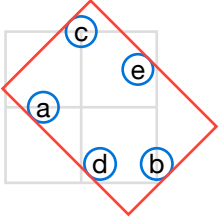
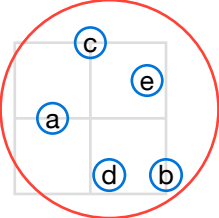
rect-around("a", "b", "c", "d", "e", VALUE)

		
stroke: red	padding: .3, stroke: red	padding: .3, radius: .3, stroke: red

4.12.1 Anchors of the fit

<pre> rect-around("a", "b", "c", "d", "e", name: "f", stroke: red) content("f.VALUE", [x], frame: "rect", stroke: none, fill: lime) </pre>		
		
east	north-east	center

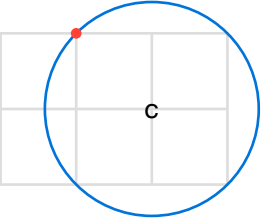
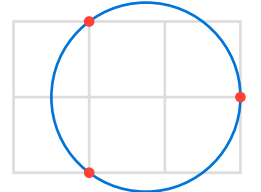
4.12.2 Rotated or round

	<pre>rotate(45deg) rect-around("a", "b", "c", "d", "e", stroke: red)</pre>
	<pre>rect-around("a", "b", "c", "d", "e", name: "f", stroke: none) circle("f.center", "f.north-east", stroke: red)</pre>

Other frame shapes are in Section 6.

4.13 Circle through points

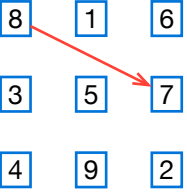
CeTZ manual: circle

	<pre>content((2, 1), [c]) circle((2, 1), (1, 2), stroke: blue) circle((1, 2), radius: 2pt, fill: red, stroke: none)</pre>
	<pre>let (a, b, c) = ((1, 0), (3, 1), (1, 2)) circle-through(a, b, c, stroke: blue) for p in (a, b, c) { circle(p, radius: 2pt, fill: red, stroke: none) }</pre>

4.14 Matrices

CeTZ manual: anchor coordinates

Not in CeTZ: `matrix`, `matrix of nodes`. Nested for loops place the cells and name them. For pure alignment, a Typst table or `$mat(..)$` inside one `content` is simpler.

	<pre>let m = ((8, 1, 6), (3, 5, 7), (4, 9, 2)) for (i, row) in m.enumerate() { for (j, v) in row.enumerate() { content((j, -i), [#v], name: "m-" + str(i) + "-" + str(j), frame: "rect", padding: .1, stroke: blue) } } line("m-0-0", "m-1-2", stroke: red, mark: (end: "straight"))</pre>
---	--

4.14.1 Cell styles

```
for (i, row) in ((8, 1, 6), (3, 5, 7), (4, 9, 2)).enumerate() {  
  for (j, v) in row.enumerate() {  
    let hit = VALUE  
    content((j * .6, -i * .6), text(if hit { red } else { blue })[#v])  
  }  
}
```

8 1 6 3 5 7 4 9 2 i == 1	8 1 6 3 5 7 4 9 2 j == 1	8 1 6 3 5 7 4 9 2 i == 1 and j == 1	8 1 6 3 5 7 4 9 2 calc.odd(i + j)
---	---	--	--

4.14.2 Alignment in a column

```
for (i, row) in (("12345", "67890"), ("123", "67"), ("1", "6")).enumerate() {  
  for (j, v) in row.enumerate() {  
    content((j * 1.6, -i * .5), v, anchor: VALUE)  
  }  
}
```

12345 67890 123 67 1 6 "west"	12345 67890 123 67 1 6 "east"	12345 67890 123 67 1 6 "center"
--	--	--

4.14.3 Spacing

```
for (i, row) in ((1, 2, 3), (4, 5, 6)).enumerate() {  
  for (j, v) in row.enumerate() {  
    let (dx, dy) = VALUE  
    content((j * dx, -i * dy), [#v], frame: "rect", padding: .1, stroke: blue)  
  }  
}
```

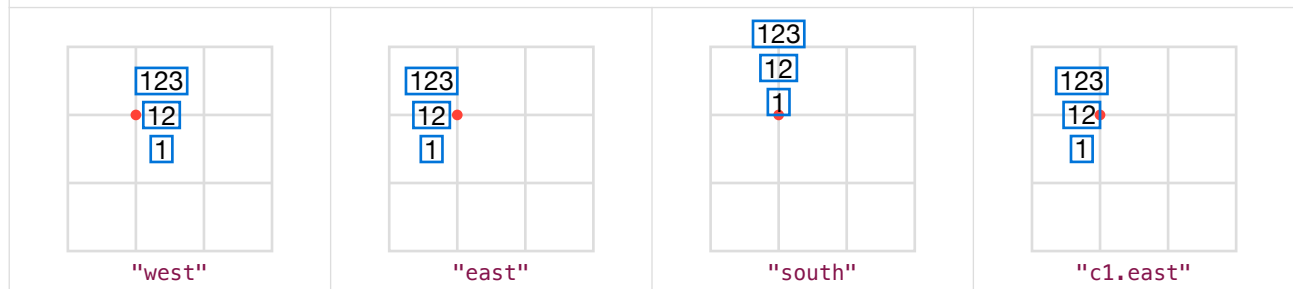
1 2 3 4 5 6 (.6, .6)	1 2 3 4 5 6 (1.2, .6)	1 2 3 4 5 6 (.6, 1.2)
--	---	---

4.14.4 Anchoring

CeTZ manual: group

The group's anchor moves onto its "default" anchor.

```
circle((1, 1), radius: 2pt, fill: red, stroke: none)
group(anchor: VALUE, {
  anchor("default", (1, 1))
  for (i, v) in ("123", "12", "1").enumerate() {
    content((1, 1 - i * .5), v, name: "c" + str(i),
      frame: "rect", padding: .05, stroke: blue)
  }
})
```



4.14.5 Empty cells

	<pre>let m = ((\$a_1\$, none, \$a_3\$), (\$a_4\$, none, \$a_6\$), (\$a_7\$, \$a_8\$, none)) for (i, row) in m.enumerate() { for (j, v) in row.enumerate() { if v != none { content((j * .8, -i * .8), v, frame: "circle", padding: .05, stroke: blue) } } }</pre>
$\begin{array}{rcl} 1\ m^2 & 2\ m^2 & - \\ 4\ m^2 & - & 6\ m^2 \\ - & - & 9\ m^2 \end{array}$	<pre>let m = ((1, 2, none), (4, none, 6), (none, none, 9)) for (i, row) in m.enumerate() { for (j, v) in row.enumerate() { content((j * .9, -i * .5), if v == none [--] else [#v \$m^2\$]) } }</pre>

4.14.6 Tables and math matrices

<table border="1"> <tr><td>aaa</td><td>bbb</td></tr> <tr><td>ccc</td><td></td></tr> <tr><td>eee</td><td>fff</td></tr> </table>	aaa	bbb	ccc		eee	fff	<pre>content((0, 0), table(columns: (2cm, 2cm), stroke: blue, [aaa], [bbb], [ccc], [], [eee], [fff]))</pre>
aaa	bbb						
ccc							
eee	fff						
<table border="1"> <tr><td>1</td><td>aaa bbb ccc</td></tr> <tr><td>2</td><td>ddd</td></tr> </table>	1	aaa bbb ccc	2	ddd	<pre>content((0, 0), table(columns: (1cm, 2cm), stroke: blue, [1], [aaa \ bbb \ ccc], [2], [ddd]))</pre>		
1	aaa bbb ccc						
2	ddd						

```
content((0, 0), $mat(delim: VALUE, a_1, a_2, a_3; a_4, a_5, a_6; a_7, a_8, a_9)$)
```

$\begin{pmatrix} a_1 & a_2 & a_3 \\ a_4 & a_5 & a_6 \\ a_7 & a_8 & a_9 \end{pmatrix}$ "{"	$\begin{bmatrix} a_1 & a_2 & a_3 \\ a_4 & a_5 & a_6 \\ a_7 & a_8 & a_9 \end{bmatrix}$ "["	$\left\{ \begin{array}{ccc} a_1 & a_2 & a_3 \\ a_4 & a_5 & a_6 \\ a_7 & a_8 & a_9 \end{array} \right\}$ "{"
--	--	--

<pre>content((0, 0), \$mat(delim: VALUE, a_1, a_2, a_3; a_4, a_5, a_6; a_7, a_8, a_9)\$)</pre>		
$\begin{vmatrix} a_1 & a_2 & a_3 \\ a_4 & a_5 & a_6 \\ a_7 & a_8 & a_9 \end{vmatrix}$ " "	$\left\ \begin{vmatrix} a_1 & a_2 & a_3 \\ a_4 & a_5 & a_6 \\ a_7 & a_8 & a_9 \end{vmatrix} \right\ $ " "	$\begin{vmatrix} a_1 & a_2 & a_3 \\ a_4 & a_5 & a_6 \\ a_7 & a_8 & a_9 \end{vmatrix}$ #none

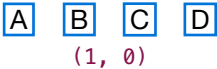
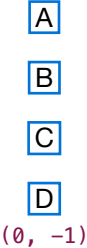
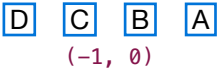
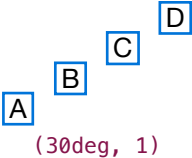
Braces beside any content are in Section 7.

Not in CeTZ: ampersand replacement. Typst code has no active &.

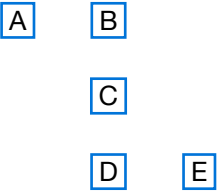
4.15 Chains

CeTZ manual: relative coordinates

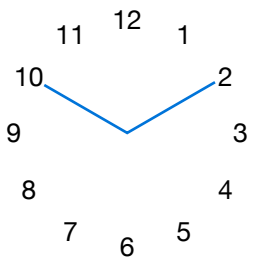
Not in CeTZ: chains library. A loop places each element relative to the previous one; line joins them by name.

<pre>set-style(content: (frame: "rect", padding: .1, stroke: blue)) content((0, 0), [A], name: "n0") for (i, t) in ("B", "C", "D").enumerate() { content((rel: VALUE, to: "n" + str(i)), t, name: "n" + str(i + 1)) }</pre>			
			

4.15.1 Changing direction

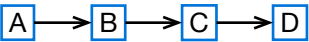
	<pre>set-style(content: (frame: "rect", padding: .1, stroke: blue)) let steps = ((1.2, 0), (0, -1), (0, -1), (1.2, 0)) content((0, 0), [A], name: "n0") for (i, s) in steps.enumerate() { content((rel: s, to: "n" + str(i)), "BCDE".at(i), name: "n" + str(i + 1)) }</pre>
---	---

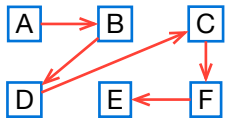
4.15.2 Placed on a curve

	<pre>for i in range(1, 13) { content((90deg - i * 30deg, 1.5), [#i], name: "h" + str(i), frame: "circle", stroke: none) } line("h10", (0, 0), "h2", stroke: blue)</pre>
---	---

With stroke: none the frame is invisible but still stops the line.

4.15.3 Joining

	<pre>set-style(content: (frame: "rect", padding: .1, stroke: blue)) let names = ("A", "B", "C", "D") for (x, n) in names.enumerate() { content((x * 1.2, 0), n, name: n) } for (a, b) in names.windows(2) { line(a, b, mark: (end: "straight")) }</pre>
---	---



```
set-style(content: (frame: "rect", padding: .1, stroke: blue))
for (i, n) in ("a", "b", "c", "d", "e", "f").enumerate() {
  content((calc.rem(i, 3) * 1.2, -calc.quo(i, 3)), upper(n), name: n)
}
for (a, b) in ("a", "b", "d", "c", "f", "e").windows(2) {
  line(a, b, stroke: red, mark: (end: "straight"))
}
```

4.15.4 Branches

A

B

C

1

2

3

```
set-style(content: (frame: "rect", padding: .1, stroke: blue))
for (x, n) in ("A", "B", "C").enumerate() { content((x * 1.2, 0), n, name: n) }
for y in range(1, 4) {
  content((rel: (0, -y * .8), to: "B"), [#y], name: "B-" + str(y))
}
line("C", "B-1")
line("C", "B-3")
```

For trees, see Section 9.

5 Groups, layers, and styles

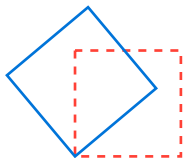
5.1 Transformations

[CeTZ manual: rotate](#)

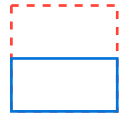
[CeTZ manual: scale](#)

[CeTZ manual: translate](#)

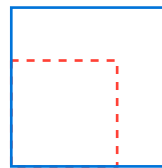
```
rect((0, 0), (2, 2), stroke: (paint: red, dash: "dashed"))
VALUE
rect((0, 0), (2, 2), stroke: blue)
```



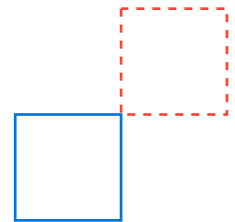
rotate(40deg)



scale(y: 50%)



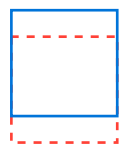
scale(1.5)



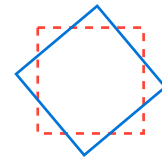
scale(-1)



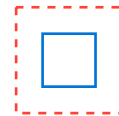
translate(x: .5)



translate(y: .5)



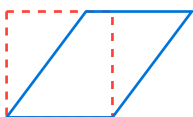
rotate(40deg, origin: (1, 1))



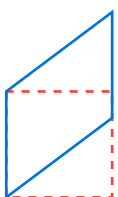
scale(50%, origin: (1, 1))

5.1.1 Slant

[CeTZ manual: set-transform](#)



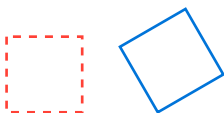
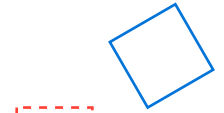
```
rect((0, 0), (2, 2), stroke: (paint: red, dash: "dashed"))
transform(cetz.matrix.transform-shear-x(.75))
rect((0, 0), (2, 2), stroke: blue)
```



```
rect((0, 0), (2, 2), stroke: (paint: red, dash: "dashed"))
transform(((1, 0, 0, 0), (.75, 1, 0, 0), (0, 0, 1, 0), (0, 0, 0, 1)))
rect((0, 0), (2, 2), stroke: blue)
```

5.1.2 Order

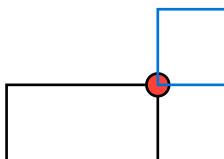
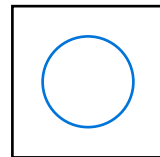
Each call multiplies onto the current transform, so later calls act first on the drawn shape.

	<pre>rect((0, 0), (1, 1), stroke: (paint: red, dash: "dashed")) translate(x: 2) rotate(30deg) rect((0, 0), (1, 1), stroke: blue)</pre>
	<pre>rect((0, 0), (1, 1), stroke: (paint: red, dash: "dashed")) rotate(30deg) translate(x: 2) rect((0, 0), (1, 1), stroke: blue)</pre>

5.1.3 New origin

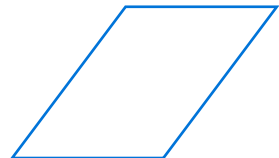
[CeTZ manual: set-origin](#)

[CeTZ manual: set-viewport](#)

	<pre>rect((0, 0), (2, 1), name: "r") set-origin("r.north-east") circle((0, 0), radius: .15, fill: red) rect((0, 0), (1, 1), stroke: blue)</pre>
	<pre>rect((0, 0), (2, 2), name: "r") set-viewport("r.south-west", "r.north-east", bounds: (10, 10)) circle((5, 5), radius: 3, stroke: blue)</pre>

5.1.4 Axis vectors of the canvas

[CeTZ manual: canvas](#)

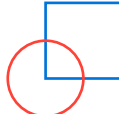
	<pre>#cetz.canvas(x: 1, y: .5, { rect((0, 0), (2, 2), stroke: blue) })</pre>
	<pre>#cetz.canvas(x: (1, 0), y: (.75, 1), { rect((0, 0), (2, 2), stroke: blue) })</pre>

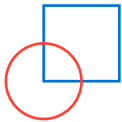
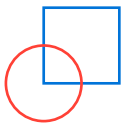
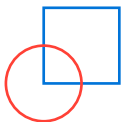
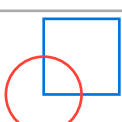
5.2 Placing the canvas

[CeTZ manual: canvas](#)

5.2.1 In the text

Without baseline, a canvas is a block. With baseline, it is an inline box and the given *y* (or coordinate) sits on the text baseline.

text before  text after	<pre>text before #cetz.canvas(length: 5mm, { rect((0, 0), (2, 2), stroke: blue) circle((0, 0), stroke: red) }) text after</pre>
--	---

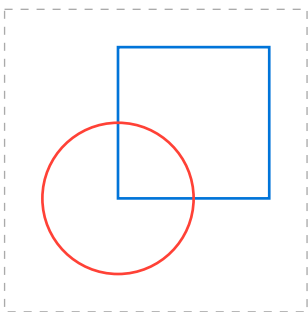
	<pre>text before #cetz.canvas(length: 5mm, baseline: (0, 0), { rect((0, 0), (2, 2), stroke: blue) circle((0, 0), stroke: red) }) text after</pre>
	<pre>text before #cetz.canvas(length: 5mm, baseline: (0, 1), { rect((0, 0), (2, 2), stroke: blue) circle((0, 0), stroke: red) }) text after</pre>
	<pre>text before #cetz.canvas(length: 5mm, baseline: "c.south", { rect((0, 0), (2, 2), stroke: blue) circle((0, 0), stroke: red, name: "c") }) text after</pre>
	<pre>text before #box(stroke: gray, inset: 3pt, cetz.canvas(length: 5mm, baseline: (0, 0), { rect((0, 0), (2, 2), stroke: blue) circle((0, 0), stroke: red) })) text after</pre>

5.2.2 Bounding box

[CeTZ manual: hide](#)

[CeTZ manual: floating](#)

Every element grows the canvas. `hide(..., bounds: true)` reserves space without drawing; `floating` draws without reserving space.

	<pre>rect((0, 0), (2, 1), stroke: red) line((-1, 0), (3, 1), stroke: blue)</pre>
	<pre>rect((0, 0), (2, 1), stroke: red) floating(line((-1, 0), (3, 1), stroke: blue))</pre>
	<pre>hide(rect((-1.5, -1.5), (2.5, 2.5)), bounds: true) rect((0, 0), (2, 2), stroke: blue) circle((0, 0), stroke: red)</pre>

5.2.3 Current bounding box

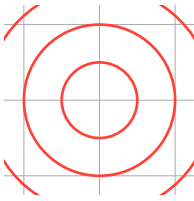
[CeTZ manual: rect-around](#)

	<pre>group(name: "pic", { circle((0, 0), radius: .15, fill: blue) circle((2, 1), radius: .15, fill: blue) }) rect("pic.south-west", "pic.north-east", stroke: red)</pre>
	<pre>circle((0, 0), radius: .15, fill: blue, name: "a") circle((2, 1), radius: .15, fill: blue, name: "b") rect-around("a", "b", padding: .1, stroke: red)</pre>

Not in CeTZ: trim left, trim right. Wrap parts that may stick out in floating.

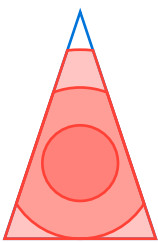
5.2.4 Clipping

Not in CeTZ: \clip. Clip the whole canvas with a Tpst box(clip: true); cut filled shapes with boolean(.., op: "intersection").



```
#box(clip: true, width: 2.5cm, height: 2.5cm,
align(center + horizon, cetz.canvas({
  grid((-2, -2), (2, 2), help-lines: true)
  for r in (.5, 1, 1.5) {
    circle((0, 0), radius: r, stroke: red)
  }
})))
```

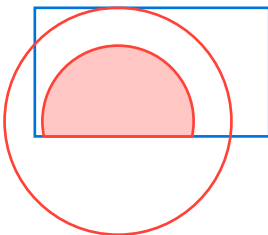
CeTZ manual: boolean



```
line((-1, -1), (0, 2), (1, -1), close: true, stroke: blue, name: "t")
for r in (.5, 1, 1.5) {
  boolean("t", circle((0, 0), radius: r), op: "intersection",
    fill: red.transparentize(70%), stroke: red)
}
```

5.2.5 Partial clipping

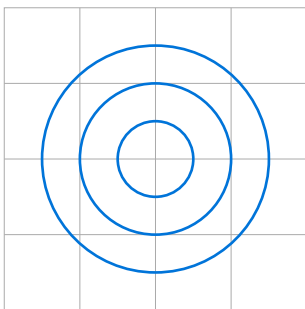
CeTZ manual: boolean



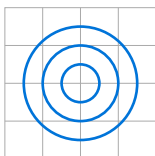
```
rect((-1.1, -0.2), (2, 1.5), stroke: blue, name: "r")
circle((0, 0), radius: 1.5, stroke: red)
boolean("r", circle((0, 0), radius: 1), op: "intersection",
  fill: red.lighten(70%), stroke: red)
```

5.2.6 Scaling

CeTZ manual: canvas



```
#cetz.canvas(length: 1cm, {
  grid((-2, -2), (2, 2), help-lines: true)
  for r in (.5, 1, 1.5) {
    circle((0, 0), radius: r, stroke: blue)
  }
})
```



```
#cetz.canvas(length: 5mm, {
  grid((-2, -2), (2, 2), help-lines: true)
  for r in (.5, 1, 1.5) {
    circle((0, 0), radius: r, stroke: blue)
  }
})
```

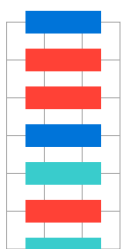
5.2.7 Absolute position on the page

Not in CeTZ: remember picture, overlay, current page anchors. Put the canvas in #set page(background: ..) or foreground, or position it with place(top + right, dx: .., dy: .., ..).

5.3 Groups and scopes

CeTZ manual: scope

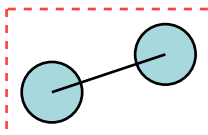
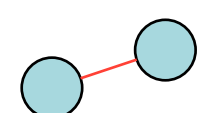
Style and transform changes inside scope or group end with it.

	<pre>grid((0, 0), (3, 6), help-lines: true) set-style(stroke: (thickness: 3mm, paint: blue)) line((.5, 6), (2.5, 6)) scope({ set-style(stroke: (paint: red)) line((.5, 5), (2.5, 5)) line((.5, 4), (2.5, 4)) }) line((.5, 3), (2.5, 3)) scope({ set-style(stroke: (paint: teal)) line((.5, 2), (2.5, 2)) line((.5, 1), (2.5, 1), stroke: (paint: red)) line((.5, 0), (2.5, 0)) })</pre>
---	---

5.3.1 Group or scope

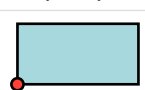
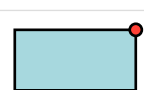
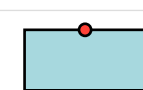
CeTZ manual: group

A group is one element with bounding-box anchors; names inside it are reached through it. A scope leaks its names.

	<pre>group(name: "g", padding: .2, { circle((0, 0), radius: .4, fill: eastern.lighten(60%), name: "a") circle((1.5, .5), radius: .4, fill: eastern.lighten(60%), name: "b") }) rect("g.south-west", "g.north-east", stroke: (paint: red, dash: "dashed")) line("g.a", "g.b")</pre>
	<pre>scope({ circle((0, 0), radius: .4, fill: eastern.lighten(60%), name: "a") }) circle((1.5, .5), radius: .4, fill: eastern.lighten(60%), name: "b") line("a", "b", stroke: red)</pre>

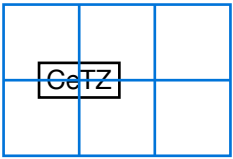
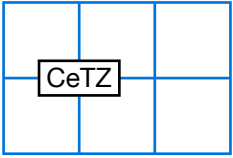
5.3.2 Placing a group by an anchor

The group moves so that the given anchor lands on its "default" anchor, here set to the origin.

<pre>group(anchor: VALUE, { anchor("default", (0, 0)) rect((0, 0), (2, 1), fill: eastern.lighten(60%)) }) circle((0, 0), radius: .1, fill: red)</pre>			
			
"south-west"	"north-east"	"center"	"north"

5.3.3 Layers

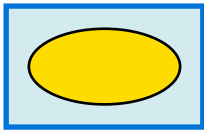
CeTZ manual: on-layer

	<pre>content((1, 1), [CeTZ], frame: "rect", fill: white, padding: .1) grid((0, 0), (3, 2), stroke: blue)</pre>
	<pre>content((1, 1), [CeTZ], frame: "rect", fill: white, padding: .1) on-layer(-1, grid((0, 0), (3, 2), stroke: blue))</pre>

5.4 Background

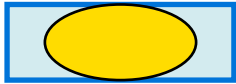
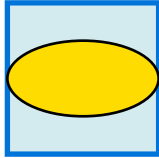
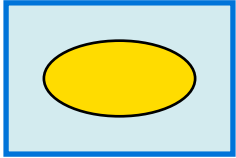
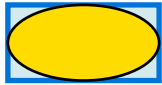
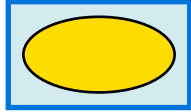
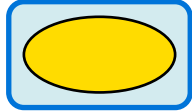
5.4.1 Framing

CeTZ manual: canvas

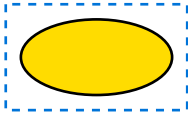
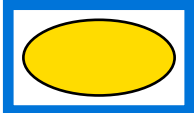
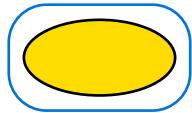
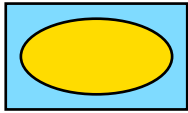
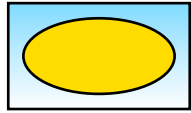
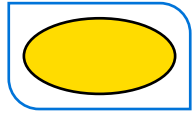
	<pre>#cetz.canvas(padding: .3, background: eastern.lighten(80%), stroke: 2pt + blue, { circle((0, 0), radius: (1, .5), fill: yellow) })</pre>
---	---

Inside the canvas, rect-around on a lower layer does the same and takes every rect style.

CeTZ manual: rect-around

<pre>circle((0, 0), radius: (1, .5), fill: yellow, name: "e") on-layer(-1, rect-around("e", VALUE, fill: eastern.lighten(80%), stroke: 2pt + blue))</pre>		
 padding: (x: .5)	 padding: (y: .5)	 padding: .5
 padding: 0	 padding: .2	 padding: .2, radius: .2

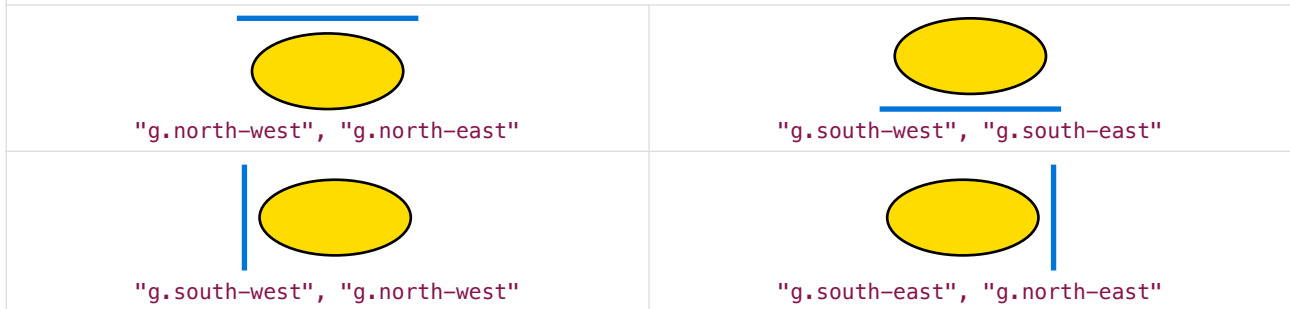
5.4.2 Frame style

<pre>circle((0, 0), radius: (1, .5), fill: yellow, name: "e") on-layer(-1, rect-around("e", padding: .2, VALUE))</pre>		
 stroke: (paint: blue, dash: "dashed")	 stroke: 4pt + blue	 radius: .5, stroke: blue
 fill: aqua	 fill: gradient.linear(aqua, white, angle: 90deg)	 radius: (north-east: .4, south-west: .4), stroke: blue

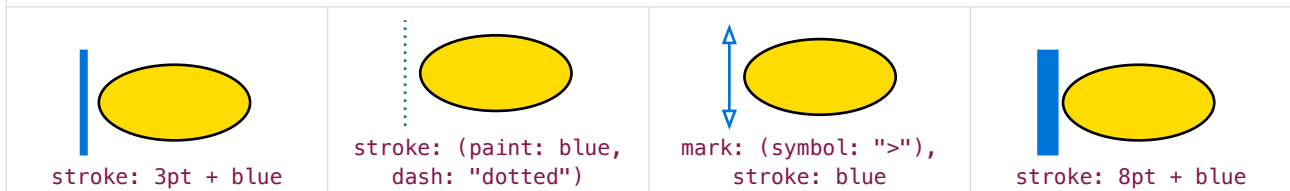
Not in CeTZ: double lines. Draw two rect-around with different padding.

5.4.3 Partial framing

```
group(name: "g", padding: .2,
  circle((0, 0), radius: (1, .5), fill: yellow))
line(VALUE, stroke: 2pt + blue)
```



```
group(name: "g", padding: .2,
  circle((0, 0), radius: (1, .5), fill: yellow))
line("g.south-west", "g.north-west", VALUE)
```



Lines past the corners: offset the ends with rel.

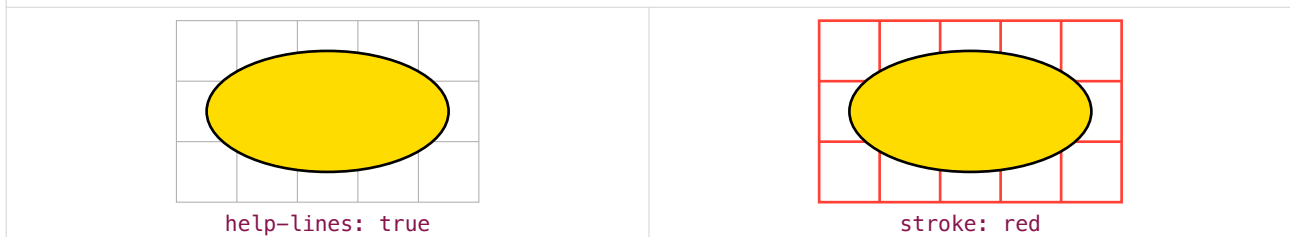
```
group(name: "g", padding: .2,
  circle((0, 0), radius: (1, .5), fill: yellow))
on-layer(-1, rect("g.south-west", "g.north-east",
  fill: eastern.lighten(80%), stroke: 2pt + blue))
line(VALUE, stroke: 2pt + blue)
```



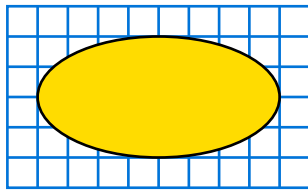
5.4.4 Gridding

CeTZ manual: grid

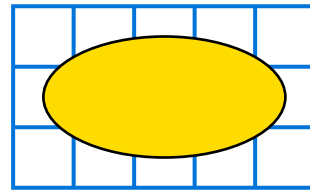
```
circle((0, 0), radius: (2, 1), fill: yellow)
on-layer(-1, grid((-2.5, -1.5), (2.5, 1.5), VALUE))
```



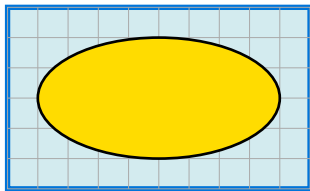
```
circle((0, 0), radius: (2, 1), fill: yellow)
on-layer(-1, grid((-2.5, -1.5), (2.5, 1.5), VALUE))
```



step: .5, stroke: blue



stroke: 1.5pt + blue



```
circle((0, 0), radius: (2, 1), fill: yellow)
on-layer(-1, {
  rect((-2.5, -1.5), (2.5, 1.5),
    fill: eastern.lighten(80%), stroke: 2pt + blue)
  grid((-2.5, -1.5), (2.5, 1.5), step: .5, help-lines: true)
})
```

5.5 Colors

5.5.1 Named colors

[Typst manual: color](#)

```
rect((0, 0), (1.6, .8), fill: VALUE)
```



black



gray



silver



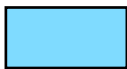
white



navy



blue



aqua



teal



eastern



purple



fuchsia



maroon



red



orange



yellow



olive



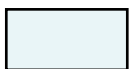
green



lime

5.5.2 Lighter and darker

```
rect((0, 0), (1.6, .8), fill: eastern.lighten(VALUE))
```



90%



70%



50%



30%



10%

```
rect((0, 0), (1.6, .8), fill: eastern.darken(VALUE))
```



10%



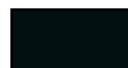
30%



50%



70%



90%

5.5.3 Mixing

```
rect((0, 0), (1.6, .8), fill: VALUE)
```



color.mix(red, eastern)



color.mix(red, eastern, space: rgb)



color.mix((red, 80%), (eastern, 20%))



color.mix(red, eastern, black)

5.5.4 Defining a color

<code>rect((0, 0), (1.6, .8), fill: VALUE)</code>		
 <code>rgb(75%, 50%, 25%)</code>	 <code>rgb("#bf8040")</code>	 <code>luma(60%)</code>
 <code>cmyk(0%, 60%, 100%, 20%)</code>	 <code>oklch(65%, 0.12, 250deg)</code>	 <code>color.hsl(200deg, 60%, 50%)</code>
	<code>let tan = rgb(75%, 50%, 25%) rect((0, 0), (2, 1), fill: tan)</code>	
	<code>let pale = red.lighten(75%) rect((0, 0), (2, 1), fill: pale)</code>	

5.6 Opacity

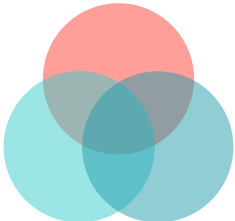
[Typst manual: transparentize](#)

<code>line((0, 0), (2, 1), stroke: 8pt + red) line((0, 1), (2, 0), stroke: 8pt + eastern.transparentize(VALUE))</code>				
 100%	 75%	 50%	 25%	 0%
<code>rect((0, 0), (1, 1), fill: red, stroke: none) rect(.5, 0), (1.5, 1), fill: eastern.transparentize(VALUE), stroke: none)</code>				
 90%	 75%	 50%	 25%	 0%
<code>content((0, 0), text(20pt, fill: blue.transparentize(VALUE))[text], frame: "rect", padding: .1)</code>				
 0%	 25%	 50%	 75%	 100%

Not in CeTZ: opacity for a whole element, named levels such as `semitransparent`. Make each paint transparent with `transparentize`.

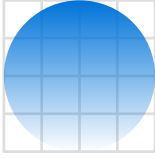
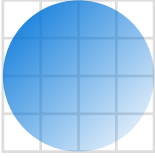
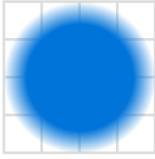
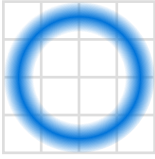
5.6.1 Blend modes

Not in CeTZ: `blend mode`, `blend group`. Overlap transparent fills, or compute the result with `color.mix`.

	<code>set-style(stroke: none) circle((90deg, .6), fill: red.transparentize(50%)) circle((210deg, .6), fill: teal.transparentize(50%)) circle((330deg, .6), fill: eastern.transparentize(50%))</code>
---	--

5.6.2 Fading

Not in CeTZ: `path fading`, `scope fading`, `\tikzfading`. Fill with a Typst gradient whose stops are transparent.

<pre>let c = blue let t = c.transparentize(100%) grid((-1, -1), (1, 1), step: .5, stroke: silver) circle((0, 0), radius: 1, stroke: none, fill: VALUE)</pre>		
 <code>gradient.linear(c, t, angle: 90deg)</code>	 <code>gradient.linear(t, c)</code>	 <code>gradient.linear(c, t, angle: 45deg)</code>
 <code>gradient.radial((c, 0%), (c, 70%), (t, 100%))</code>	 <code>gradient.radial((t, 0%), (t, 60%), (c, 80%), (t, 100%))</code>	 <code>gradient.radial(c, t, center: (30%, 30%))</code>

[Typst manual: gradient](#)

	<pre>let c = red content((0, 0), text(28pt, weight: "bold", fill: gradient.linear(c, c.transparentize(100%)))[CeTZ], frame: "rect", stroke: blue, padding: .1)</pre>
--	--

5.6.3 Transparency group

Not in CeTZ: `transparency group`. One path gets one opacity: put the parts in a compound-path.

	<pre>grid((0, 0), (3, 3), help-lines: true) set-style(stroke: 8mm + eastern.transparentize(50%)) line((.5, .5), (2.5, 2.5)) line((.5, 2.5), (2.5, .5))</pre>
	<pre>grid((0, 0), (3, 3), help-lines: true) compound-path({ line((.5, .5), (2.5, 2.5)) line((.5, 2.5), (2.5, .5)) }, stroke: 8mm + eastern.transparentize(50%))</pre>

5.7 Own commands

[Typst manual: function](#)

A Typst function that returns draw elements works like a TikZ command.

	<pre>let flag(pos) = group({ set-origin(pos) for (i, c) in (blue, white, red).enumerate() { rect((i * .5, 0), (rel: (.5, 1)), fill: c) } }) flag((0, 0)) flag((2, .5))</pre>
---	--

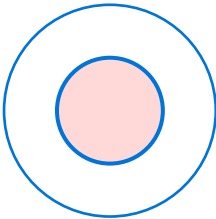
contents more	<pre>let tag(pos, body) = content(pos, body, frame: "rect", fill: yellow, padding: .15) tag((0, 0))[contents] tag((2, 0))[more]</pre>
-------------------------------------	---

The flag  sits in the text.	<pre>#let flag = cetz.canvas(length: 3mm, baseline: (0, 0), { for (i, c) in (blue, white, red).enumerate() { rect((i, 0), (rel: (1, 1.5)), fill: c) } }) The flag #flag sits in the text.</pre>
---	---

5.8 Own styles

CeTZ manual: set-style

A style is a dictionary of named arguments; spread it with ...

	<pre>let mine = (stroke: 1.5pt + blue, fill: red.lighten(80%)) circle((0, 0), radius: 2, stroke: blue) circle((0, 0), ..mine)</pre>
	<pre>let mine(c) = (stroke: c, fill: c.lighten(50%)) rect((0, 0), (2, 1), ..mine(red)) rect((3, 0), (5, 1), ..mine(eastern))</pre>
	<pre>let mine(c: gray) = (stroke: c, fill: c.lighten(50%)) rect((0, 0), (2, 1), ..mine()) rect((3, 0), (5, 1), ..mine(c: eastern))</pre>

5.8.1 Styles per element type

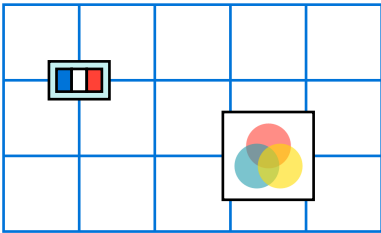
CeTZ manual: styling

	<pre>set-style(stroke: blue, rect: (fill: eastern.lighten(70%)), circle: (fill: red.lighten(70%), stroke: (dash: "dashed"))) rect((0, 0), (1, 1)) circle((2, .5), radius: .5) rect((3, 0), (4, 1))</pre>
---	--

5.9 Pictures in a canvas

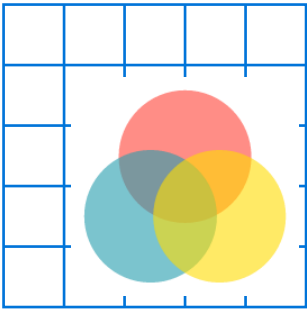
CeTZ manual: content

5.9.1 In a content element

	<pre>grid((0, 0), (5, 3), stroke: blue) let flag = cetz.canvas(length: 2mm, { for (i, c) in (blue, white, red).enumerate() { rect((i, 0), (rel: (1, 1.5)), fill: c) } }) content((1, 2), flag, frame: "rect", padding: .1, fill: teal.lighten(70%)) content((3.5, 1), image("/assets/picture.png", width: 1cm), frame: "rect", fill: white, padding: .1)</pre>
---	--

5.9.2 Image declared once

[Typst manual: image](#)



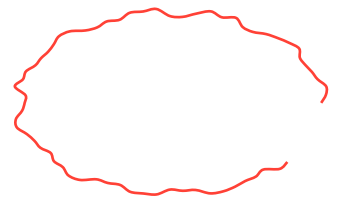
```
let cube = image("/assets/picture.png", width: 3cm)
grid((0, 0), (5, 5), stroke: blue)
content((3, 2), cube)
```

5.10 Freehand drawing

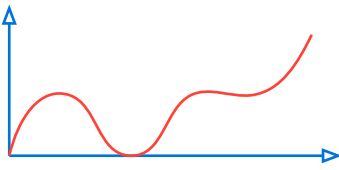
Not in CeTZ: random steps decoration. Jitter sampled points and draw a catmull curve through them, or a smooth hobby curve through hand-placed points.

[CeTZ manual: catmull](#)

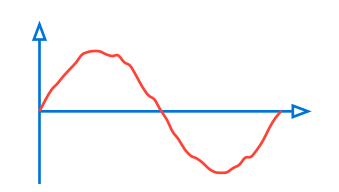
[CeTZ manual: hobby](#)



```
let pts = range(65).map(i => {
  let r = 1 + .05 * calc.fract(calc.sin(i) * 1e4) // jitter
  (2.5 * r * calc.cos(i * 5deg), 1.5 * r * calc.sin(i * 5deg))
})
catmull(..pts, stroke: red)
```



```
line((0, 0), (5.5, 0), mark: (end: ">"), stroke: blue)
line((0, 0), (0, 2.5), mark: (end: ">"), stroke: blue)
hobby((0, 0), (1, 1), (2, 0), (3, 1), (4, 1), (5, 2),
  stroke: red)
```



```
line((0, 0), (4.5, 0), mark: (end: ">"), stroke: blue)
line((0, -1.2), (0, 1.5), mark: (end: ">"), stroke: blue)
let pts = range(41).map(i => {
  let x = i / 10
  (x, calc.sin(x * 90deg) + .05 * calc.fract(calc.sin(i) * 1e4))
})
catmull(..pts, stroke: red)
```

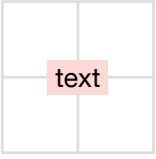
6 Shapes and text

CeTZ has no node shapes. Draw the shape, then place content on one of its anchors.

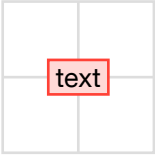
6.1 Framed content

[CeTZ manual: content](#)

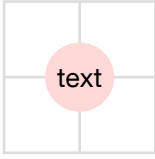
```
grid((0, 0), (2, 2), stroke: silver)
content((1, 1), [text], padding: .1, fill: red.lighten(80%), VALUE)
```



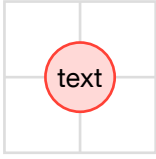
frame: "rect", stroke: none



frame: "rect", stroke: red



frame: "circle", stroke: none



frame: "circle", stroke: red

fill and stroke apply only when frame is set.

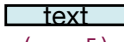
6.1.1 Frame style

<code>content((0, 0), [text], frame: "rect", padding: .1, VALUE)</code>		
 stroke: 2pt + blue	 stroke: (dash: "dashed")	 stroke: red
 angle: 45deg	 fill: gradient.radial(aqua, white)	 wrap: text.with(red)

content frames have no corner radius and no double stroke. Frame the named element with `rect-around`:

	<code>content((0, 0), [text], name: "t", padding: .1) rect-around("t", radius: .1, stroke: blue)</code>
	<code>content((0, 0), [text], name: "t", padding: .1) rect-around("t", stroke: blue) rect-around("t", padding: .06, stroke: blue)</code>

6.1.2 Padding

<code>content((0, 0), [text], frame: "rect", fill: eastern.lighten(80%), padding: VALUE)</code>			
 0	 .1	 .5	 (x: .5)
 (y: .5)	 (top: 1)	 (bottom: 1)	 (.1, .6, .3, 0)

The default padding is 0. top and bottom replace TikZ text height and text depth. Arrays follow CSS order: (y, x), (top, x, bottom), (top, right, bottom, left). Dictionaries take top, bottom, left, right, x, y, rest.

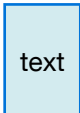
6.1.3 Outer separation

Not in CeTZ: outer sep: anchors always lie on the frame. Move the coordinate away from the anchor instead, e.g. ("a.east", -.3, "a.center") is 0.3 beyond a.east.

	<code>content((0, 0), [text], name: "a", frame: "rect", padding: .1) line(("a.east", -.3, "a.center"), (rel: (1, 0)), mark: (end: ">")) line(("a.south", -.3, "a.center"), (rel: (0, -.6)), mark: (end: ">"))</code>
---	--

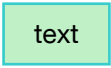
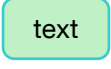
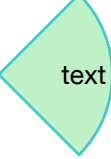
6.1.4 Fixed size

Two coordinates set the frame; the body fills it. A circle of given size is a circle with content on top.

<code>content((0, 0), (rel: VALUE), align(center + horizon)[text], frame: "rect", fill: eastern.lighten(80%), stroke: blue)</code>		
 (1, 1.5)	 (3, .5)	 (1.5, 1.5)
	<code>circle((0, 0), radius: .75, name: "c", fill: eastern.lighten(80%), stroke: blue) content("c", [text])</code>	

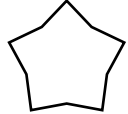
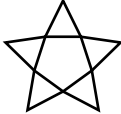
6.2 Built-in shapes

CeTZ manual: shapes

<pre>set-style(fill: green.lighten(70%), stroke: teal) VALUE content((0, 0), [text])</pre>		
 <code>rect((-0.7, -0.4), (0.7, 0.4))</code>	 <code>rect((-0.7, -0.4), (0.7, 0.4), radius: 0.15)</code>	 <code>circle((0, 0), radius: 0.6)</code>
 <code>circle((0, 0), radius: 0.9, 0.45)</code>	 <code>polygon((0, 0), 4, radius: 0.9, 0.6)</code>	 <code>polygon((0, 0), 5, radius: 0.75, angle: 90deg)</code>
 <code>n-star((0, 0), 5, radius: 0.9, angle: 54deg)</code>	 <code>arc((0, 0), start: 0deg, stop: 180deg, radius: 0.9, mode: "CLOSE", anchor: "center")</code>	 <code>arc((0, 0), start: -45deg, stop: 45deg, radius: 1.5, mode: "PIE", anchor: "center")</code>

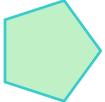
6.2.1 Star

CeTZ manual: n-star

<pre>n-star((0, 0), VALUE, radius: 0.8, fill: green.lighten(70%), stroke: teal)</pre>			
 4	 5	 6	 8
<pre>n-star((0, 0), 5, angle: 54deg, radius: 0.8, VALUE)</pre>			
 inner-radius: 20%	 inner-radius: 50%	 inner-radius: 70%	 show-inner: true

6.2.2 Regular polygon

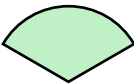
CeTZ manual: polygon

<pre>polygon((0, 0), VALUE, radius: 0.7, fill: green.lighten(70%), stroke: teal)</pre>					
 3	 4	 5	 6	 7	 8

<pre> polygon((0, 0), 4, radius: VALUE, fill: green.lighten(70%), stroke: teal) content((0, 0), [text]) </pre>			

6.2.3 Arc, segment and sector

CeTZ manual: arc

<pre>arc((0, 0), start: 30deg, stop: 150deg, fill: green.lighten(70%), mode: VALUE)</pre>		
		
"OPEN" "CLOSE" "PIE"		

<pre>arc((0, 0), start: 0deg, delta: VALUE, radius: .8, mode: "PIE", fill: green.lighten(70%))</pre>			
			
45deg 90deg 180deg 270deg			

6.2.4 Rounded rectangle

CeTZ manual: rect

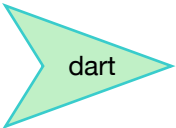
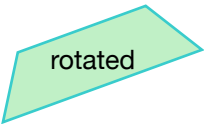
<pre> rect((0, 0), (2, 1), fill: green.lighten(70%), radius: VALUE) </pre>			

6.3 Shapes from lines

CeTZ manual: line

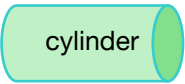
A closed line is any polygon; its centroid anchor centers the text. The examples in this and the next sections start with set-style(fill: green.lighten(70%), stroke: teal).

	<pre> line((-1.4, -.4), (1.4, -.4), (.9, .4), (-.9, .4), close: true, name: "s") content("s.centroid", [trapezium]) </pre>
	<pre> line((-.8, -1), (1.6, 0), (-.8, 1), close: true, name: "s") content("s.centroid", [triangle]) </pre>
	<pre> line((0, .5), (.8, 0), (0, -1.2), (-.8, 0), close: true, name: "s") content("s.centroid", [kite]) </pre>

	<pre>line((1.4, 0), (-.8, .8), (-.3, 0), (-.8, -.8), close: true) content((.35, 0), [dart])</pre>
	<pre>polygon((0, 0), 8, radius: (1.3, .6), angle: 22.5deg) content((0, 0), [chamfered])</pre>
	<pre>boolean(op: "difference", { rect((-1, -.4), (1, .4)) }, { circle((-1, 0), radius: .3) }) content((.15, 0), [concave])</pre>
	<pre>rotate(20deg) line((-1.4, -.4), (1.4, -.4), (.9, .4), (-.9, .4), close: true, name: "s") content("s.centroid", [rotated])</pre>

6.3.1 Cylinder and tape

CeTZ manual: merge-path

	<pre>merge-path(close: true, { line((1, .5), (-1, .5)) arc(), start: 90deg, delta: 180deg, radius: (.2, .5)) line(), (1, -.5)) }) circle((1, 0), radius: (.2, .5), fill: green.lighten(40%)) content((0, 0), [cylinder])</pre>
	<pre>merge-path(close: true, { bezier((-1, .4), (1, .4), (-.4, .7), (.4, .1)) line(), (1, -.4)) bezier(), (-1, -.4), (.4, -.7), (-.4, -.1)) }) content((0, 0), [tape])</pre>

6.4 Symbol shapes

CeTZ manual: boolean

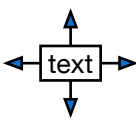
	<pre>circle((0, 0), radius: .6, name: "c") line("c.north-east", "c.south-west") content((0, 0), [text])</pre>
	<pre>circle((0, 0), radius: .6, name: "g") line("g.south-east", (rel: (.6, -.6)), stroke: 3pt + teal) content((0, 0), [text])</pre>
	<pre>boolean(op: "union", { circle((0, 0), radius: (1.2, .6)) }, { for i in range(10) { circle((angle: i * 36deg, radius: (1.2, .6)), radius: .3) } }) content((0, 0), [cloud])</pre>
	<pre>n-star((0, 0), 17, radius: 1.2, inner-radius: 75%) content((0, 0), [starburst])</pre>
	<pre>line((-1, -.4), (.7, -.4), (1, 0), (.7, .4), (-1, .4), close: true) content((-1, 0), [signal])</pre>

Not in CeTZ: random starburst; cloud puffs and similar shape options. Compute the points in a loop, as for the cloud.

6.5 Arrow shapes

	<pre>line((0, -.25), (1.4, -.25), (1.4, -.5), (2, 0), (1.4, .5), (1.4, .25), (0, .25), close: true) content((.7, 0), [text])</pre>
	<pre>line((-1, 0), (-.5, .5), (-.5, .25), (.5, .25), (.5, .5), (1, 0), (.5, -.5), (.5, -.25), (-.5, -.25), (-.5, -.5), close: true) content((0, 0), [text])</pre>

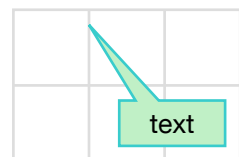
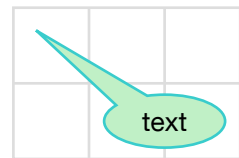
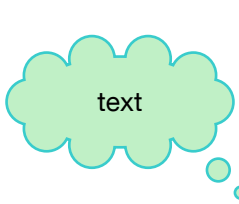
Not in CeTZ: arrow box. Frame the content and draw a line with a mark from each side.

	<pre>content((0, 0), [text], name: "t", frame: "rect", padding: .1) for a in ("north", "south", "east", "west") { line("t." + a, "t." + a, -.5, "t.center"), mark: (end: "triangle", fill: blue) }</pre>
---	--

6.6 Callouts

CeTZ manual: boolean

The pointer is a triangle joined to the body with `boolean(..., op: "union")`. Its tip is an absolute coordinate.

	<pre>grid((0, 0), (3, 2), stroke: silver) boolean(op: "union", { rect((1.4, .2), (2.8, .8)) }, { line((1.8, .5), (2.2, .5), (1, 1.8), close: true) }) content((2.1, .5), [text])</pre>
	<pre>grid((0, 0), (3, 2), stroke: silver) boolean(op: "union", { circle((2, .5), radius: (.8, .35)) }, { line((1.8, .3), (2, .7), (.3, 1.7), close: true) }) content((2, .5), [text])</pre>
	<pre>boolean(op: "union", { circle((0, 0), radius: (1.2, .6)) }, { for i in range(10) { circle((angle: i * 36deg, radius: (1.2, .6)), radius: .3) } }) content((0, 0), [text]) circle((1.3, -.9), radius: .15) circle((1.6, -1.2), radius: .08)</pre>

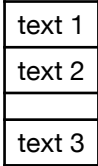
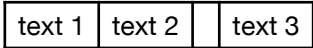
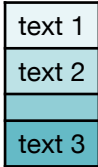
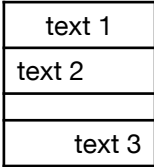
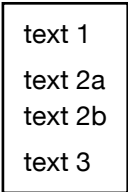
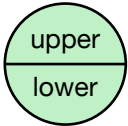
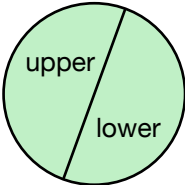
6.7 Cross out and strike out

	<pre>content((0, 0), [text], name: "t") line("t.north-west", "t.south-east", stroke: red) line("t.south-west", "t.north-east", stroke: red)</pre>
	<pre>content((0, 0), [text], name: "t") line("t.south-west", "t.north-east", stroke: red)</pre>

6.8 Multi-part content

CeTZ manual: content

Not in CeTZ: rectangle split, circle split, \nodepart. Put a Typst table in content, or split a shape with a line.

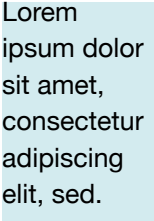
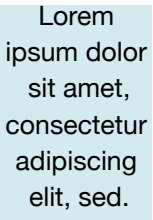
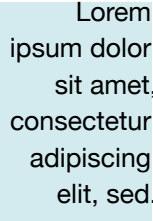
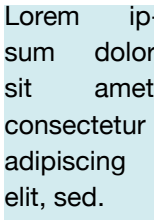
<pre>content((0, 0), table(VALUE, [text 1], [text 2], [], [text 3]))</pre>		
 columns: 1	 columns: 4	text 1 text 2 text 3 columns: 1, stroke: none
 columns: 1, rows: (auto, 1cm)	 columns: 1, fill: (_, y) => eastern.lighten(90% - y * 20%)	 columns: 2cm, align: (_, y) => (center, left, right, right).at(y)
	<pre>content((0, 0), frame: "rect", padding: .1, table(columns: 1, stroke: none, [text 1], [text 2a \ text 2b], [text 3]))</pre>	
	<pre>circle((0, 0), radius: .8, name: "c", fill: green.lighten(70%)) line("c.west", "c.east") content((0, .3), [upper]) content((0, -.3), [lower])</pre>	
	<pre>circle((0, 0), radius: 1.2, name: "c", fill: green.lighten(70%)) line((name: "c", anchor: 70deg), (name: "c", anchor: 250deg)) content((- .45, .4), [upper]) content((.45, -.45), [lower])</pre>	

6.9 Text attributes

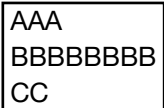
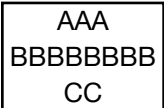
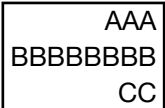
CeTZ manual: content

6.9.1 Width and alignment

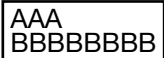
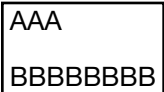
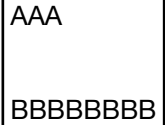
```
content((0, 0), box(width: 2cm, VALUE),
  frame: "rect", fill: eastern.lighten(80%), stroke: none)
```

 lorem(9)	 align(center, lorem(9))	 align(right, lorem(9))	 par(justify: true, lorem(9))
---	--	--	---

```
content((0, 0), align(VALUE)[AAA \ BBBBBBBB \ CC], frame: "rect", padding: .1)
```

 left	 center	 right
---	---	--

```
content((0, 0), par(leading: VALUE)[AAA \ BBBBBBBB], frame: "rect", padding: .1)
```

 0.2em	 0.5cm	 1cm
--	--	---

6.9.2 Colors and fonts

```
content((0, 0), text(VALUE)[Text.])
```

Text. red	Text. style: "italic"	Text. weight: "bold"	Text. font: "DejaVu Sans Mono"	Text. tracking: 2pt	Text. stroke: .3pt + blue
--------------	-----------------------------	-------------------------	--------------------------------------	------------------------	---------------------------------

```
content((0, 0), text(size: VALUE)[Text.])
```

Text. 6pt	Text. 8pt	Text. 10pt	Text. 14pt	Text. 20pt	Text. 28pt
--------------	--------------	---------------	---------------	---------------	---------------

The wrap style applies a function to every content body:

one two	<pre>set-style(content: (wrap: text.with(red, weight: "bold"))) content((0, 0), [one]) content((1, 0), [two])</pre>
---------	---

6.10 Anchors

CeTZ manual: anchors

6.10.1 Content anchors

```
content((0, 0), text(20pt)[text], name: "t", frame: "rect", padding: .2)
circle((name: "t", anchor: VALUE), radius: .07, fill: red, stroke: none)
```

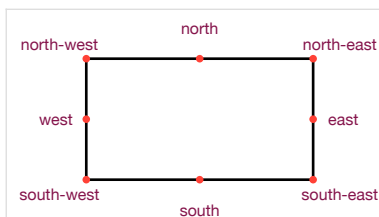
 "north-west"	 "north"	 "north-east"	 "text"
 "west"	 "mid-west"	 "base-west"	 "base"
 "east"	 "mid-east"	 "base-east"	 "mid"
 "south-west"	 "south"	 "south-east"	 "center"
 0deg	 120deg	 -60deg	 10%

An angle is a border anchor: the ray from center at that angle meets the frame. A ratio or number is a path anchor along the frame, starting at north-west. A string anchor can also be written "t.north".

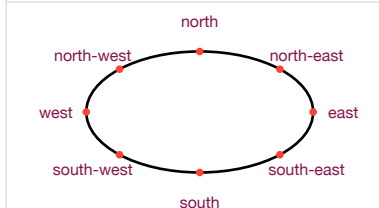
6.10.2 Shape anchors

The maps below use this helper:

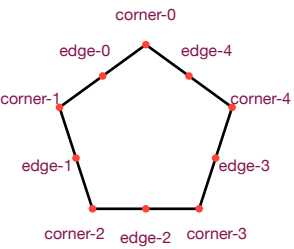
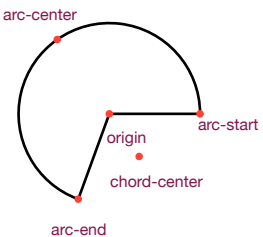
```
let compass = ("north", "north-east", "east", "south-east",
  "south", "south-west", "west", "north-west")
let mark-anchors(s, names) = for n in names {
  circle(s + "." + n, radius: .05, fill: red, stroke: none)
  content((s + "." + n, -.4, s + ".center"), text(6pt, fill: maroon, n))
}
```



```
rect((0, 0), (3, 1.6), name: "s")
mark-anchors("s", compass)
```



```
circle((0, 0), radius: (1.5, .8), name: "s")
mark-anchors("s", compass)
```

	<pre> polygon((0, 0), 5, radius: 1.2, angle: 90deg, name: "s") for i in range(5) { mark-anchors("s", ("corner-" + str(i), "edge-" + str(i))) } </pre>
	<pre> line((-1.4, -.4), (1.4, -.4), (.9, .4), (-.9, .4), close: true, name: "s") for n in ("start", "mid", "centroid") { circle("s." + n, radius: .05, fill: red, stroke: none) content("s." + n, text(6pt, fill: maroon, n), anchor: "south", padding: .08) } </pre>
	<pre> arc((0, 0), start: 0deg, stop: 250deg, radius: 1.2, mode: "PIE", name: "s") mark-anchors("s", ("arc-start", "arc-end", "arc-center", "chord-center", "origin")) </pre>

On rect and circle, an angle anchor is scaled by the aspect ratio, so 45deg (north-east) is a corner of any rect. On content the angle is exact.

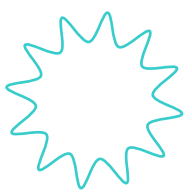
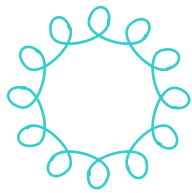
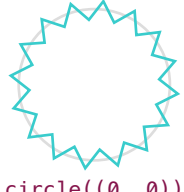
Not in CeTZ: n-star anchors corner-i and edge-i: listed, but they panic in 0.5.2. Use compass or angle border anchors, or polar coordinates from the star's origin.

7 Decorations

A path decoration takes an element as its target and draws a new path along it; the target itself is not drawn. To also draw the original path, store the element in a variable and use it twice. The pictures below draw the original path in grey.

7.1 Path morphing

CeTZ manual: path decorations

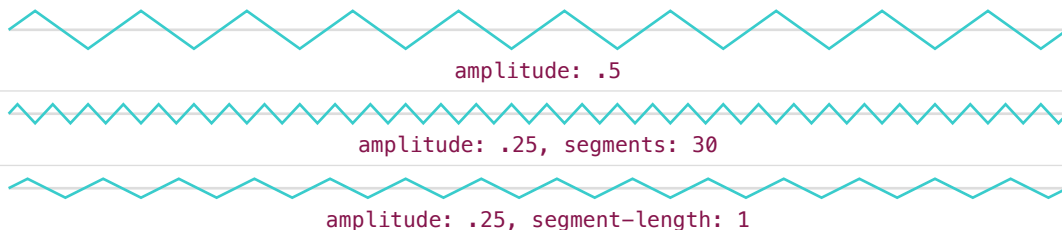
cetz.decorations.VALUE(circle((0, 0)), amplitude: .4, segments: 12)			
 zigzag	 wave	 coil	 square
<pre> set-style(stroke: silver) let p = VALUE p cetz.decorations.zigzag(p, amplitude: .25, segments: 16, stroke: teal) </pre>			
 line((0, 0), (2, 2))	 circle((0, 0))	 arc((0, 0), start: 0deg, stop: 180deg, radius: (1.5, 1))	 bezier((0, 0), (3, 0), (1, 2))

Every decoration takes segments (default 10) or segment-length, and amplitude (default 1, measured peak to peak). segment-length is rounded down to whole segments, which are then stretched to fill the path; the align key has no effect in 0.5.2.

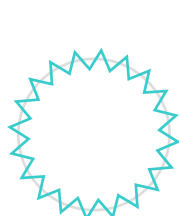
7.1.1 Zigzag

CeTZ manual: zigzag

```
let p = line((0, 0), (14, 0), stroke: silver)
p
cetz.decorations.zigzag(p, VALUE)
```



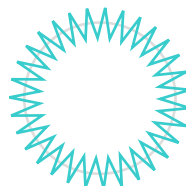
```
let p = circle((0, 0), stroke: silver)
p
cetz.decorations.zigzag(p, VALUE)
```



amplitude: .25,
segments: 20



amplitude: .8,
segments: 20



amplitude: .4,
segment-length: .2

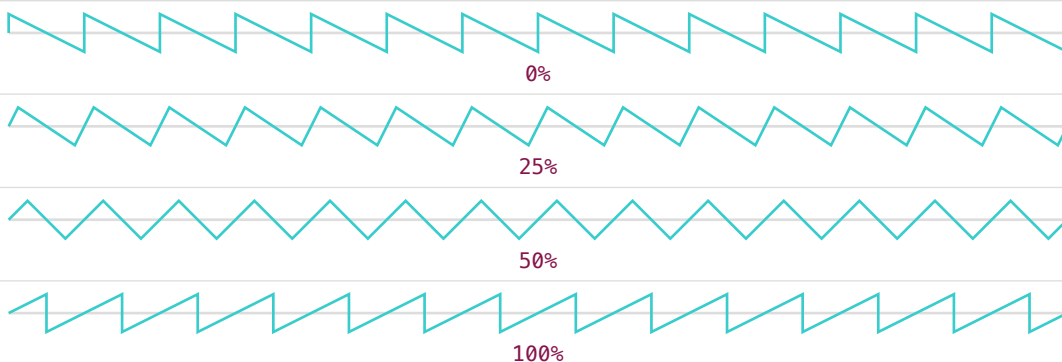


amplitude: .4,
segments: 5

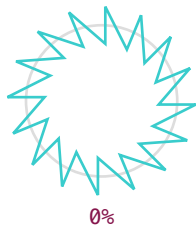
7.1.2 Sawtooth

CeTZ manual: zigzag: factor

```
let p = line((0, 0), (14, 0), stroke: silver)
p
cetz.decorations.zigzag(p, amplitude: .5, segments: 14, factor: VALUE)
```



```
let p = circle((0, 0), stroke: silver)
p
cetz.decorations.zigzag(p, amplitude: .5, segments: 16, factor: VALUE)
```



7.1.3 Wave

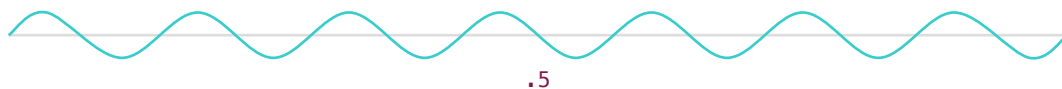
CeTZ manual: wave

```
let p = line((0, 0), (14, 0), stroke: silver)
p
cetz.decorations.wave(p, VALUE)
```

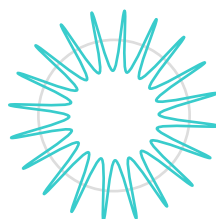
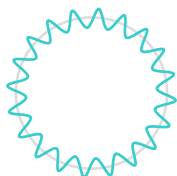


The tension key of wave has no effect in 0.5.2; set it on the catmull root instead.

```
set-style(catmull: (tension: VALUE))
let p = line((0, 0), (14, 0), stroke: silver)
p
cetz.decorations.wave(p, amplitude: .6, segments: 7)
```



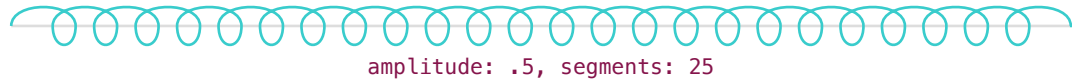
```
let p = circle((0, 0), stroke: silver)
p
cetz.decorations.wave(p, VALUE)
```



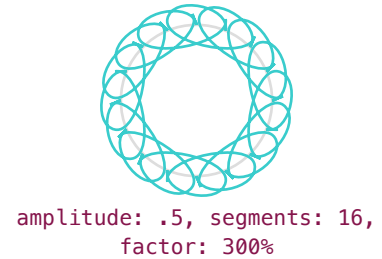
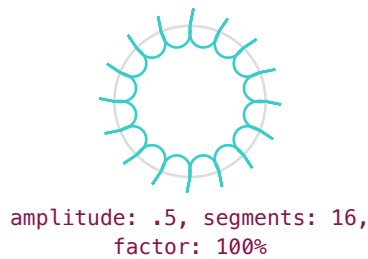
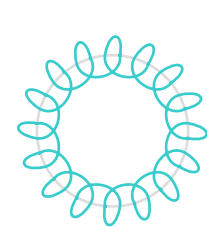
7.1.4 Coil

CeTZ manual: coil

```
let p = line((0, 0), (14, 0), stroke: silver)
p
cetz.decorations.coil(p, VALUE)
```



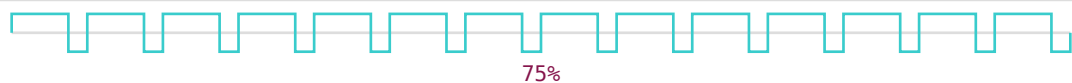
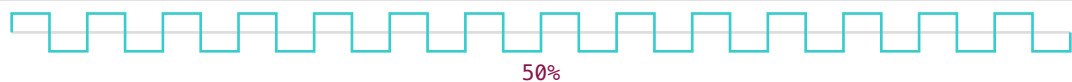
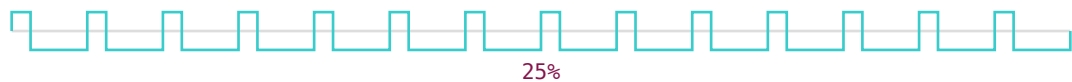
```
let p = circle((0, 0), stroke: silver)
p
cetz.decorations.coil(p, VALUE)
```



7.1.5 Square wave

CeTZ manual: path decorations: square

```
let p = line((0, 0), (14, 0), stroke: silver)
p
cetz.decorations.square(p, amplitude: .5, segments: 14, factor: VALUE)
```



7.1.6 Amplitude per segment

CeTZ manual: path decorations

An array cycles per half segment; a function maps the position (0% to 100%) to an amplitude.

```
let p = line((0, 0), (14, 0), stroke: silver)
p
cetz.decorations.zigzag(p, segments: 28, amplitude: VALUE)
```



<pre>let p = line((0, 0), (14, 0), stroke: silver) p cetz.decorations.zigzag(p, segments: 28, amplitude: VALUE)</pre>	
	<code>(.3, -.1, .5, .2, -.3, .1, .4)</code>
	<code>t => t / 100%</code>
	<code>t => calc.sin(t / 100% * 180deg)</code>

7.1.7 Morphings without a CeTZ decoration

CeTZ manual: bezier

Not in CeTZ: straight zigzag, random steps, bent, bumps, lineto, curveto. Use an amplitude array with zeros for a straight zigzag and for pseudo-random steps, bezier-through for a bent path, a loop of arcs for bumps, and amplitude: 0 to replace a path by its chords. curveto leaves the path unchanged.

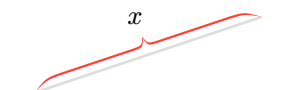
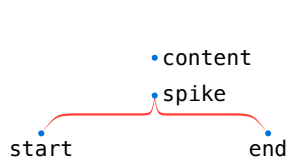
	<code>cetz.decorations.zigzag(line((0, 0), (4, 0)), segments: 20, amplitude: (0,) * 6 + (.4,) * 4, stroke: teal)</code>
	<code>cetz.decorations.zigzag(line((0, 0), (4, 0)), segments: 12, amplitude: (.3, -.1, .4, .2, -.3, .1, .2), stroke: teal)</code>
	<code>bezier-through((0, 0), (2, .5), (4, 0), stroke: teal)</code>
	<pre>for i in range(8) { arc((i / 2, 0), start: 180deg, stop: 0deg, radius: .25, stroke: teal) }</pre>
	<pre>circle((0, 0), stroke: silver) cetz.decorations.zigzag(circle((0, 0)), amplitude: 0, segments: 5, stroke: teal)</pre>

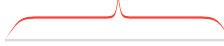
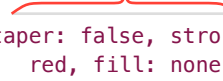
7.2 Braces

CeTZ manual: braces

7.2.1 Brace

CeTZ manual: brace

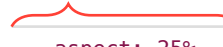
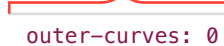
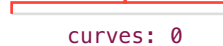
	<pre>line((0, 0), (3, 1), stroke: silver) cetz.decorations.brace((0, 0), (3, 1), name: "b", fill: red) content("b.content", [x\$])</pre>
	<pre>cetz.decorations.brace((0, 0), (3, 0), name: "b", fill: red, amplitude: .5, content-offset: .5) for (a, s) in (start: "north", end: "north", spike: "west", content: "west") { circle("b." + a, radius: .04, fill: blue, stroke: none) content("b." + a, text(7pt, raw(a)), anchor: s, padding: .1) }</pre>

cetz.decorations.brace((0, 0), (3, 0), VALUE)			
			
			

Not in CeTZ: raise and braces along a curved path. A brace spans the straight line between two points; move both points to raise it.

7.2.2 Flat brace

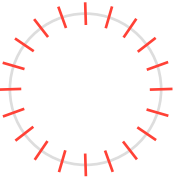
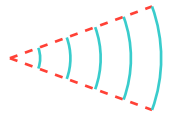
CeTZ manual: flat-brace

cetz.decorations.flat-brace((0, 0), (3, 0), stroke: red, VALUE)			
			
			

7.3 Path replacing

CeTZ manual: interpolation

Not in CeTZ: border, ticks, waves, expanding waves, show path construction. Loop over path anchors and draw each piece; an interpolation coordinate with an angle gives the normal. There is no public API to visit the segments of a path.

	<pre> circle((0, 0), name: "c", stroke: silver) for i in range(20) { let (p, q) = ((name: "c", anchor: i * 5%), (name: "c", anchor: i * 5% + 1%)) line((p, -.15, 90deg, q), (p, .15, 90deg, q), stroke: red) } </pre>
	<pre> circle((0, 0), name: "c", stroke: silver) for i in range(20) { let (p, q) = ((name: "c", anchor: i * 5%), (name: "c", anchor: i * 5% + 1%)) line(p, (p, .4, 45deg, q), stroke: red) } </pre>
	<pre> line((0, 0), (20deg, 2), stroke: (paint: red, dash: "dashed")) line((0, 0), (-20deg, 2), stroke: (paint: red, dash: "dashed")) for r in (.4, .8, 1.2, 1.6, 2) { arc((0, 0), start: -20deg, stop: 20deg, radius: r, anchor: "origin", stroke: teal) } </pre>

7.4 Markings

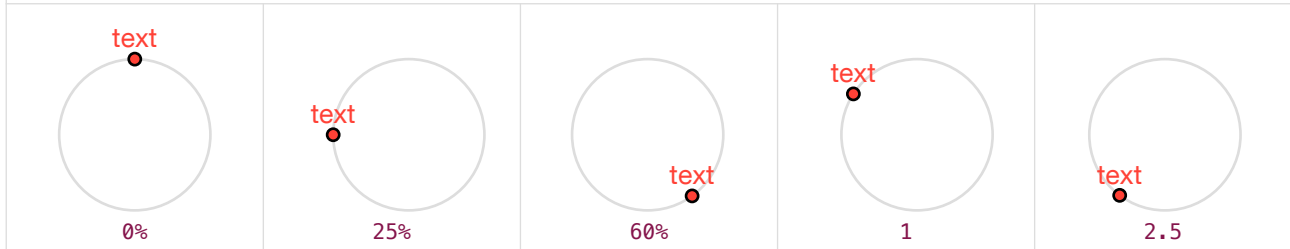
CeTZ manual: path anchors

A path anchor is a ratio of the path length or an absolute distance from its start. A circle starts at its north and runs counter-clockwise.

7.4.1 Mark at one position

CeTZ manual: path anchors

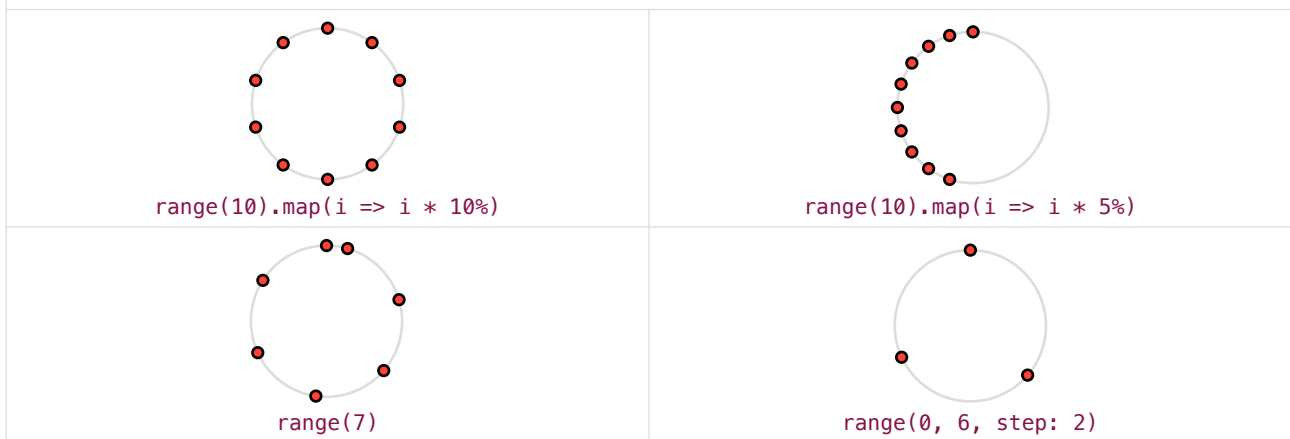
```
circle((0, 0), name: "c", stroke: silver)
circle((name: "c", anchor: VALUE), radius: .08, fill: red)
content((name: "c", anchor: VALUE), text(red)[text],
  anchor: "south", padding: .15)
```



7.4.2 Marks with a step

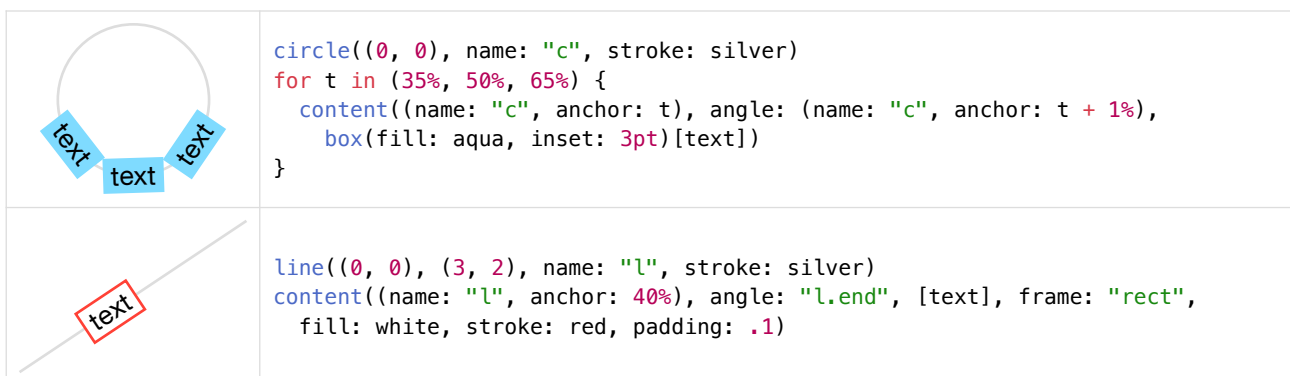
CeTZ manual: path anchors

```
circle((0, 0), name: "c", stroke: silver)
for t in VALUE {
  circle((name: "c", anchor: t), radius: .07, fill: red)
}
```



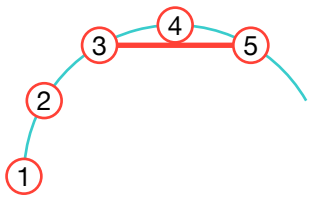
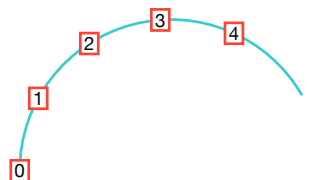
7.4.3 Marks rotated with the path

CeTZ manual: content: angle



7.4.4 Numbered marks

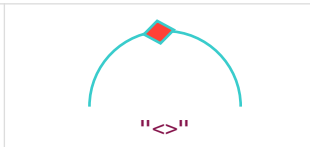
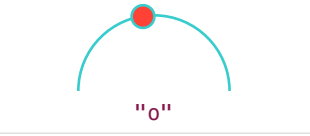
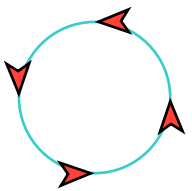
CeTZ manual: content

	<pre>arc((0, 0), start: 180deg, stop: 30deg, radius: 2, name: "a", stroke: teal) for i in range(1, 6) { content((name: "a", anchor: (i - 1) * 20%), [#i], name: "m" + str(i), frame: "circle", fill: white, stroke: red, padding: .05) } line("m3", "m5", stroke: 2pt + red)</pre>
	<pre>arc((0, 0), start: 180deg, stop: 30deg, radius: 2, name: "a", stroke: teal) for d in range(5) { content((name: "a", anchor: d), text(7pt)[#d], frame: "rect", fill: white, stroke: red, padding: .05) }</pre>

7.4.5 Arrow tips on the path

CeTZ manual: marks: pos

pos moves a mark along its path; shorten-to: none keeps the path at full length.

<pre>arc((0, 0), start: 0deg, stop: 180deg, radius: 1, stroke: teal, mark: (end: VALUE, pos: 40%, shorten-to: none, scale: 2, fill: red))</pre>			
			
			
	<pre>circle((0, 0), name: "c", stroke: teal) for t in (0%, 25%, 50%, 75%) { mark((name: "c", anchor: t), (name: "c", anchor: t + 1%), symbol: ">>", scale: 2, fill: red) }</pre>		

7.5 Footprints

CeTZ manual: content

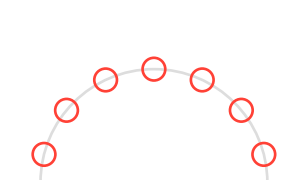
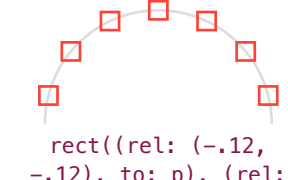
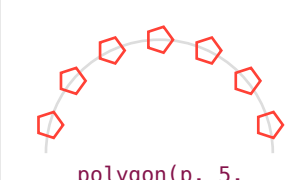
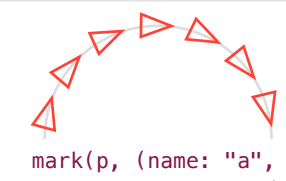
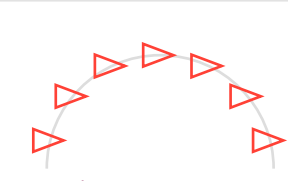
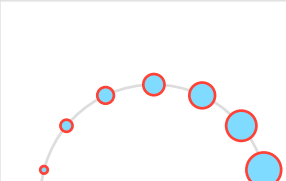
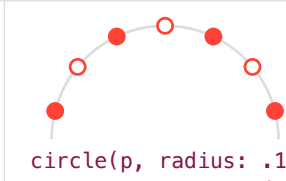
Not in CeTZ: The footprints decoration. Place a symbol or content at path anchors in a loop, as in the markings above.

7.6 Shapes along a path

CeTZ manual: polygon

Not in CeTZ: The shapes decorations (crosses, triangles, shape backgrounds). Place shapes at path anchors in a loop.

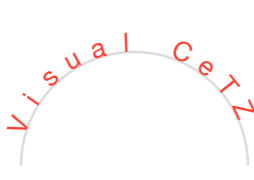
```
arc((0, 0), start: 180deg, stop: 0deg, radius: 1.5, name: "a",
    stroke: silver)
set-style(stroke: red)
for i in range(7) {
    let p = (name: "a", anchor: 8% + i * 14%)
    VALUE
}
```

 circle(p, radius: .15)	 rect((rel: (-.12, -.12), to: p), (rel: (.24, .24)))	 polygon(p, 5, radius: .18)	 n-star(p, 5, radius: .2)
 mark(p, (name: "a", anchor: 9% + i * 14%), symbol: ">", anchor: "center", scale: 2)	 mark(p, 0deg, symbol: ">", anchor: "center", scale: 2)	 circle(p, radius: .05 + i * .03, fill: aqua)	 circle(p, radius: .1, fill: if calc.even(i) { red } else { white })

7.7 Text along a path

CeTZ manual: content: angle

Not in CeTZ: text along path. Place one character per path anchor and rotate it towards the next anchor. The step is fixed, so proportional fonts are spaced unevenly.

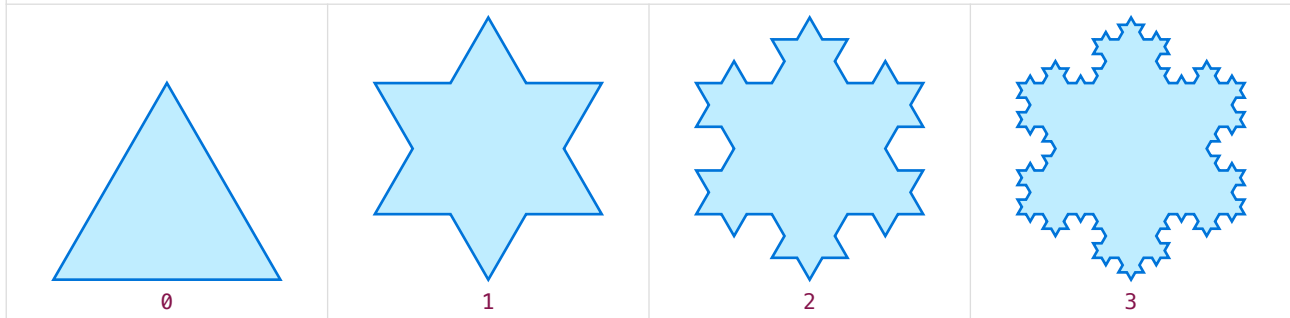
	<pre>arc((0, 0), start: 180deg, stop: 0deg, radius: 1.5, name: "a", stroke: silver) for (i, ch) in "Visual CeTZ".clusters().enumerate() { let d = .5 + i * .35 content((name: "a", anchor: d), angle: (name: "a", anchor: d + .1), text(red, ch), anchor: "south") }</pre>
---	--

7.8 Fractals

CeTZ manual: interpolation

Not in CeTZ: Koch curve type 1, Koch curve type 2, Koch snowflake, Cantor set. Write a recursive function that returns points. Interpolation coordinates keep it short.

```
let koch(a, b, n) = if n == 0 { (a,) } else {
  let (c, e) = ((a, 100% / 3, b), (a, 200% / 3, b))
  let d = (c, 100%, 60deg, e)
  koch(a, c, n - 1) + koch(c, d, n - 1) + koch(d, e, n - 1) + koch(e, b, n - 1)
}
let (a, b, c) = ((0, 0), (1.5, 2.6), (3, 0))
line(..koch(a, b, VALUE), ..koch(b, c, VALUE), ..koch(c, a, VALUE), close: true,
  fill: aqua.lighten(50%), stroke: blue)
```

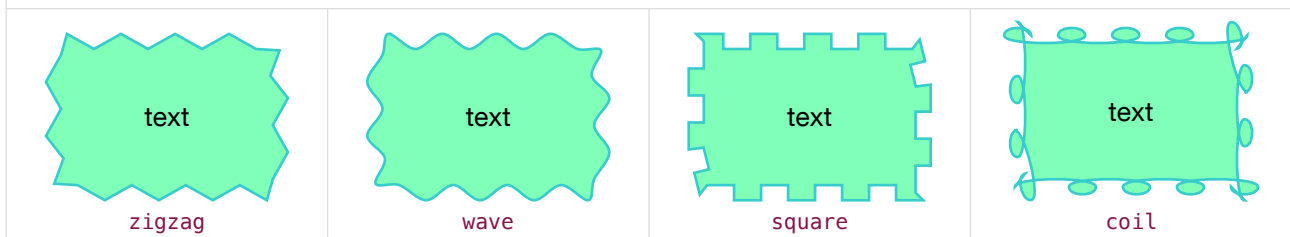


7.9 Applications

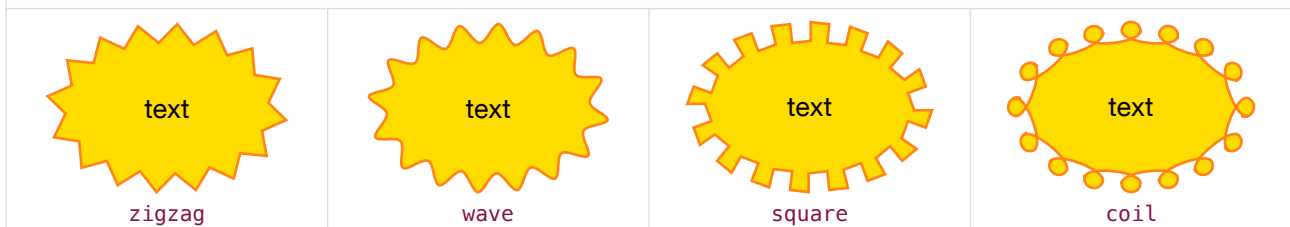
7.9.1 Decorated shapes

CeTZ manual: path decorations

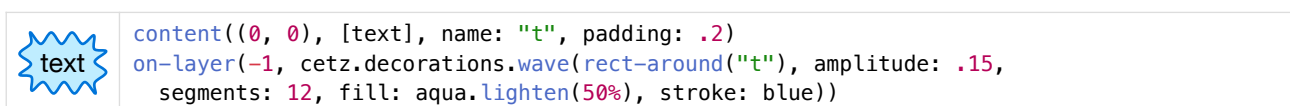
```
cetz.decorations.VALUE(rect((0, 0), (3, 2)), amplitude: .2, segments: 14,
  fill: lime.lighten(50%), stroke: teal)
content((1.5, 1), [text])
```



```
cetz.decorations.VALUE(circle((0, 0), radius: (1.5, 1)), amplitude: .25,
  segments: 16, fill: yellow, stroke: orange)
content((0, 0), [text])
```



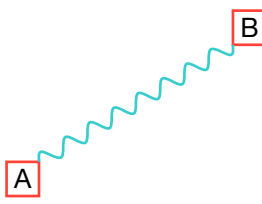
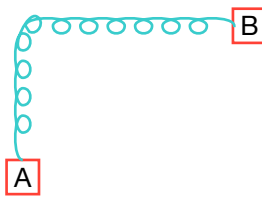
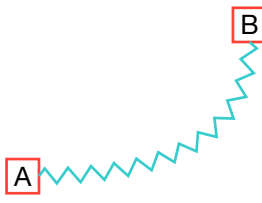
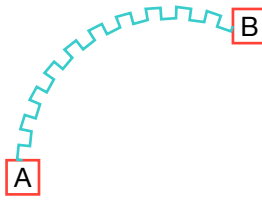
Decorate a node frame with rect-around; on-layer puts it behind the text.



7.9.2 Decorated links

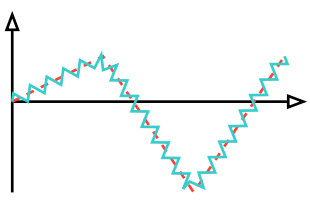
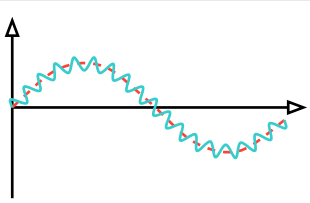
CeTZ manual: path decorations

Here A and B are framed content elements.

	<pre>cetz.decorations.wave(line("A.north-east", "B.south-west"), amplitude: .2, segments: 8)</pre>
	<pre>cetz.decorations.coil(line("A.north", ("A", " -", "B")), "B.west"), amplitude: .2, segments: 12)</pre>
	<pre>cetz.decorations.zigzag(bezier("A.east", "B.south", (3, 0)), amplitude: .2, segments: 12)</pre>
	<pre>cetz.decorations.square(arc-through("A.north", (.8, 1.8), "B.west"), amplitude: .15, segments: 10)</pre>

7.9.3 Decorated plots

CeTZ manual: path decorations

	<pre>line((0, -2), (0, 2), mark: (end: ">")) line((0, 0), (6.5, 0), mark: (end: ">")) let p = line((0, 0), (2, 1), (4, -2), (6, 1), stroke: (paint: red, dash: "dashed")) p cetz.decorations.zigzag(p, amplitude: .3, segment-length: .4, stroke: teal)</pre>
	<pre>line((0, -2), (0, 2), mark: (end: ">")) line((0, 0), (6.5, 0), mark: (end: ">")) let p = catmull(..range(7).map(x => (x, calc.sin(x))), stroke: (paint: red, dash: "dashed")) p cetz.decorations.wave(p, amplitude: .3, segment-length: .4, stroke: teal)</pre>

7.9.4 Partial decoration

CeTZ manual: path decorations: start, stop, rest

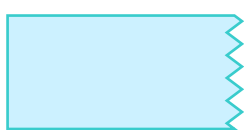
start and stop limit the decoration; rest draws ("LINE") or drops (none) the remaining path.

<pre>cetz.decorations.zigzag(line((0, 0), (14, 0)), amplitude: .4, segment-length: .5, VALUE)</pre>


```
cetz.decorations.zigzag(line((0, 0), (14, 0)), amplitude: .4, segment-length: .5,
VALUE)
```



Decorate only some sides of a closed path by merging plain and decorated parts.



```
merge-path(close: true, fill: aqua.lighten(60%), {
  line((0, 0), (3, 0))
  cetz.decorations.zigzag(line((0, 0), (3, 1.5)), amplitude: .2, segments: 5)
  line((0, 0), (0, 1.5))
})
```



```
let z = cetz.decorations.zigzag.with(amplitude: .2, segments: 8)
merge-path(close: true, fill: aqua.lighten(60%), {
  z(line((0, 0), (3, 0)))
  line((0, 0), (3, 1.5))
  z(line((0, 0), (0, 1.5)))
})
```

Chain decorations on one path by splitting it with start and stop.

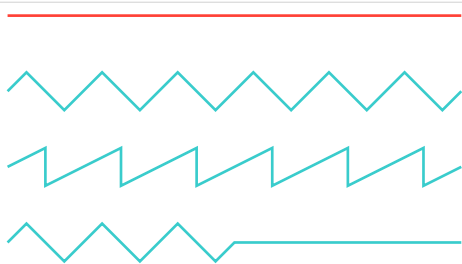


```
let p = line((0, 0), (8, 0))
cetz.decorations.zigzag(p, amplitude: .4, stop: 50%, rest: none)
cetz.decorations.coil(p, amplitude: .4, start: 50%, rest: none)
```

7.9.5 Style for all decorations

CeTZ manual: set-style

Each decoration reads its style from its root key: zigzag, wave, coil, square, brace, flat-brace.



```
set-style(zigzag: (amplitude: .5, segment-length: 1,
  stroke: teal))
line((0, 2), (6, 2), stroke: red)
cetz.decorations.zigzag(line((0, 1), (6, 1)))
cetz.decorations.zigzag(line((0, 0), (6, 0)), factor: 100%)
cetz.decorations.zigzag(line((0, -1), (6, -1)), stop: 50%)
```

7.9.6 Path and its decoration

CeTZ manual: zigzag



```
let p = arc((0, 0), start: 0deg, stop: 180deg,
  radius: (2.5, 1.5), stroke: 10pt + red)
p
cetz.decorations.zigzag(p, amplitude: .8, segments: 12,
  stroke: 2pt + teal)
```

8 Plots

CeTZ has no `plot` command. Sample with Typst arrays, then draw with `line`, `catmull` or `hobby`; place data marks with `mark`. For data plots with real axes, use `lilaq` (Section 8.6).

8.1 Graphs from points and functions

8.1.1 From a list of points

CeTZ manual: line



```
line((0, 0), (1, 1), (2, 0), (3, 1), (4, 1), (5, 2), stroke: blue)
```

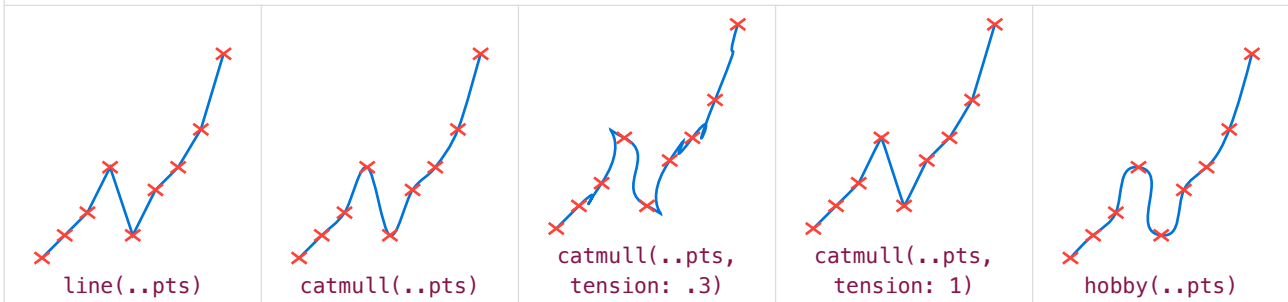
8.1.2 From data

CeTZ manual: catmull

```
// From a file: let pts = csv("table.csv").map(row => row.map(float))
let pts = ((0, .3), (.3, .6), (.6, .9), (.9, 1.5), (1.2, .6), (1.5, 1.2), (1.8, 1.5), (2.1, 2),
(2.4, 3))
set-style(stroke: blue, mark: (anchor: "center", fill: red, stroke: red))
```

VALUE

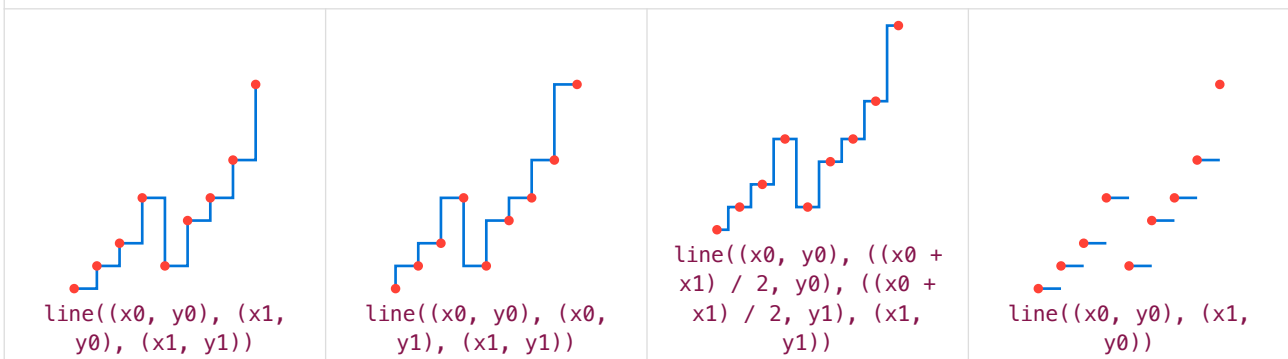
```
for p in pts { mark(p, 0deg, "x") }
```



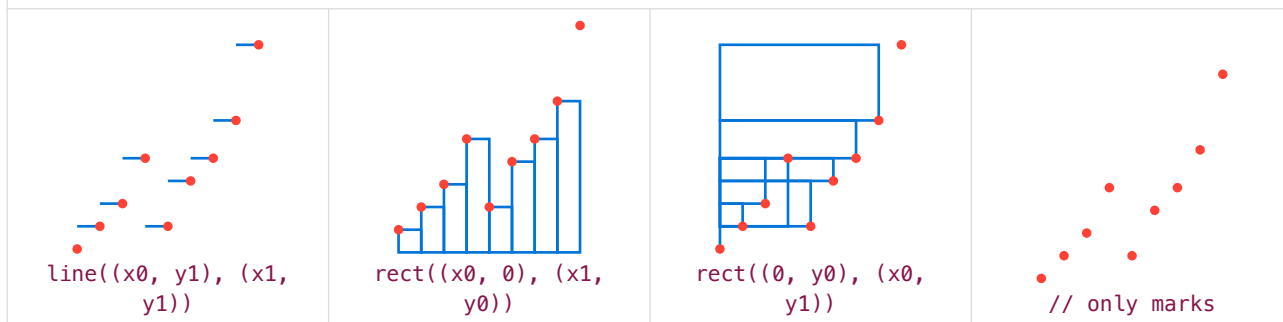
8.1.3 Plot types

CeTZ manual: rect

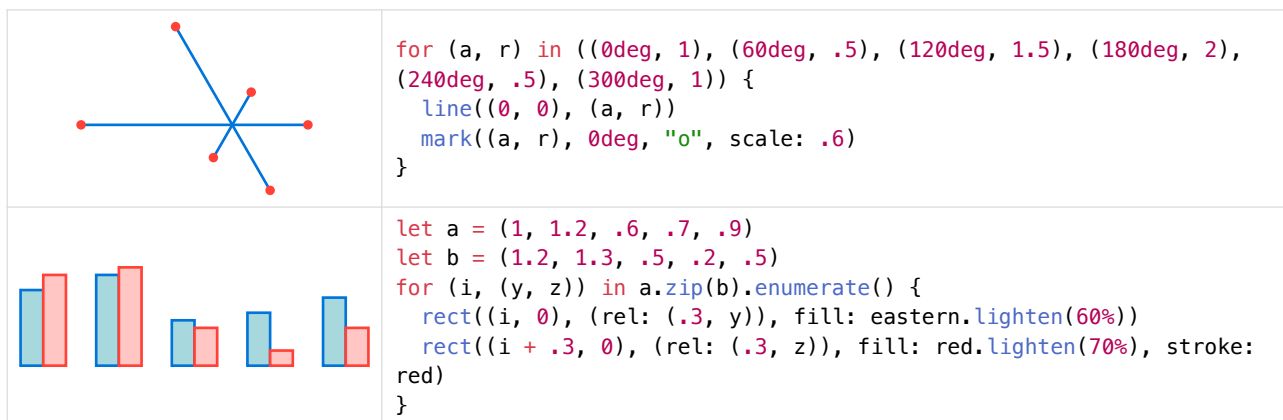
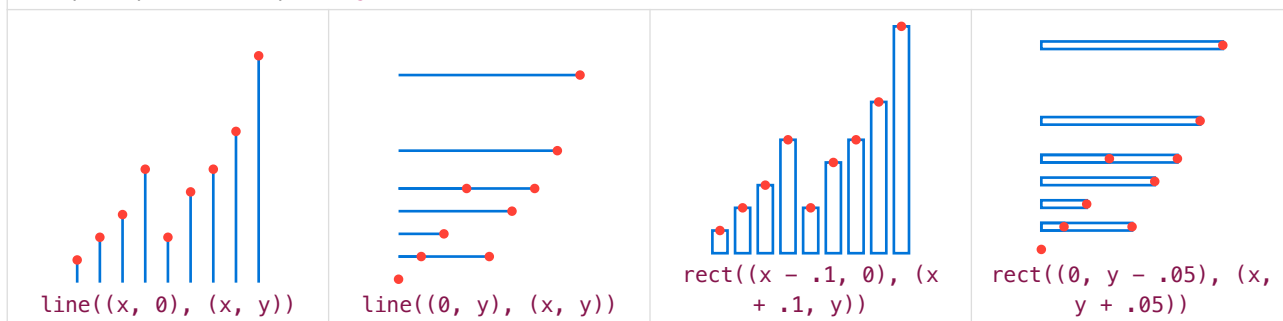
```
for ((x0, y0), (x1, y1)) in pts.windows(2) {
  VALUE
}
for p in pts { mark(p, 0deg, "o", scale: .6) }
```



```
for ((x0, y0), (x1, y1)) in pts.windows(2) {
  VALUE
}
for p in pts { mark(p, 0deg, "o", scale: .6) }
```



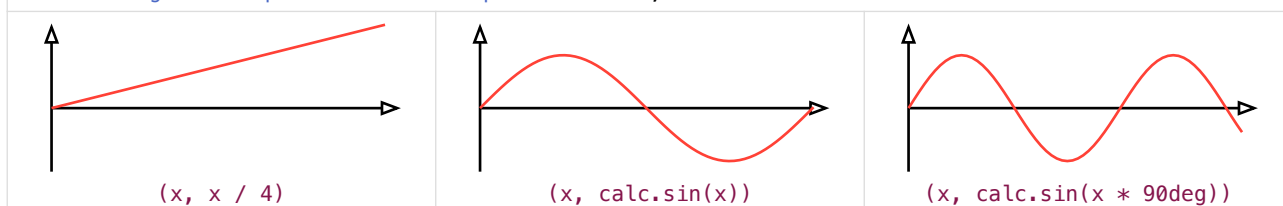
```
for (x, y) in pts {
  VALUE
}
for p in pts { mark(p, 0deg, "o", scale: .6) }
```



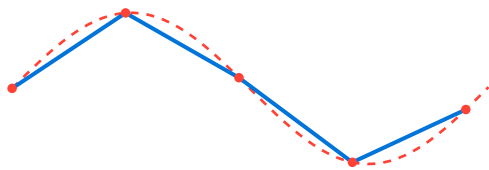
8.1.4 Graph of a function

CeTZ manual: line

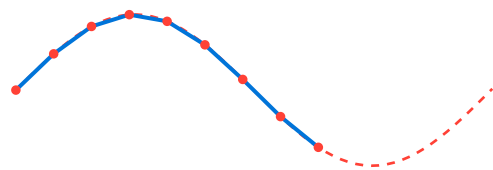
```
line((0, 0), (6.6, 0), mark: (end: ">"))
line((0, -1.2), (0, 1.6), mark: (end: ">"))
line(..range(64).map(i => i / 10).map(x => VALUE), stroke: red)
```



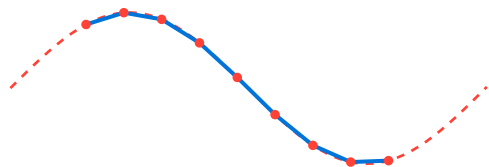
```
let f(x) = calc.sin(x)
line(..range(64).map(i => i / 10).map(x => (x, f(x))), stroke: (paint: red, dash: "dashed"))
let xs = VALUE
line(..xs.map(x => (x, f(x))), stroke: 1.5pt + blue)
for x in xs { mark((x, f(x)), 0deg, "o", scale: .6) }
```



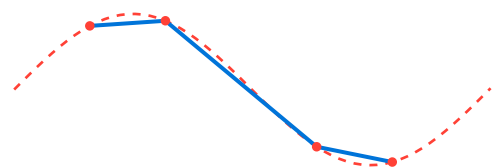
range(5).map(i => 1.5 * i)



range(9).map(i => i / 2)



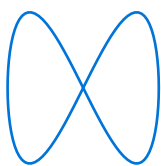
range(9).map(i => 1 + i / 2)



(1, 2, 4, 5)

8.1.5 Parametric curves

CeTZ manual: ortho



```
let ts = range(101).map(i => i / 100 * 2 * calc.pi)
line(..ts.map(t => (calc.sin(t), calc.sin(2 * t))), stroke: 1pt)
```



```
ortho(line(..range(0, 721, step: 10).map(t =>
  (calc.sin(t * 1deg), t / 360, calc.cos(t * 1deg))), stroke: 1pt))
```

8.2 Marks on data points

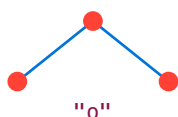
CeTZ manual: marks

mark(pos, angle, symbol) draws one mark; anchor: "center" (set in the setup) centres it on pos. Marks keep their size under scale; circles do not. Arrow tips: Section 2.

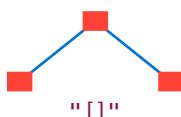
8.2.1 Mark symbols

CeTZ manual: mark

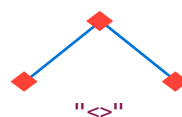
```
line((0, 0), (1, .8), (2, 0))
for p in ((0, 0), (1, .8), (2, 0)) { mark(p, 0deg, VALUE, scale: 1.5) }
```



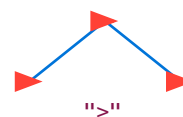
"o"



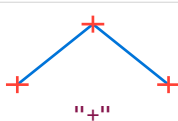
"["



"<"



">"



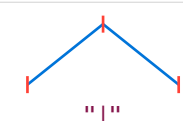
"+"



"x"



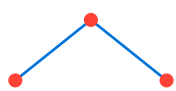
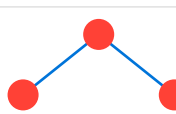
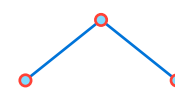
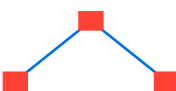
"*"



"|"

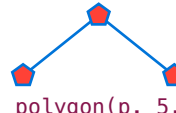
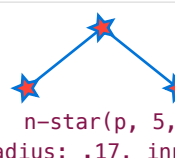
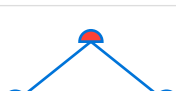
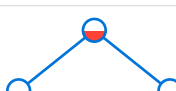
8.2.2 Mark style

CeTZ manual: marks

<pre>line((0, 0), (1, .8), (2, 0)) for p in ((0, 0), (1, .8), (2, 0)) { mark(p, VALUE) }</pre>			
			
0deg, "o"	0deg, "o", fill: none	0deg, "o", scale: 2.5	0deg, "o", fill: aqua
			
0deg, "[", scale: 1.5	90deg, " ", scale: 1.5	0deg, "+", scale: 2	45deg, "+", scale: 2

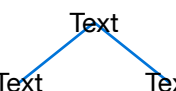
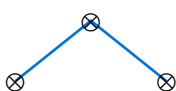
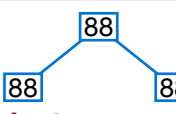
8.2.3 Shapes as marks

CeTZ manual: n-star

<pre>line((0, 0), (1, .8), (2, 0)) for p in ((0, 0), (1, .8), (2, 0)) { VALUE }</pre>			
			
circle(p, radius: .12, fill: red)	polygon(p, 3, radius: .15, angle: 90deg)	polygon(p, 5, radius: .15, angle: 90deg, fill: red)	n-star(p, 5, radius: .17, inner-radius: 45%, fill: red)
			
n-star(p, 10, radius: .17, inner-radius: 40%)	n-star(p, 6, radius: .17, show-inner: true)	n-star(p, 3, radius: .17, inner-radius: 0%, angle: 90deg)	n-star(p, 3, radius: .17, inner-radius: 0%, angle: -90deg)
			
arc(p, start: 0deg, stop: 180deg, radius: .15, mode: "CLOSE", anchor: "origin", fill: red)	circle(p, radius: .15, fill: gradient.linear(white, red, angle: 90deg).sharp(2))	polygon(p, 4, radius: .17, fill: gradient.linear(red, white).sharp(2))	circle(p, radius: .15, stroke: none, fill: gradient.radial(white, blue, focal-center: (30%, 30%)))

8.2.4 Text marks

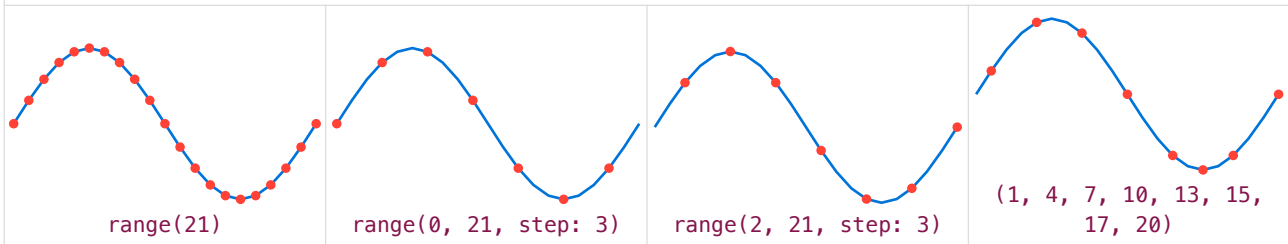
CeTZ manual: content

<pre>line((0, 0), (1, .8), (2, 0)) for p in ((0, 0), (1, .8), (2, 0)) { content(p, VALUE) }</pre>			
			
A [A]	Text [Text]	text(red)[♥]	\$plus.o\$
			
\$times.o\$	emoji.tiger	[88], frame: "rect", padding: .05, fill: white	text(8pt)[#p.at(0)]

8.2.5 Which points get a mark

CeTZ manual: mark

```
let pts = range(21).map(i => (i / 5, calc.sin(i * 18deg)))
line(..pts)
for i in VALUE { mark(pts.at(i), 0deg, "o", scale: .6) }
```



8.3 Graphs from gnuplot

Not in CeTZ: plot function through gnuplot. Typst cannot run programs. Compute samples with `calc`, or run gnuplot (or Python) once and read its table with `csv`.

8.4 Axes in the style of pgfplots

CeTZ has no axis object: scale maps data units to the canvas, a loop writes the tick labels. The setup below extends the one of Section 8.1.2.

```
let f(x) = x * x - x + 4
let axes(xs, ys) = {
  let (x0, x1, y0, y1) = (xs.first(), xs.last(), ys.first(), ys.last())
  rect((x0, y0), (x1, y1), name: "frame", stroke: gray)
  for x in xs { content((x, y0), text(7pt)[#x], anchor: "north", padding: 3pt) }
  for y in ys { content((x0, y), text(7pt)[#y], anchor: "east", padding: 3pt) }
}
```

8.4.1 Axes

CeTZ manual: scale

Not in CeTZ: Logarithmic axes (`semilogxaxis`, `loglogaxis`). Map the data with `calc.log(x)` and label ticks 10^k , or use `lilaq` with `xscale: "log"` (Section 8.6).

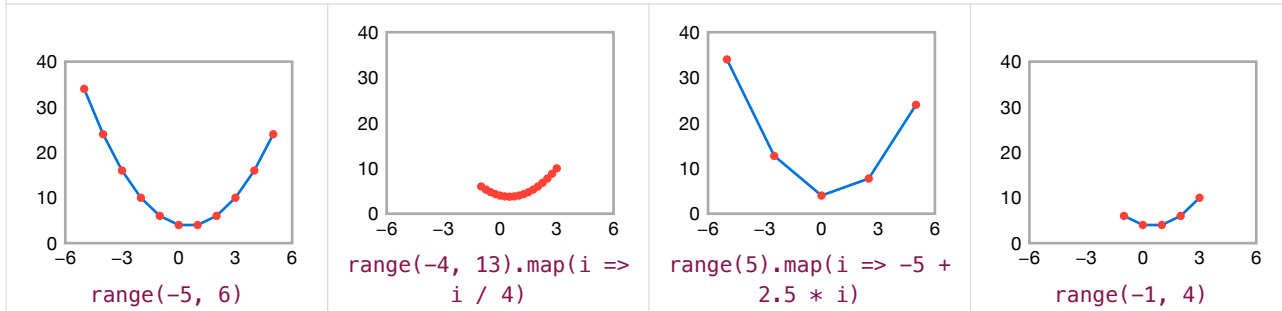


8.4.2 Domain and samples

CeTZ manual: line

Not in CeTZ: Clipping to the axis window (xmin, ymax). Restrict the samples instead: `pts.filter((x, y) => y <= 20)`. lilaq clips to `xlim` and `ylim`.

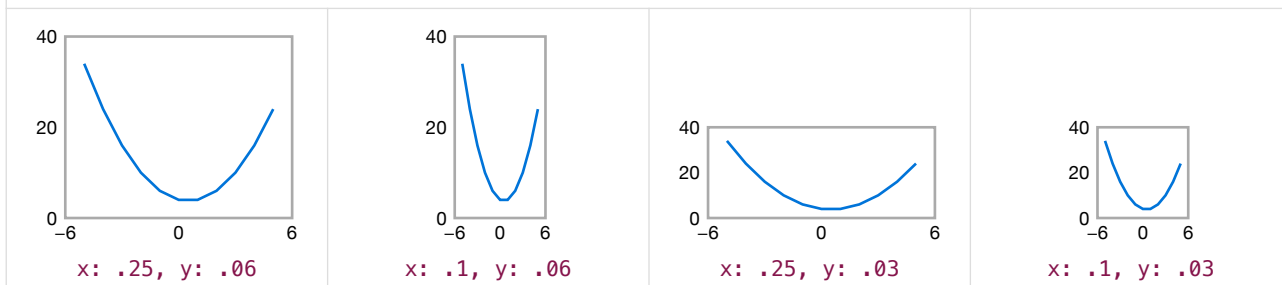
```
scale(x: .25, y: .06)
axes(range(-6, 7, step: 3), range(0, 41, step: 10))
let pts = VALUE.map(x => (x, f(x)))
line(..pts)
for p in pts { mark(p, 0deg, "o", scale: .5) }
```



8.4.3 Units and size

CeTZ manual: scale

```
scale(VALUE)
axes(range(-6, 7, step: 6), range(0, 41, step: 20))
line(..range(-5, 6).map(x => (x, f(x))))
```

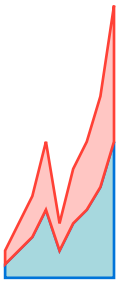
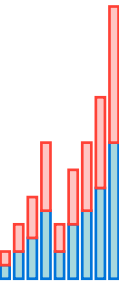


8.4.4 More plot types

CeTZ manual: line

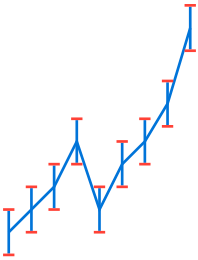
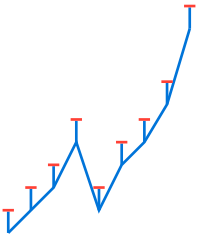
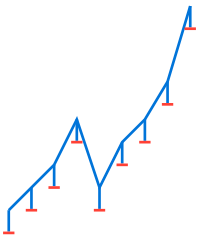
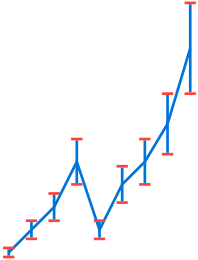
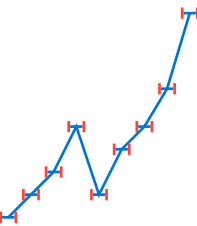
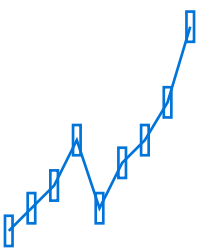
Steps, combs and bars: Section 8.1.3.



	<pre>let top = pts.map((x, y) => (x, 2 * y)) line((0, 0), ..pts, (2.4, 0), close: true, fill: eastern.lighten(60%)) line(..pts, ..top.rev(), close: true, fill: red.lighten(70%), stroke: red)</pre>
	<pre>for (x, y) in pts { rect((x - .1, 0), (x + .1, y), fill: eastern.lighten(60%)) rect((x - .1, y), (x + .1, 2 * y), fill: red.lighten(70%), stroke: red) }</pre>

8.4.5 Error bars

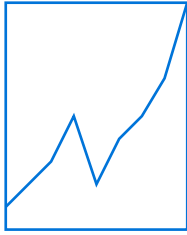
CeTZ manual: marks

<pre>line(..pts) for (x, y) in pts { VALUE }</pre>		
		
<pre>line((x, y - .3), (x, y + .3), mark: (symbol: " "))</pre>	<pre>line((x, y), (x, y + .3), mark: (end: " "))</pre>	<pre>line((x, y - .3), (x, y), mark: (start: " "))</pre>
		
<pre>line((x, .8 * y), (x, 1.2 * y), mark: (symbol: " "))</pre>	<pre>line((x - .1, y), (x + .1, y), mark: (symbol: " "))</pre>	<pre>rect((x - .05, y - .2), (x + .05, y + .2))</pre>

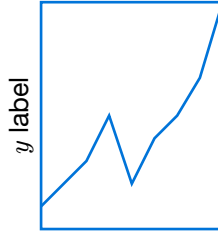
8.4.6 Labels and title

CeTZ manual: anchors

```
rect((0, 0), (2.4, 3), name: "frame")
line(..pts)
VALUE
```



```
content("frame.south", [$x$
label], anchor: "north",
padding: .1)
```



```
content("frame.west", [$y$
label], angle: 90deg, anchor:
"south", padding: .1)
```



```
content("frame.north",
[*Title*], anchor: "south",
padding: .1)
```

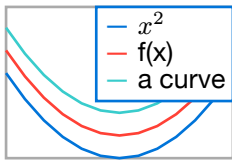
8.4.7 Legend

CeTZ manual: content

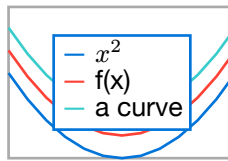
The legend is Typst content: a `std.grid` (grid is CeTZ's) inside a framed content element.

```
rect((0, 0), (3, 2), name: "frame", stroke: gray)
let hues = (blue, red, teal)
for (i, c) in hues.enumerate() {
  line(..range(13).map(k => (k / 4, calc.pow(k / 4 - 1.5, 2) / 2 + .3 * i)), stroke: c)
}
let keys = hues.zip(($x^2$, [f(x)], [a curve])).map(((c, b)) => [#text(c)[-] #b])
```

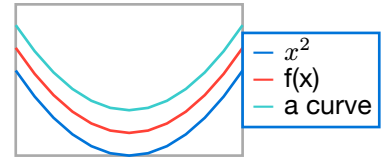
```
content(VALUE, std.grid(gutter: 3pt, ..keys), frame: "rect", fill: white, padding: .1)
```



```
"frame.north-east", anchor:
"north-east"
```

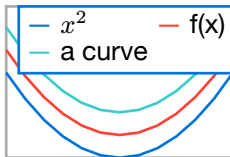


```
"frame.center", anchor:
"center"
```

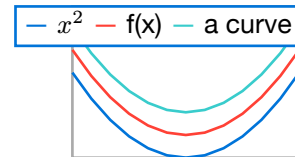


```
"frame.east", anchor: "west"
```

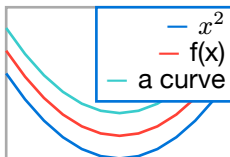
```
content("frame.north-east", VALUE, anchor: "north-east", frame: "rect", fill: white, padding: .1)
```



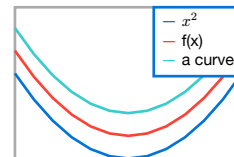
```
std.grid(columns: 2, gutter: 3pt, ..keys)
```



```
std.grid(columns: 3, gutter: 3pt, ..keys)
```



```
std.grid(align: right, gutter: 3pt, ..keys)
```



```
text(6pt, std.grid(gutter: 3pt, ..keys))
```

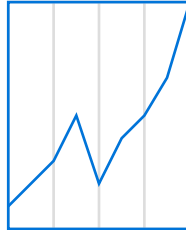
8.4.8 Grids

CeTZ manual: grid

```
VALUE
rect((0, 0), (2.4, 3))
line(..pts)
```



```
grid((0, 0), (2.4, 3),
step: .6, stroke: silver)
```



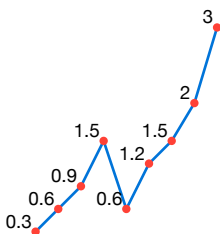
```
grid((0, 0), (2.4, 3), step:
(x: .6, y: 3), stroke: silver)
```



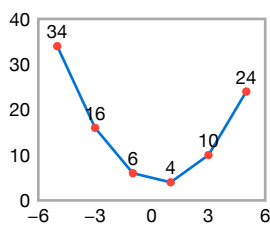
```
grid((0, 0), (2.4, 3), step:
(x: 2.4, y: .5), stroke:
silver)
```

8.4.9 Values at data points

CeTZ manual: content



```
line(..pts)
for (x, y) in pts {
  mark((x, y), 0deg, "o", scale: .5)
  content((x, y), text(7pt)[#y], anchor: "south-east", padding: .05)
}
```



```
scale(x: .25, y: .06)
axes(range(-6, 7, step: 3), range(0, 41, step: 10))
let pts = range(-5, 6, step: 2).map(x => (x, f(x)))
line(..pts)
for (x, y) in pts {
  mark((x, y), 0deg, "o", scale: .5)
  content((x, y), text(7pt)[#y], anchor: "south", padding: 3pt)
}
```

8.5 3D graphs

CeTZ manual: ortho

ortho(x: -60deg, y: 0deg, z: -30deg) puts the z axis up; x: sets the elevation, z: the azimuth. sorted: true (default) draws far faces first.

```

let surface(f, n: 8, map: color.map.viridis, mesh: false, stroke: .3pt) = {
  let g = range(n + 1).map(i => -1 + 2 * i / n)
  let c = gradient.linear(..map)
  for (x0, x1) in g.windows(2) {
    for (y0, y1) in g.windows(2) {
      let q = ((x0, y0), (x1, y0), (x1, y1), (x0, y1)).map((x, y)) => (x, y, f(x, y)))
      let paint = c.sample((q.map(p => p.at(2)).sum() / 4 + 1) * 50%)
      line(..q, close: true, fill: if mesh { none } else { paint },
        stroke: if mesh { paint } else { stroke })
    }
  }
}

```

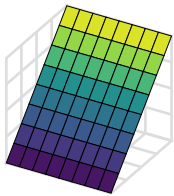
8.5.1 Axes and frame

CeTZ manual: on-xy

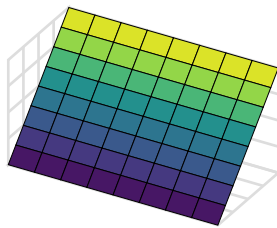
```

ortho(x: -60deg, y: 0deg, z: -30deg, {
  scale(VALUE)
  on-layer(-1, {
    on-xy(z: -1, grid((-1, -1), (1, 1), step: .5, stroke: silver))
    on-xz(y: 1, grid((-1, -1), (1, 1), step: .5, stroke: silver))
    on-zy(x: -1, grid((-1, -1), (1, 1), step: .5, stroke: silver))
  })
  surface((x, y) => y)
})

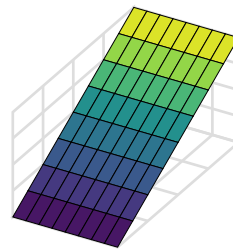
```



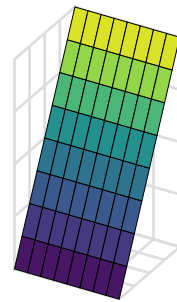
x: 1



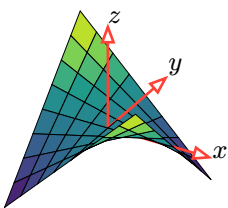
x: 2



y: 2



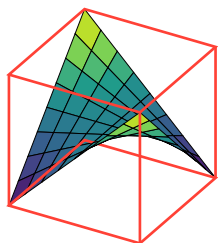
z: 2



```

ortho(x: -60deg, y: 0deg, z: -30deg, {
  surface((x, y) => -x * y)
  for (p, l) in (((1.6, 0, 0), $x$), ((0, 1.6, 0), $y$), ((0, 0, 1.6), $z$)) {
    line((0, 0, 0), p, mark: (end: ">"), stroke: 1pt + red)
    content(p, l, anchor: "south-west")
  }
})

```



```

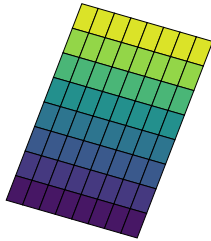
ortho(x: -60deg, y: 0deg, z: -30deg, {
  surface((x, y) => -x * y)
  for z in (-1, 1) { on-xy(z: z, rect((-1, -1), (1, 1), stroke: red)) }
  for (x, y) in ((-1, -1), (1, -1), (1, 1), (-1, 1)) {
    line((x, y, -1), (x, y, 1), stroke: red)
  }
})

```

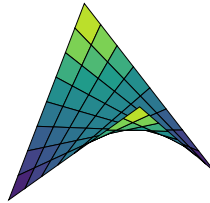
8.5.2 Surfaces and 3D curves

CeTZ manual: ortho

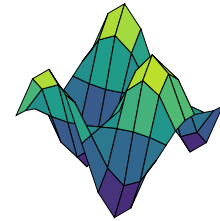
```
ortho(x: -60deg, y: 0deg, z: -30deg, surface(VALUE))
```



$(x, y) \Rightarrow y$

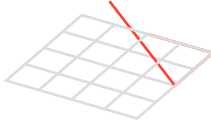


$(x, y) \Rightarrow -x * y$

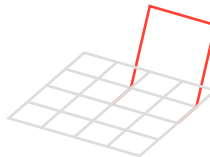


$(x, y) \Rightarrow \text{calc.sin}(3 * x) * \text{calc.cos}(3 * y)$

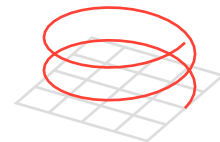
```
ortho(x: -60deg, y: 0deg, z: -30deg, {
  on-xy(grid((-1, -1), (1, 1), step: .5, stroke: silver))
  line(..VALUE, stroke: 1pt + red)
})
```



$((0, 0, 1), (1, 0, 0), (1, 1, 0), (0, 1, 0))$



$((0, 0, 0), (0, .5, 0), (0, 1, 1), (1, 1, 1), (1, .5, 0), (1, 0, 0))$



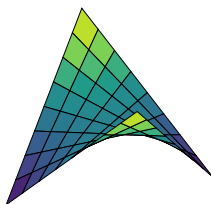
$\text{range}(0, 721, \text{step: } 10).map(t \Rightarrow (\text{calc.cos}(t * 1\text{deg}), \text{calc.sin}(t * 1\text{deg}), t / 720))$

8.5.3 Surface style

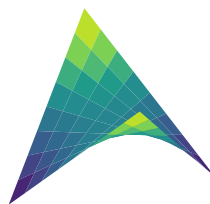
CeTZ manual: line

Not in CeTZ: `shader=interp` (smooth color across a face). Each face has one fill; raise `n` for a smoother look.

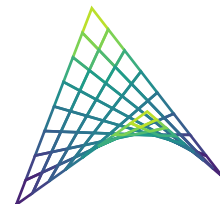
```
ortho(x: -60deg, y: 0deg, z: -30deg, surface((x, y) => -x * y, VALUE))
```



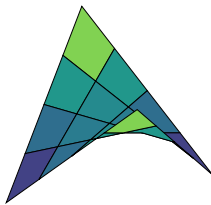
`stroke: .3pt`



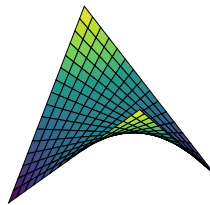
`stroke: none`



`mesh: true`



`n: 4`



`n: 16`



`n: 4, mesh: true`

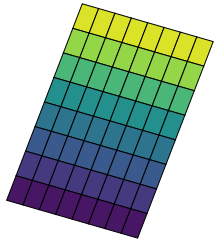


```
ortho(x: -60deg, y: 0deg, z: -30deg, {
  let c = gradient.linear(..color.map.viridis)
  for x in range(-4, 5).map(i => i / 4) {
    for y in range(-4, 5).map(i => i / 4) {
      content((x, y, -x * y), std.circle(radius: 2pt, fill: c.sample((1 - x * y) * 50%)))
    }
  }
})
```

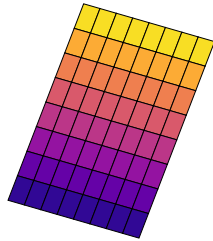
8.5.4 Color maps

CeTZ manual: fill

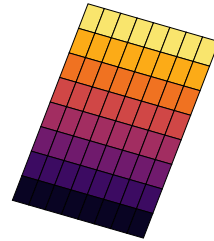
```
ortho(x: -60deg, y: 0deg, z: -30deg, surface((x, y) => y, map: VALUE))
```



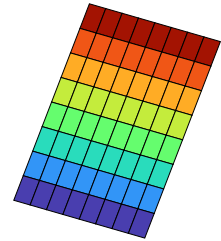
color.map.viridis



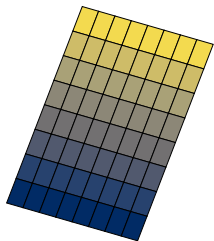
color.map.plasma



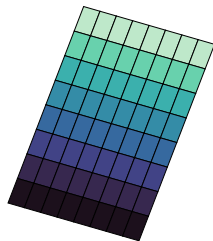
color.map.inferno



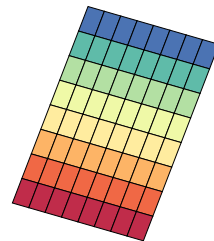
color.map.turbo



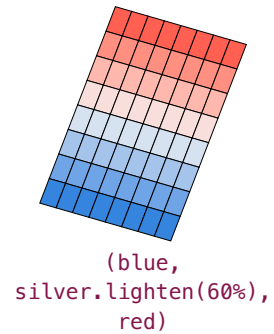
color.map.cividis



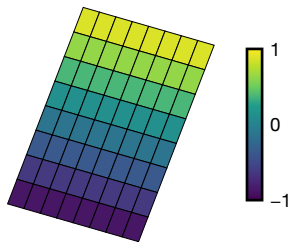
color.map.mako



color.map.spectral



(blue,
silver.lighten(60%),
red)



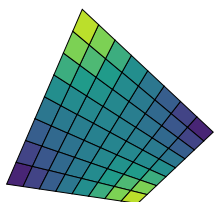
```
ortho(x: -60deg, y: 0deg, z: -30deg, surface((x, y) => y))
rect((1.8, -1), (2, 1), fill: gradient.linear(..color.map.viridis, dir:
btt))
for z in (-1, 0, 1) { content((2, z), text(7pt)[#z], anchor: "west",
padding: 3pt) }
```

8.5.5 Viewpoint

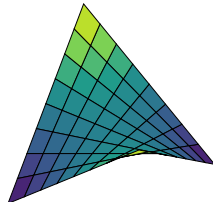
CeTZ manual: ortho

Not in CeTZ: Perspective view. perspective exists in 0.5.2, but a bug discards its projection, so it draws the same picture as ortho. Until it is fixed, project the points yourself: scale x and y by $d/(d - z)$ for a camera at distance d .

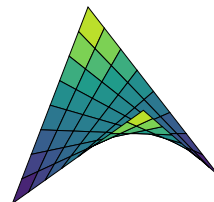
```
ortho(x: VALUE, y: 0deg, z: -30deg, surface((x, y) => -x * y))
```



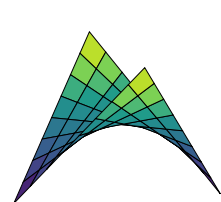
-20deg



-45deg

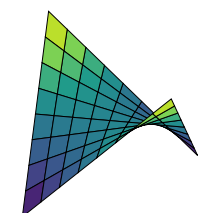


-60deg

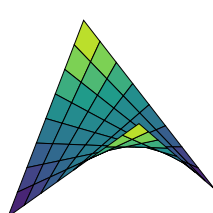


-80deg

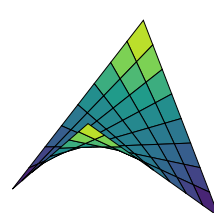
```
ortho(x: -60deg, y: 0deg, z: VALUE, surface((x, y) => -x * y))
```



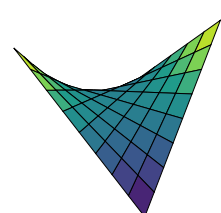
-10deg



-30deg



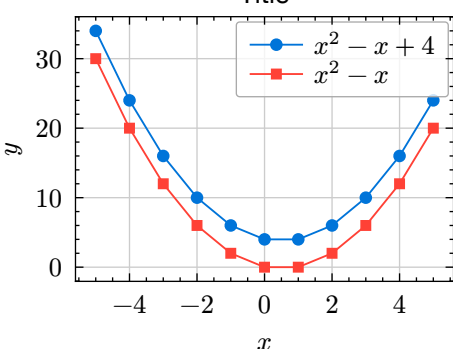
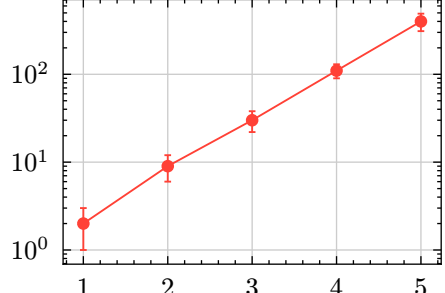
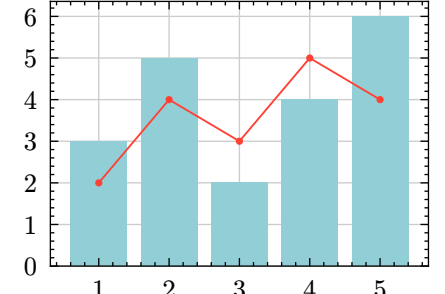
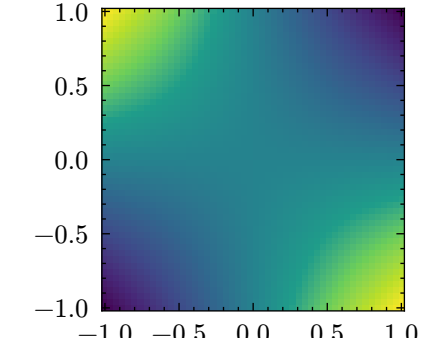
-60deg



-120deg

8.6 Data plots with lilaq

Use lilaq (@preview/lilaq:0.6.0) for plots with axes, ticks, legends and log scales; its diagrams are plain content and sit next to a CeTZ canvas. cetz-plot 0.1.3 pins CeTZ 0.4.2, so its plots cannot mix with 0.5.2 drawings.

	<pre>// #import "@preview/lilaq:0.6.0" as lq lq.diagram(width: 5cm, height: 3.5cm, title: [Title], xlabel: \$x\$, ylabel: \$y\$, lq.plot(range(-5, 6), x => x * x - x + 4, mark: "o", color: blue, label: \$x^2 - x + 4\$), lq.plot(range(-5, 6), x => x * x - x, mark: "s", color: red, label: \$x^2 - x\$),)</pre>
	<pre>lq.diagram(width: 5cm, height: 3.5cm, yscale: "log", lq.plot((1, 2, 3, 4, 5), (2, 9, 30, 110, 400), yerr: (1, 3, 8, 20, 90), mark: "o", color: red),)</pre>
	<pre>lq.diagram(width: 5cm, height: 3.5cm, lq.bar((1, 2, 3, 4, 5), (3, 5, 2, 4, 6), fill: eastern.lighten(50%)), lq.plot((1, 2, 3, 4, 5), (2, 4, 3, 5, 4), color: red),)</pre>
	<pre>lq.diagram(width: 4cm, height: 4cm, lq.colormesh(lq.linspace(-1, 1), lq.linspace(-1, 1), (x, y) => -x * y),)</pre>

9 Loops and trees

9.1 Loops

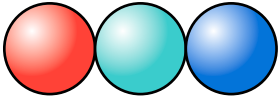
9.1.1 One variable

[Typst manual: loops](#)

	<pre>for x in range(1, 11) { circle((x, 0), radius: .4, fill: eastern) }</pre>
---	--

9.1.2 Two variables

[Typst manual: destructuring](#)

	<pre>for (x, t) in range(1, 11).zip(range(90, -1, step: -10)) { let fill = eastern.lighten(t * 1%) circle((x, 0), radius: .5, fill: fill) }</pre>
	<pre>for (x, c) in ((1, red), (3, teal), (5, blue),) { let ball = gradient.radial(white, c, center: (30%, 30%)) circle((x, 0), fill: ball) }</pre>

9.1.3 Arrays with a step

[Typst manual: range](#)

1,3	2,3	3,3	4,3
1,2	2,2	3,2	4,2
1,1	2,1	3,1	4,1

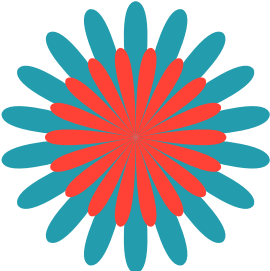
7,3	8,3	9,3	10,3
7,2	8,2	9,2	10,2
7,1	8,1	9,1	10,1

```
for x in (..range(1, 5), ..range(7, 11)) {  
  for y in range(1, 4) {  
    rect((x - .5, y - .5), (rel: (1, 1)),  
      stroke: eastern)  
    content((x, y), [#x,#y])  
  }  
}
```

<code>range(1, 7)</code>	1, 2, 3, 4, 5, 6
<code>range(1, 12, step: 2)</code>	1, 3, 5, 7, 9, 11
<code>range(90, 76, step: -2).map(str.from-unicode)</code>	Z, X, V, T, R, P, N
<code>range(1, 8).map(n => \$2^#n\$)</code>	2 ¹ , 2 ² , 2 ³ , 2 ⁴ , 2 ⁵ , 2 ⁶ , 2 ⁷
<code>range(7).map(i => i / 2)</code>	0, 0.5, 1, 1.5, 2, 2.5, 3
<code>"ABCDEFGH".clusters().map(l => \$#l _1\$)</code>	A ₁ , B ₁ , C ₁ , D ₁ , E ₁ , F ₁ , G ₁ , H ₁

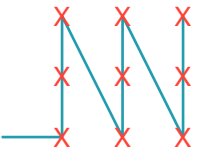
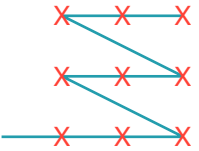
9.1.4 Calculation on the variable

[CeTZ manual: merge-path](#)

	<pre>let petal(a, r, c) = merge-path(fill: c, stroke: none, { bezier((0, 0), (a, r), (a + 10deg, r)) bezier((0, 0), (a, r), (a - 10deg, r)) }) for a in range(10, 370, step: 20) { petal(a * 1deg, 3, eastern) } for a in range(0, 360, step: 20) { petal(a * 1deg, 2, red) }</pre>
---	---

9.1.5 Nested loops

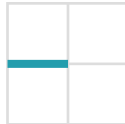
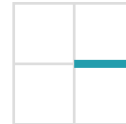
Typst manual: loops

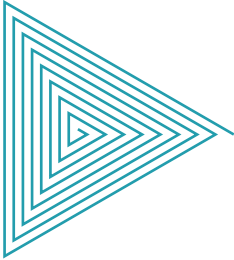
	<pre>let pts = for x in (1, 2, 3) { for y in (0, 1, 2) { ((x, y),) } } line((0, 0), ..pts, stroke: eastern) for p in pts { content(p, text(fill: red)[X]) }</pre>
	<pre>let pts = for y in (0, 1, 2) { for x in (1, 2, 3) { ((x, y),) } } line((0, 0), ..pts, stroke: eastern) for p in pts { content(p, text(fill: red)[X]) }</pre>

9.2 Turtle graphics

CeTZ manual: relative coordinates

Not in CeTZ: the turtle library. Chain relative polar steps (rel: (angle, length)) in one line; a loop turns the heading. For bent steps (how), draw each step with arc-through.

<code>line((0, 0), (rel: VALUE), stroke: 3pt + eastern)</code>				
				
(90deg, 1)	(135deg, 1)	(180deg, 1)	(45deg, 1)	(0deg, 1)

	<pre>let h = 90deg let steps = () for (turn, d) in ((-90deg, 2), (90deg, 1), (90deg, 1)) { h += turn steps.push((rel: (h, d))) } line((0, 0), ..steps, stroke: 3pt + eastern, mark: (end: ">"))</pre>
	<pre>line((0, 0), ..range(5).map(i => (rel: (-i * 144deg, 2))), close: true, stroke: eastern, fill: red.lighten(80%))</pre>
	<pre>line((0, 0), ..range(1, 26).map(i => (rel: (90deg - i * 120deg, i / 5))), stroke: eastern)</pre>

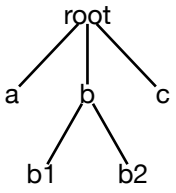
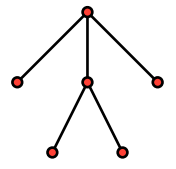
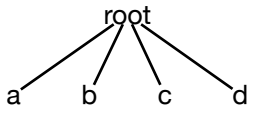
9.3 Trees

CeTZ manual: tree library

Examples below run after `import cetz.tree` and share `let family = ([root], [a], ([b], [b1], [b2]), [c])`.

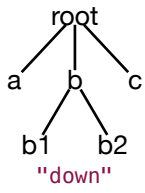
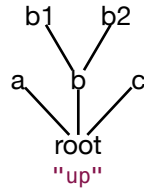
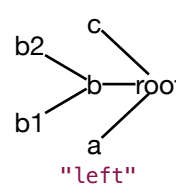
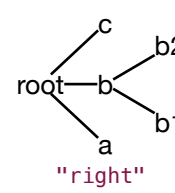
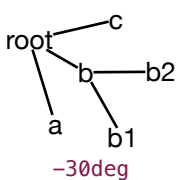
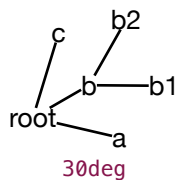
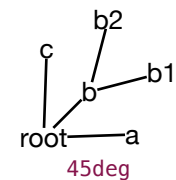
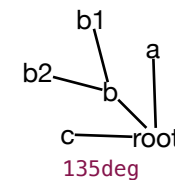
9.3.1 Structure

CeTZ manual: tree

	<pre>tree.tree([root], [a], ([b], [b1], [b2]), [c]))</pre>
	<pre>tree.tree((none, none, (none, none, none), none), draw-node: _ => circle((0, 0), radius: .08, fill: red))</pre>
	<pre>tree.tree([root], ..("a", "b", "c", "d"))</pre>

9.3.2 Direction

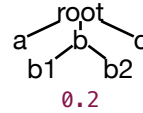
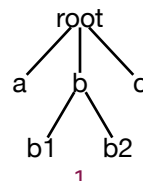
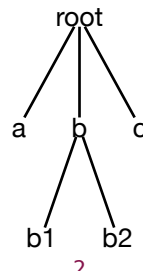
CeTZ manual: direction

<pre>tree.tree(family, direction: VALUE)</pre>			
 "down"	 "up"	 "left"	 "right"
<pre>rotate(VALUE) tree.tree(family, direction: "right")</pre>			
 -30deg	 30deg	 45deg	 135deg

Not in CeTZ: per-child grow and the mirrored grow'. Draw an odd child outside the tree and line it to the parent's anchor; reverse a child array to mirror its order.

9.3.3 Level distance

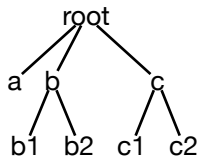
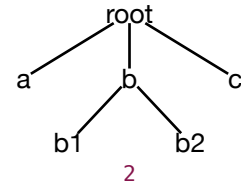
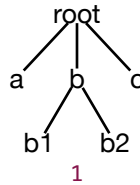
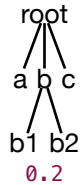
CeTZ manual: grow

<pre>tree.tree(family, grow: VALUE)</pre>		
 0.2	 1	 2

9.3.4 Sibling distance

CeTZ manual: spread

```
tree.tree(family, spread: VALUE)
```



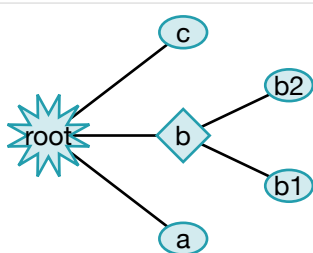
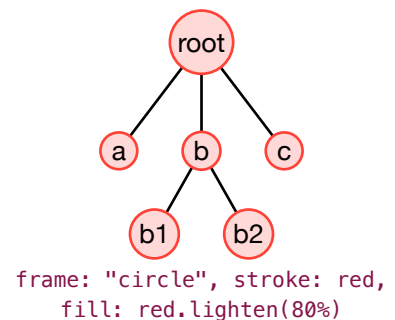
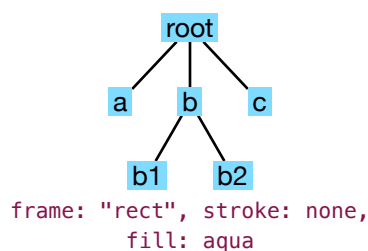
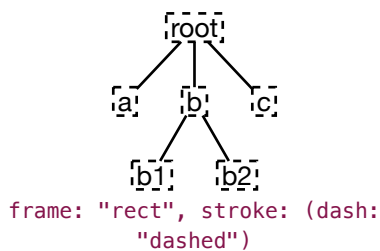
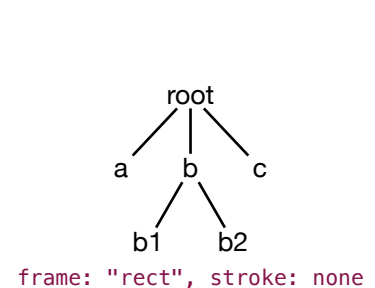
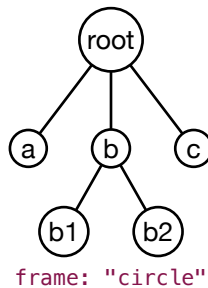
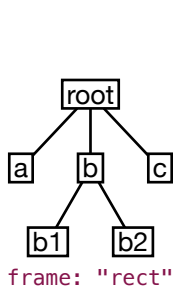
```
tree.tree([root], [a], ([b], [b1], [b2]), ([c], [c1], [c2])), spread: .5)
```

Not in CeTZ: level distance and sibling distance per level or per child. grow and spread apply to the whole tree. The layout packs subtrees tidily, so they never overlap.

9.3.5 Node shapes

CeTZ manual: draw-node

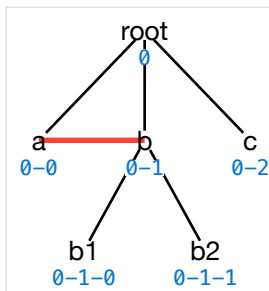
```
tree.tree(family, draw-node: n => content((0, 0), n.content, padding: .1, VALUE))
```



```
tree.tree(family, direction: "right", draw-node: n => {
  set-style(fill: eastern.lighten(80%), stroke: eastern)
  if n.depth == 0 { n-star((0, 0), 12, radius: .8) }
  else if n.children == () { circle((0, 0), radius: (.45, .3)) }
  else { polygon((0, 0), 4, radius: .5) }
  content((0, 0), n.content)
})
```

9.3.6 Node names

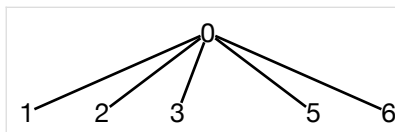
CeTZ manual: name



```
tree.tree(family, name: "t", grow: 1.5, spread: 1.5)
for a in ("0", "0-0", "0-1", "0-1-0", "0-1-1", "0-2") {
  content((rel: (0, -.45), to: "t." + a), text(fill: blue, raw(a)),
    frame: "rect", stroke: none, fill: white)
}
line("t.g0-0", "t.g0-1", stroke: 2pt + red)
```

9.3.7 Missing nodes

CeTZ manual: draw-edge

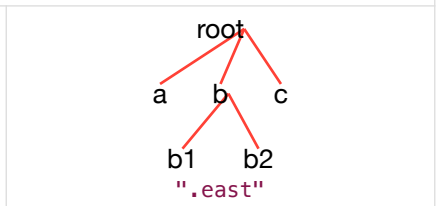
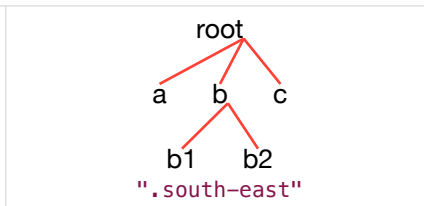
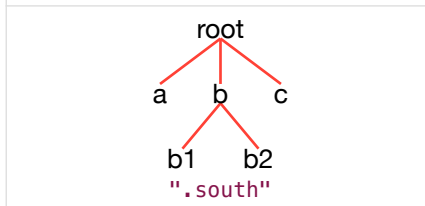


```
tree.tree([0], [1], [2], [3], none, [5], [6]),
draw-edge: (p, c) => if c.content != none {
  line(p.group-name, c.group-name)
})
```

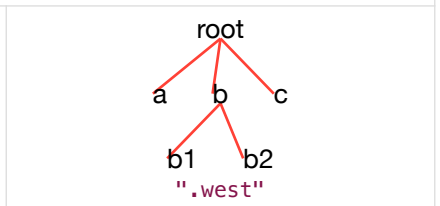
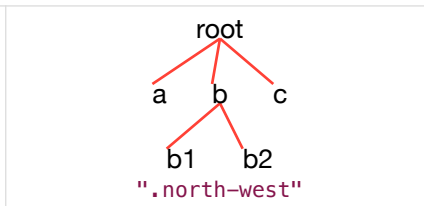
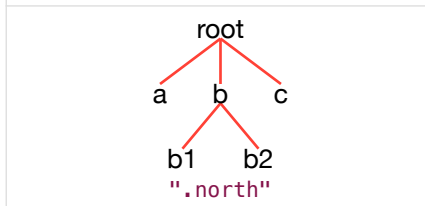
9.3.8 Edge anchors

CeTZ manual: draw-edge

```
tree.tree(family, draw-edge: (p, c) =>
  line(p.group-name + VALUE, c.group-name + ".north", stroke: red))
```

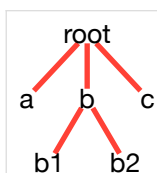


```
tree.tree(family, draw-edge: (p, c) =>
  line(p.group-name + ".south", c.group-name + VALUE, stroke: red))
```

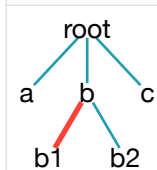


9.3.9 Edge style

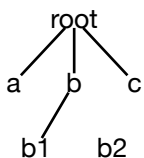
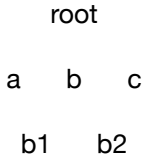
CeTZ manual: draw-edge



```
tree.tree(family, draw-edge: (p, c) =>
  line(p.group-name, c.group-name, stroke: 2pt + red))
```

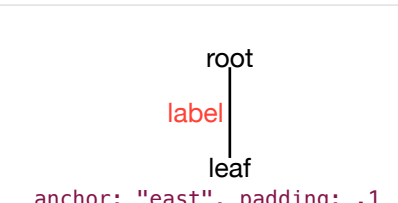
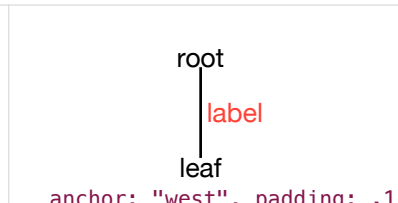
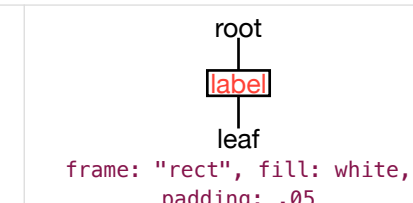
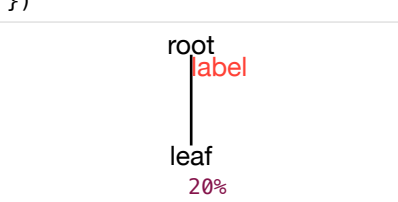
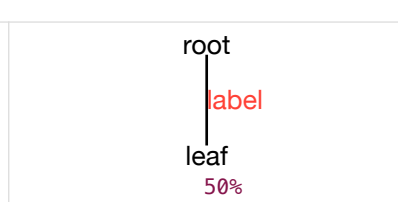
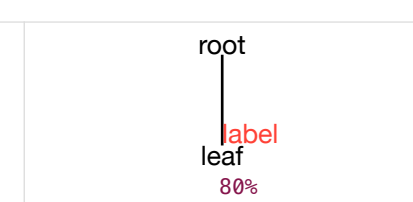


```
tree.tree(family, draw-edge: (p, c) => {
  let s = if c.name == "0-1-0" { 2pt + red } else { eastern }
  line(p.group-name, c.group-name, stroke: s)
})
```

	<pre>tree.tree(family, draw-edge: (p, c) => { if c.name != "0-1-1" { line(p.group-name, c.group-name) } })</pre>
	<pre>tree.tree(family, draw-edge: none)</pre>

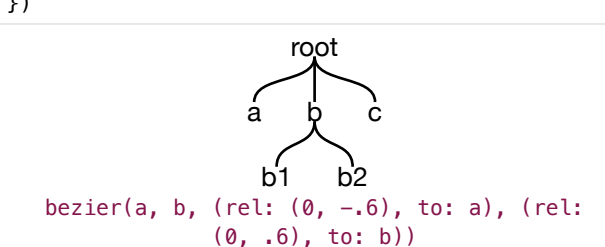
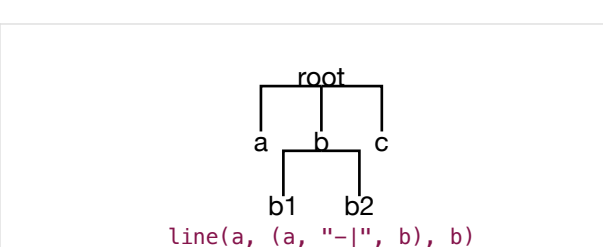
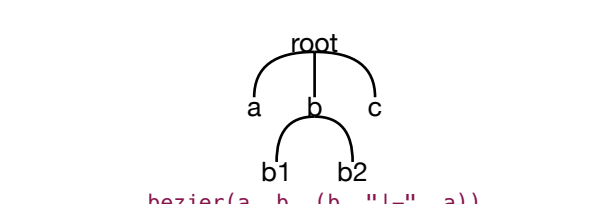
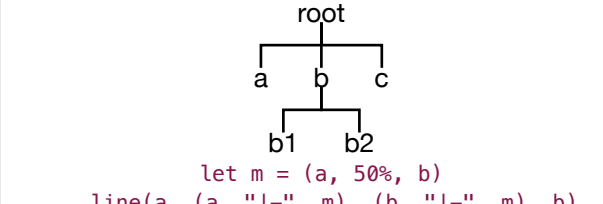
9.3.10 Edge labels

CeTZ manual: content

<pre>tree.tree([root], [leaf]), grow: 1.5, draw-edge: (p, c) => { line(p.group-name, c.group-name) content((p.group-name, 50%, c.group-name), text(fill: red)[label], VALUE) })</pre>		
 anchor: "east", padding: .1	 anchor: "west", padding: .1	 frame: "rect", fill: white, padding: .05
<pre>tree.tree([root], [leaf]), grow: 1.5, draw-edge: (p, c) => { line(p.group-name, c.group-name) content((p.group-name, VALUE, c.group-name), text(fill: red)[label], anchor: "west") })</pre>		
 20%	 50%	 80%

9.3.11 Edge paths

CeTZ manual: perpendicular

<pre>tree.tree(family, draw-edge: (p, c) => { let (a, b) = (p.group-name + ".south", c.group-name + ".north") VALUE })</pre>	
 bezier(a, b, (rel: (0, -.6), to: a), (rel: (0, .6), to: b))	 line(a, (a, "- ", b), b)
 bezier(a, b, (b, "- ", a))	 let m = (a, 50%, b) line(a, (a, "- ", m), (b, "- ", m), b)

	<pre>tree.tree(family, direction: "right", draw-edge: (p, c) => { let (a, b) = (p.group-name + ".east", c.group-name + ".west") let m = (a, 50%, b) line(a, (m, " -", a), (m, " -", b), b, stroke: blue) })</pre>
--	--

9.3.12 Radial and custom child positions

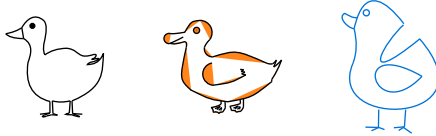
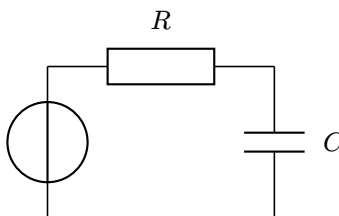
Not in CeTZ: grow cyclic, sibling angle, clockwise from and grow via three points. Place the children in a loop, at polar coordinates (angle, radius) or at linear steps ($i * dx, dy$).

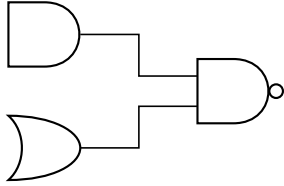
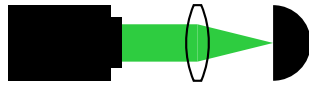
	<pre>content((0, 0), [root], name: "r", padding: .1) for a in (30, 0, -30, -60) { content((a * 1deg, 1.6), \$#a\$, name: "c") line("r", "c", stroke: eastern) }</pre>
--	---

10 Not ported

Visual TikZ sections without a CeTZ counterpart. Single options that CeTZ lacks are marked *Not in CeTZ* in their own chapters.

Visual TikZ	Instead	Example
7.14–7.16 Matrices, chains	Loops that place and name elements; a Typst table or mat inside one content. Section 4	
11 Absolute position on a page	Typst page(background:) or foreground: with place; <code>pinit</code> to point at text. Section 5	<pre>#import "@preview/cetz:0.5.2": canvas, draw #set page(background: place(top + right, dx: -1cm, dy: 1cm, canvas({ draw.circle((0, 0), fill: red) })))</pre>
14 Blend modes, fadings, transparency groups	Transparent paints and gradient stops; <code>color.mix</code> for a blended colour. Section 5	
17.7 Multi-part shapes	A Typst table inside content, or a shape split with a line. Section 6	
18.4–18.7 Footprint, shape, text and fractal decorations	Loops over path anchors; recursive point functions. Section 7	
20 Freehand drawing	hobby or catmull through jittered points (Section 5); <code>scrawl</code> for a sketched look	 <pre>#import "@preview/scrawl:0.1.0": * #scrawl-ellipse(paint: red)[freehand] #scrawl-box(paint: blue)[box]</pre>
21.1 tikzpeople	<code>pixel-family</code>	

Visual TikZ	Instead	Example																				
		<pre>#import "@preview/pixel-family:0.2.1": * #alice(size: 1.2cm) #bob(size: 1.2cm, shirt: aqua)</pre>																				
21.2 tikzducks	<code>figchild: 561 CeTZ figures; emoji.duck</code>	 <pre>#import "@preview/figchild:0.1.0": * #stack(dir: ltr, spacing: 5mm, canvas(fc-duck(scale: .4)), canvas(fc-duck-a(scale: .4, fill: orange)), canvas(fc-duck-b(scale: .4, stroke: blue)))</pre>																				
21.3 tikzsymbols	Built-in emoji and sym modules	 <pre>#text(20pt)[#emoji.face.smile #emoji.cooking #emoji.teapot #emoji.knife #sym.suit.heart]</pre>																				
22.3 gnuplot	Sample with calc, or read precomputed data with csv. Section 8																					
25 Variation tables (tkz-tab)	<code>functable</code>	<table border="1"><tr><td colspan="2">x</td><td>$-\infty$</td><td>0</td><td>$+\infty$</td></tr><tr><td>fact.</td><td>$2x$</td><td>$-$</td><td>0</td><td>$+$</td></tr><tr><td></td><td>$f'(x)$</td><td>$-$</td><td>0</td><td>$+$</td></tr><tr><td colspan="2">x^2</td><td>$+\infty$</td><td>0</td><td>$+\infty$</td></tr></table> <pre>#import "@preview/functable:0.2.0": sign-table #sign-table(factors: ((expr: \$2x\$, fn: x => 2 * x, zeros: ((value: \$0\$, approx: 0),)),), summary-label: \$f'(x)\$, variation: true, variation-label: \$x^2\$, start-value: \$+oo\$, end-value: \$+oo\$, variation-values: ((at: 0, label: \$0\$)),)</pre>	x		$-\infty$	0	$+\infty$	fact.	$2x$	$-$	0	$+$		$f'(x)$	$-$	0	$+$	x^2		$+\infty$	0	$+\infty$
x		$-\infty$	0	$+\infty$																		
fact.	$2x$	$-$	0	$+$																		
	$f'(x)$	$-$	0	$+$																		
x^2		$+\infty$	0	$+\infty$																		
27 Turtle graphics	Relative polar steps in one line. Section 9																					
29 Electrical circuits	<code>zap, built on CeTZ 0.5.2</code>	 <pre>#import "@preview/zap:0.6.0" #zap.circuit({ import zap: * vsource("v", (0, 0), (0, 2)) resistor("r", (0, 2), (3, 2), label: \$R\$) capacitor("c", (3, 2), (3, 0), label: \$C\$) wire((3, 0), (0, 0)) })</pre>																				

Visual TikZ	Instead	Example
30 Logic circuits	<code>zap</code> gates, IEC or IEEE; <code>circuiteria</code> (pins CeTZ 0.3.4) for block circuits	 <pre> #import "@preview/zap:0.6.0" #zap.circuit({ import zap: * cetz.draw.set-style(zap: (variant: "ieee")) land("a", (0, 0)) lor("b", (0, -1.5)) lnand("c", (2.5, -.75)) zwire("a.out", "c.in1") zwire("b.out", "c.in2") }) </pre>
31 Optics	<code>beam</code> (pins CeTZ 0.4.2), <code>laserly</code>	 <pre> #import "@preview/beam:0.1.1" #beam.setup({ import beam: * laser("l", (0, 0)) lens("f", (1, 0)) detector("d", (2, 0)) beam("", "l", "f") focus("", "f", "d") }) </pre>
32 Animation (animate)	One page per frame, exported as PNG images; <code>touying</code> or <code>polylux</code> for slide overlays; <code>kino</code>	<pre> // typst compile --format png anim.typ 'f-{0p}.png' #import "@preview/cetz:0.5.2": canvas, draw #set page(width: 3cm, height: 3cm, margin: 0pt) #set align(center + horizon) #for t in range(24) { page(canvas({ draw.rotate(t * 15deg) draw.rect((-1, -1), (1, 1)) }))) } </pre>